Experiments on the U-Net Bottleneck in Flow Matching

An Empirical Deep Learning Study

Maturaarbeit · Kantonsschule Freudenberg · October 2026

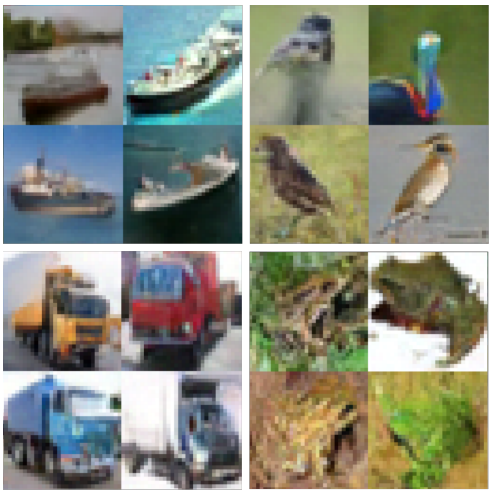


Figure 1: Generated CIFAR-10 samples from various models in this project.

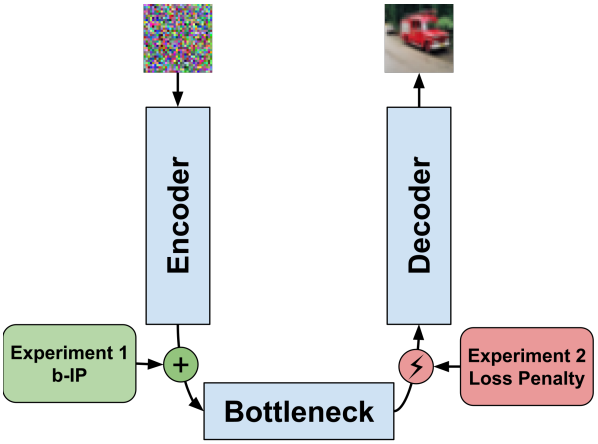


Figure 2: Scheme of the two experiments performed on the U-Net. Note that the U-Net does not directly output final generated image in practice.

Abstract

U-Net architectures are widely used as the neural networks underlying modern image-generation models. Though commonly used, their internal mechanics are not fully understood yet. This project investigates the impact of interfering with the bottleneck, the lowest-resolution stage of the U-Net, and the resulting changes in selected metrics and image quality. I use a Flow matching training formulation, which is a state-of-the-art approach for training generative models.

On class-conditional CIFAR-10 with a 10.85M-parameter U-Net, I evaluate two interventions: (1) Bottleneck Input Perturbation (b-IP), Gaussian noise injected into the pre-bottleneck activations using a custom time-dependent, standard-deviation-weighted schedule; and (2) a KL-divergence to unit-Gaussian loss penalty on the post-bottleneck activations.

Both improve FID-50k in this setting: an 8.33% improvement for b-IP ($\gamma_0 = 0.2$) and a 6.48% improvement for the KL penalty ($\beta = 0.5$) relative to the base model, with every seed of each best configuration outperforming every base-model seed, whereas standard network-input perturbation does not improve FID.

The experimental results do not fully support the hypothesized mechanisms to explain the gain: neither intervention measurably reduces a proxy metric for exposure bias, bottleneck noise does not consistently change the model's partial derivatives, and the KL penalty does not enlarge the bottleneck's effective receptive field. Consequently, the cause of the FID gains remains an open question that I narrowed down using selected metrics.

The results come from mini-scale models trained for 400 epochs with a constant learning rate and are mostly single-seeded. Because experiments were restricted to the network input and the bottleneck, they do not establish how the effect generalizes across other intermediate resolution stages of the U-Net. All code used for training, analyzing and evaluating is public.

Code Availability

The code used to build, train, evaluate and experiment on the models as well as the `ModelAnalyzer` logging framework can be found on https://github.com/leonardozh1709/unet-bottleneck-flow-matching-2026.

Note on LLM Usage

Throughout this project Large Language Models were used to:

- review text to give feedback and point out inconsistencies, mistakes and grammatical errors
- help with understanding research papers and studying the field in general
- point towards research papers when asked where a certain concept comes from (This only served as a way to find the literature, it was checked manually.)
- draft parts of non-central sections of the paper. Concretely the discussion, contributions and abstract were drafted with AI. They were carefully edited and reviewed.
- help with formatting in LaTeX.

LLMs were NOT used to:

- come up with the actual main ideas and research questions
- write technical content of the paper
- make decisions about the structure and purpose of the paper
- choose, motivate and evaluate the experiments
- decide what is included in the paper (motivation topics, appendices etc.)
- generate images, tables and diagrams directly

As for the code, LLMs were used to debug code, correctly use libraries such as `clean-fid` and write utility and plotting functions but not to write important parts of code in the repository such as the model architectures, sampling algorithms, training, configurations and experiment setups.

The following models were used:

- Claude Sonnet 5 and Sonnet 5.5 (Anthropic)
- Gemini 3.6 Flash and 3.5 Flash Lite (Google)

Given these uses (editing, drafting, comprehension, coding), they are mostly not linked to specific passages. No AI-generated content was inserted directly, without revision, into the text.

Contents

Code Availability **2**

Note on LLM Usage **2**

1 Introduction **5**
1.1 Research Motivation . . . 5
1.2 Research Question . . . 5
1.3 Contributions . . . 6
1.4 Roadmap . . . 6

2 Theoretical Framework **7**
2.1 Generative Models . . . 7
2.2 Probability Paths and Vector Fields . . . 7
2.3 Flow Matching . . . 8
2.4 Stochastic Sampling via SDE and deterministic sampling via ODE . . . 9
2.5 U-Net Model Architecture . . . 10

3 Related Work and Motivation **12**
3.1 U-Nets and the Manifold Hypothesis . . . 12
3.2 Connecting Autoencoders to Flows . . . 12
3.3 Input Perturbation and Exposure Bias in Generative Models . . . 14
3.4 Gaussian Structure in U-Nets . . . 15

4 Central Hypotheses **15**

5 Methods **16**
5.1 Dataset . . . 16
5.2 Metrics . . . 17
5.2.1 Fréchet Inception Distance (FID) . . . 17
5.2.2 Teacher-Forcing vs. Autoregressive Generation KL Gap . . . 17
5.2.3 Model Derivatives . . . 18
5.2.4 Cross-Channel Correlation . . . 18
5.2.5 Effective Receptive Field . . . 18
5.2.6 Train vs. Test Loss Increase . . . 19
5.3 Compute and Implementations . . . 19
5.4 Seeding . . . 20
5.5 Implementation Bugs . . . 20

6 Preliminary Work **21**
6.1 The Base Mini-Scale Model . . . 21
6.2 Conditional Generation . . . 23
6.3 Input Perturbation Models . . . 25
6.4 Model Initialization Activations . . . 31

7 Experiment 1: Bottleneck Input Perturbation **32**
7.1 Setup . . . 32
7.2 Hypothesis . . . 32
7.3 Results . . . 33
7.3.1 FID Results . . . 33
7.3.2 Teacher-Forcing vs. Autoregressive Generation KL Gap Results . . . 35
7.3.3 Test Set Loss Results . . . 37
7.3.4 Model Derivative Results . . . 38
7.4 Summary & Empirical Claims . . . 38

8 Experiment 2: Bottleneck KL Loss Penalty **40**
8.1 Setup . . . 40
8.2 Hypothesis . . . 42
8.3 Results . . . 43
8.3.1 FID Results . . . 43
8.3.2 Bottleneck Swap Test Results . . . 44
8.3.3 Cross-Channel Correlation Results . . . 44
8.3.4 ERF Results . . . 45
8.3.5 Test Set Loss Results . . . 46
8.4 Summary & Empirical Claims . . . 47

9 Discussion **48**
9.1 Synthesis of Results and Research Question . . . 48
9.2 Limitations and Validity of this Study . . . 50

10 Conclusion and Outlook **51**
10.1 Conclusion . . . 51
10.2 Future Work . . . 51

11 Reflection **52**
11.1 Personal Reflection . . . 53

Bibliography **55**

Appendices **58**

A Theoretical RF vs. ERF **58**

B Hardware Optimizations **59**

C Derivations **61**
C.1 The Gaussian Distribution maximizes Differential Entropy . . . 61
C.2 The Noise-Prediction is proportional to the Conditional Score . . . 62
C.3 FM and CFM Loss Gradient Equivalence . . . 63

D List of All Trained Models **64**

E Hyperparameters **65**

F Full Model Architecture Graph **67**

G Additional Samples **71**

1 Introduction

1.1 Research Motivation

In the last few years, generative models for image generation have rapidly gained popularity, mostly due to their realistic high-resolution samples and their prompt adherence [11] [14] [30].

Figure 1: High-resolution samples from Stable Diffusion 3's 8 billion parameter rectified flow model. Source: [11].

While the visual quality of samples and control/guided generation [41] has improved substantially, knowledge about what these models internally represent, **interpretability**, remains relatively scarce. Being able to interpret the internal dynamics of these models might allow us to better understand existing problems and approach them correctly. In this work, we dive into the behaviors of U-Net [34] Flow-Matching [22] models when perturbations [25] or additional penalties are present.

1.2 Research Question

Not much interpretability work on flow-matching U-Nets exists and one component of the architecture stands out: the bottleneck. It is the lowest spatial resolution stage of the network, it has the largest effective receptive field, and its weights receive smaller relative updates than the rest of the network (section 5.2.5, Figure 8). These observations and the way the U-Net architecture is structured lead us to believe that the bottleneck is functionally distinct and mostly processes global semantic information. Existing methods against exposure bias, such as DDPM-IP (Denoising Diffusion Probabilistic Models with Input Perturbations) [25], perturb the network input and treat the U-Net as a single block. We instead ask what happens when training perturbation and penalization acts on the bottleneck specifically.

> **Research Question:** How does perturbing or penalizing the activations of a flow-matching U-Net's bottleneck during training affect (i) the model's internal behavior, in particular the mismatch between training and sampling distributions and the sensitivity of the output to its internal activations, and (ii) the quality of the generated samples?

This question is split into two parts, each tied to one experiment:

1. **Bottleneck input perturbation:** Does adding noise to the pre-bottleneck activations during training reduce the gap between teacher-forced and autoregressively generated activations (exposure bias), change the model's partial derivatives, and/or improve FID relative to an unperturbed baseline and to standard network-input perturbation?

2. **Bottleneck KL penalty:** Does penalizing the KL divergence between the post-bottleneck activations and a unit Gaussian change how the bottleneck functions (effective receptive field), and/or does this improve FID?

The two experiments might not feel immediately related, but ultimately, both approaches regularize the bottleneck by either providing it with worse inputs or pushing it to output worse outputs. Here "worse" means that the effective contribution of the clean, unperturbed and unpenalized signal is smaller.

These correspond to the two hypotheses in Section 4, where falsifications are also provided. Both hypotheses separate a representational claim from a quality claim, so a result is informative even if FID does not improve. Because standard input perturbation is the natural point of comparison for the first experiment, we first analyze how it behaves in our flow-matching setting (section 6.3).

1.3 Contributions

Flow matching [22], the U-Net [34], input perturbation training for diffusion models [25], the connection between input noise and Jacobian penalties [5], the Information Bottleneck principle [37] and the manifold hypothesis [8] are existing work. The contributions of this project are the following:

- **Bottleneck input perturbation:** I introduce noise injection into the pre-bottleneck activations of a flow-matching U-Net, using a time-decaying, standard-deviation-weighted noise schedule, and compare it to standard network-input perturbation and the base model.

- **Bottleneck KL penalty:** I introduce a KL-to-unit-Gaussian penalty on the post-bottleneck activations of a flow-matching U-Net.

- **Empirical evaluation:** I evaluate both interventions on class-conditional CIFAR-10 across a search space of strengths, in terms of sample quality (FID) and several internal metrics (exposure-bias KL gap, model derivatives, effective receptive field, cross-channel covariance, train vs. test set loss gap). Both interventions improve FID at suitable strengths, but the proposed mechanisms are not supported by these metrics.

- **Analysis tooling:** I release the *ModelAnalyzer* logging framework and the metric implementations used for this analysis.

I did not test my empirical claims at larger scales or on other datasets, and two implementation bugs affect parts of the experiments (section 5.5). I state claims only where the observed effects are large and discuss limitations in Section 9.2.

1.4 Roadmap

This paper begins by unrolling some of the theoretical frameworks required for training and sampling from flow models and providing a quick summary of the U-Net model architecture. The subsequent motivation sections detail relevant concepts and previous work such as exposure bias and the manifold hypothesis. Section 4 contains this project's two hypotheses, each with separated claims and falsifications. Next, in Section 5, we proceed into descriptions of the dataset, metrics and other relevant technicalities to this project. Before getting into the main experiments, we engage in some preliminary experiments 6 on input perturbations in our setting. Afterwards, we dive into setting up and evaluating the two main experiments 7 8: bottleneck IP and the KL-to-unit-Gaussian loss penalty. Finally, we discuss the results and synthesize them with the original research question. I end the paper with a short conclusion, future outlook and reflection. The appendices provide additional content: hardware optimizations, derivations, hyperparameter-tables, additional samples and a full architecture graph of the mini-scale U-Nets used across this project.

2 Theoretical Framework

2.1 Generative Models

To formalize what it means to generate something [19] using a generative model, we start by representing the objects (Images, Videos, etc.) that we want to generate as vectors in multidimensional space. Such a vector is denoted as $\vec{x} \in \mathbb{R}^{3\times H\times W}$ for image space.

Our task when constructing a generative model is to approximate the underlying data distribution $p_{data} : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ that maps any possible image to a scalar probability density greater than or equal to zero. Note that $\int_{-\infty}^{\infty} p_{data}(x)dx = 1$ holds.

Generation can be framed as sampling from the learned distribution. We define the quality of a generated sample as how likely it is under the data distribution p_{data}. Since we only have access to a finite amount of samples from p_{data} via the dataset, we can only approximate p_{data}.

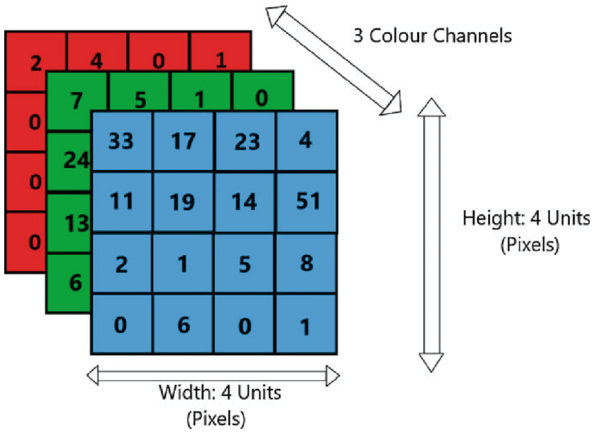


Figure 2: Example of a 4×4 image being represented by a three-dimensional tensor. The tensor consists of three feature maps of the same spatial size, one for each channel. Source: [24]

A generative model converts vectors from an initial distribution to ones from the data distribution. If we can parametrize a mapping from a simple, known distribution p_{init} that we can sample from, to an approximation of the data distribution at the end of mapping, we are able to generate samples from said approximate data distribution p_{end}. For my implementations, p_{init} will always be a normal distribution with unit variance $\mathcal{N}(0, I)$. Going forward, I will denote p_{init} and p_{end} as p_0 and p_1, respectively.

2.2 Probability Paths and Vector Fields

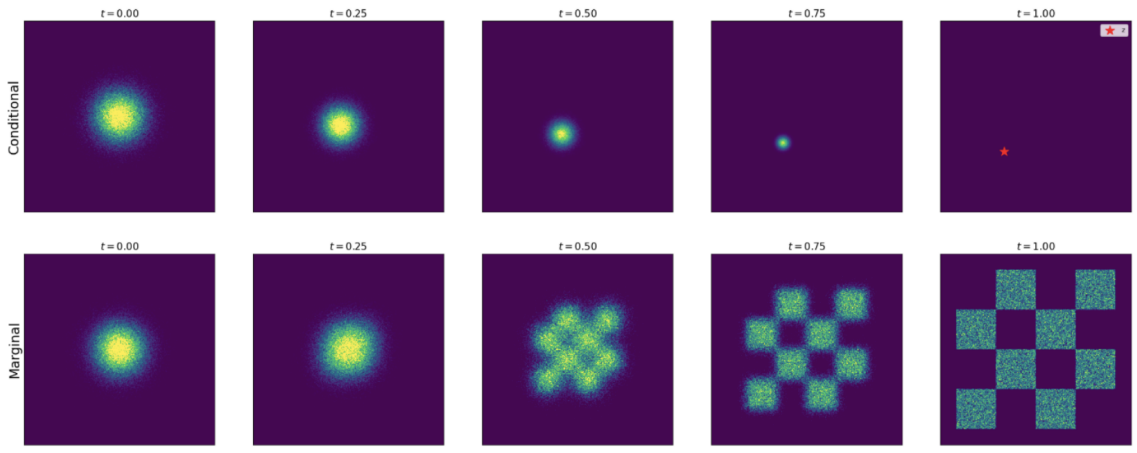


Figure 3: Example of a conditional (top) and marginal (bottom) probability path. The conditional probability path bridges a Gaussian $p_{init} = \mathcal{N}(0, I)$ and p_{data} for single data point x_1. The marginal probability path bridges a Gaussian and a data distribution p_{data}. (Here, p_{data} is an example distribution in two dimensions represented by a chessboard pattern.) Source: [19], description adapted.

Keep in mind that by *conditional* we refer to objects that are defined for a specific data point while *marginal* objects apply to an entire distribution.

One way to approximate a data distribution using a generative model is to parametrize our goal as learning a time-dependent vector field $v_\theta(x, t) : \mathbb{R}^d \times [0, 1] \to \mathbb{R}^d$ that defines a flow map $\phi_t(x)$ [19] [22].

The marginal flow maps any sample from the prior distribution $x_0 \sim p_0$ to an intermediate sample on the probability path at any time $x_t \sim p_t$. The conditional flow $\psi_t(x)$ is conditioned on the fact that the trajectory ends at a given x_1. We do not explicitly note this dependence for simplicity. We define a conditional probability path $p_t(x \mid x_1)$ (see figure 3) that bridges p_0 and a Dirac-delta distribution on a specific sample x_1, meaning $p_0(x \mid x_1) = p_0(x)$ and $p_1(x \mid x_1) = \delta_{x_1}(x)$. Here, $\delta_{x_1}(x)$ is positive if $x = x_1$ and 0 otherwise. We define the marginal probability path by averaging across conditional paths, weighted by the likelihood of their endpoint x_1:

$$p_t(x) = \mathbb{E}_{x_1 \sim p_1(x_1)}[p_t(x \mid x_1)] = \int p_t(x \mid x_1) p_1(x_1) dx_1$$

Here, $p_1(x_1)$ is the probability density at the point x_1 in the hypothetical continuous data distribution. Note that "sampling from the data distribution" is discrete in practice since the dataset is finite. The marginal probability path $p_t(x)$ is a time-dependent probability density function that describes how we bridge the two distributions p_0 and p_1. The relationship between the time-dependent vector field and the conditional flow is given by the ODE (equations (1) and (2) in [22]:

$$\frac{d}{dt}\phi_t(x) = u_t(\phi_t(x), t), \quad \phi_0(x) = x$$

Equivalently, this can be expressed in integral form:

$$\phi_t(x) = x + \int_0^t u_\tau(\phi_\tau(x), \tau) d\tau$$

We can numerically solve this ODE using our learned vector field to transport samples from p_0 to p_1. The marginal vector field $u_t(x)$ is constructed by averaging the conditional vector fields $u_t(x \mid x_1)$, weighted by how likely the clean sample x_1 is to emerge given we are at the state x (or how likely the current state x will end up as x_1):

$$u_t(x) = \mathbb{E}_{x_1 \sim p_t(x_1 \mid x)}[u_t(x \mid x_1)] = \int u_t(x \mid x_1) p_t(x_1 \mid x) dx_1$$

Lipman et al. [22] use the continuity equation to prove that the conditional vector field generates the conditional probability path and the marginal vector field generates the marginal probability path (Theorem 1 in their paper). Intuitively, if a vector field generates a probability path, its local velocity vectors transport samples from the initial distribution p_0 through all intermediate distributions $p_t(x)$ (or $p_t(x \mid x_1)$) until reaching the target distribution p_1. The vector field moves probability mass over time such that the resulting distribution satisfies the probability path at every instant t.

Early diffusion models (DDPMs [17]) utilized discrete Markov chains. In this work, I will be using a flow-matching training formulation, which uses continuous time.

2.3 Flow Matching

Flow Matching [22] is a simulation-free approach for training continuous-time flow models based on regressing conditional vector fields of fixed conditional probability paths. Simulation-free here refers to the fact that we do not need to numerically solve (simulate) ODEs during training. We define a conditional probability path that bridges the two distributions $p_0 = \mathcal{N}(0, I)$ and $p_1 \approx p_{data}$ using a linear interpolation. The conditional flow becomes:

$$\psi_t(x_0) = x_t = t x_1 + (1 - t) x_0$$

Where $x_0 \sim p_0$ and $x_1 \sim p_1$. The ground-truth conditional vector field that integrates to this choice of the conditional flow is its time-derivative:

$$u_t(x \mid x_1) = \frac{\partial}{\partial t}\psi_t = x_1 - x_0$$

This theoretically allows p_0 and p_1 to be arbitrary distributions of the same dimensionality. Our neural network will parametrize the time-dependent vector field $v_\theta(x_t, t)$ that generates the desired probability path. By evolving (numerically solving) the vector field, we flow from $x_0 \sim p_0$ to $x_1 \sim p_1$. A simple analogy is to imagine that we are learning a current that enables particles in some liquid to

flow from an arbitrary formation p_0 to a specific one p_1 by assigning each particle a velocity at every timestep.

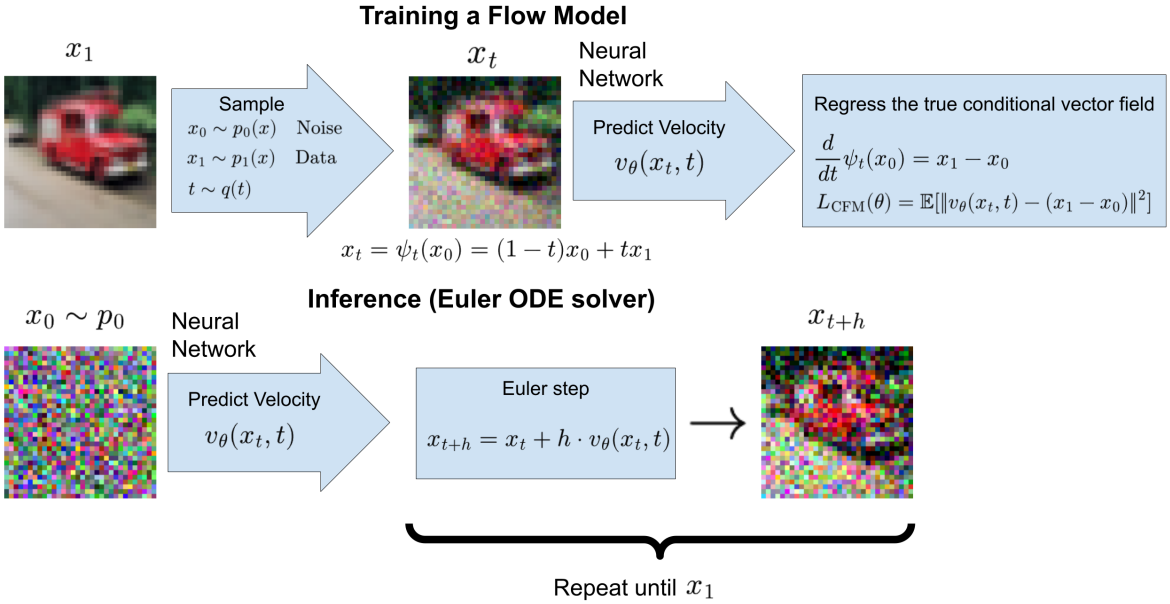


Figure 4: Flow Matching training and sampling framework. During training, noise $x_0 \sim p_0$, target data $x_1 \sim p_1$, and timestep $t \sim p(t)$ are sampled. The interpolated state is computed as $x_t = (1-t)x_0 + tx_1$. The neural network $v_\theta(x_t, t)$ learns to predict the target velocity field $x_1 - x_0$, which is the time-derivative of the conditional flow ψ_t. Timestep distribution $p(t)$ can be Uniform $\mathcal{U}[0,1]$, Logit-Normal Sigmoid$(\mathcal{N}(\mu, \sigma^2))$, or another custom schedule. During inference, samples are generated by numerically solving the learned ODE starting from $x_0 \sim p_0$ using an ODE solver such as Euler integration. Made by me in *Google Drawings*.

Because we cannot directly regress the marginal vector field $u_t(x)$ due to intractability, we train a neural network to approximate the marginal vector field $u_t(x)$ by regressing against samples of the tractable conditional vector field. The authors of [22] prove that "The [Marginal] Flow-Matching and Conditional Flow-Matching objectives have identical gradients w.r.t. θ" i. e. $\nabla_\theta \mathcal{L}_{CFM}(\theta) = \nabla_\theta \mathcal{L}_{FM}(\theta)$. I also provide a proof in Appendix C.3. The CFM loss is easy to use and can be used for large-scale training.

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}||v_\theta(x_t, t) - (x_1 - x_0)||^2$$

Here the expectation is taken over $t \sim \mathcal{U}[0,1]$, $x_1 \sim p_1$ and $x_0 \sim p_0$ so $x_t = \psi_t(x_0)$. The main reason the CFM loss works is because when we train $v_\theta(x_t, t)$ against the conditional velocity using MSE regression, its optimal prediction (across training) for an instance of inputs (x_t, t) is exactly the marginal vector field's output since training exposure implicitly weights by the endpoint likelihood $p(x_1 \mid x_t)$:

$$v_\theta^*(x, t) = \mathbb{E}_{x_1 \sim p(x_1|x_t)}[u_t(x|x_1)] = u_t(x)$$

2.4 Stochastic Sampling via SDE and deterministic sampling via ODE

A flow model is given by a velocity field parametrized by a neural network v_θ, an initial condition $x \sim p_0$ and thus an ODE [19].

$$dx_t = v_\theta(x_t, t)dt$$

To sample from such models, we cannot directly compute an analytical solution to their differential equations, instead we run numerical simulations such as the Euler algorithm:

$$x_{t+h} = x_t + hv_\theta(x_t, t)$$

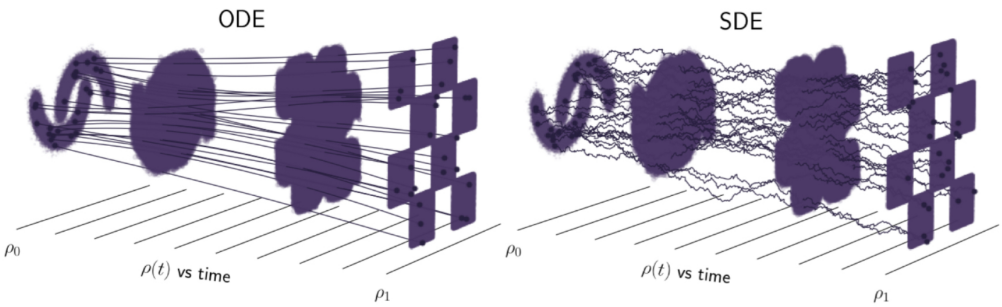


Figure 5: Example of SDE and ODE paths across time for a two-dimensional dataset where p_0 is a spiral-like shape and p_1 is a checkerboard pattern. Source: [1].

Section 4.3 of [35] establishes that for all diffusion processes, there exists a corresponding deterministic process whose trajectories share the same marginal probability densities as the SDE. This deterministic trajectory is the probability flow ODE. This result lies outside of this work's scope, I use this as a motivating fact to think of DDPMs, continuous diffusion models and flow models as different parametrizations of the same underlying object: the time-dependent vector field (see Section 3.2). Crucially, the same vector field can define both a deterministic transport ODE and a stochastic diffusion process. While flow models are deterministic and can be sampled from using the standard Euler algorithm, diffusion models allow use of a numerical SDE solver such as the Euler-Maruyama algorithm, that adds scaled Gaussian noise at every step.

2.5 U-Net Model Architecture

The U-Net model architecture was first proposed for biomedical image segmentation [34]. The U-Net is a deep convolutional neural network (CNN) with an encoder-bottleneck-decoder structure where the encoder downsamples the input to limited spatial resolutions, effectively compressing the pixel-space that the data lives in into an encoded semantic landscape. The decoder reconstructs an output from the encoded representation and concatenation skip-connections. The skip-connections allow the model to preserve fine-grained pixel information despite spatially downsampling the input in the decoder, the bottleneck does not need to keep fine pixel information.

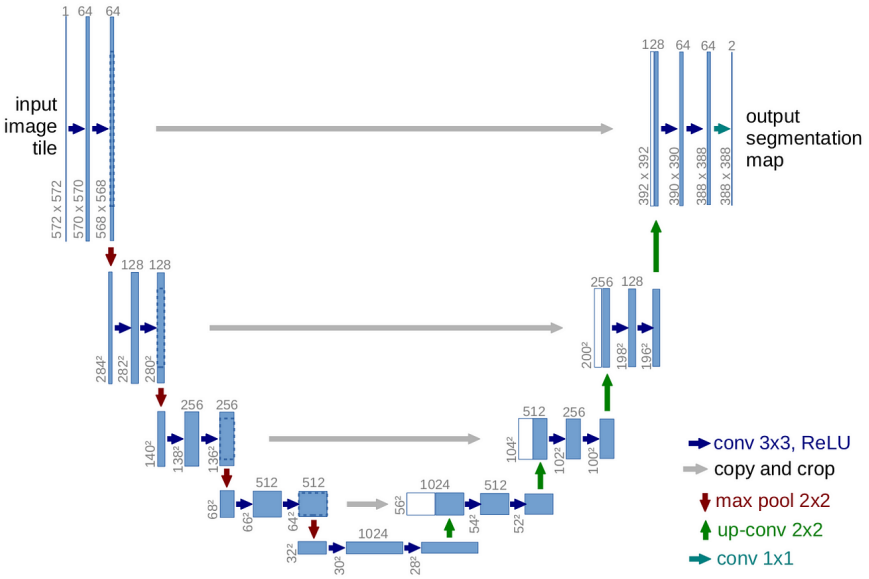


Figure 6: The U-Net model architecture as seen in [34].

The vector field $v_\theta(x_t, t)$ is parametrized using a U-Net neural network. For this project, I will be using a modified U-Net with 3 layers of which some use multi-head self-attention [39]. The encoder and decoder each consist of doubled residual blocks [15] with downsampling or upsampling operations. Overall, I roughly follow the modified U-Net for generative modelling from the DDPM paper [17]. For the full architecture used in this project's mini-scale U-Nets see Appendix F.

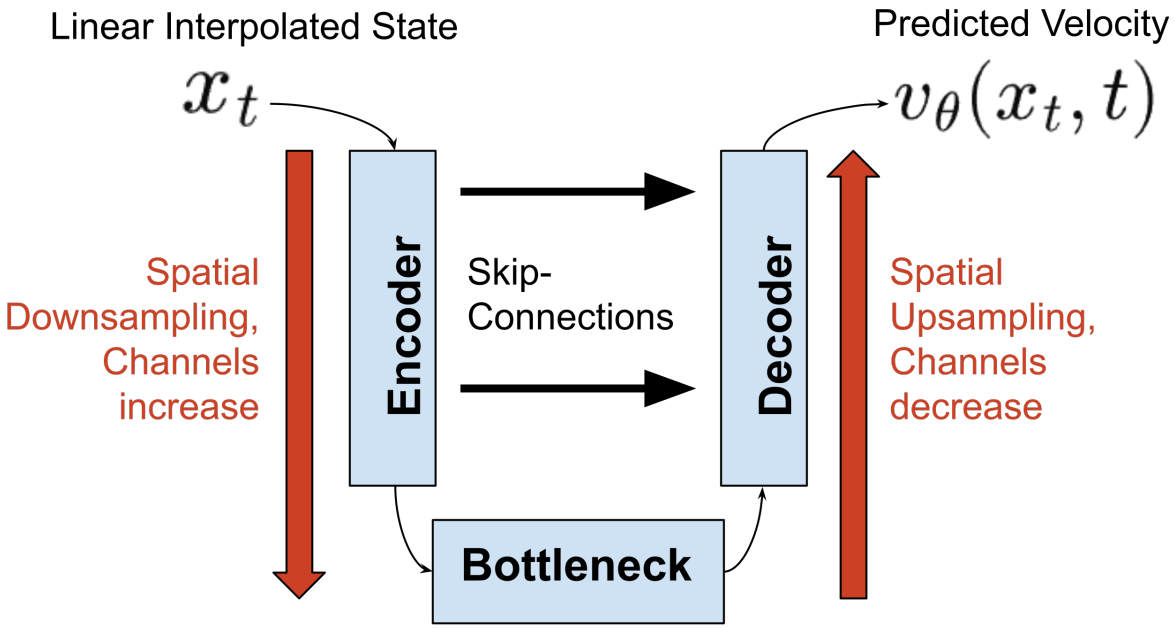


Figure 7: A scheme of a U-Net model used for Flow-Matching. Made by me in Google Drawings.

We can plot the gradient-to-weight ratio against the operation order of the U-Net's modules to see that lower layers receive relatively smaller weight updates:

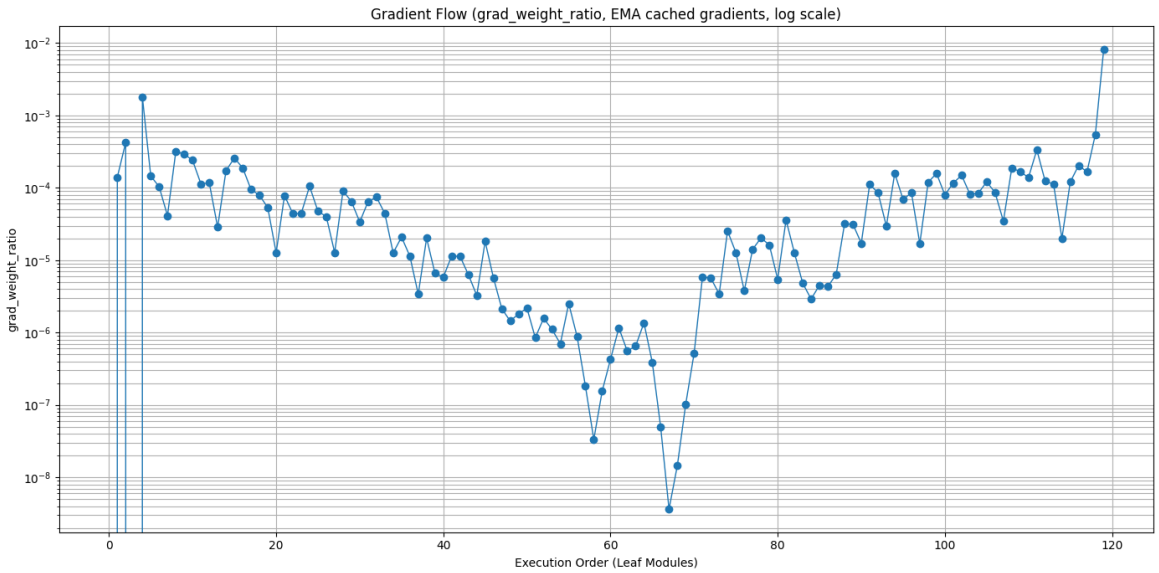


Figure 8: The gradient-to-weight ratio of leaf modules in operation order. Plotted with a logarithmic scale. This shows that the relative weight updates are several orders of magnitude smaller in the bottleneck layers. The datapoints are created using exponential moving averages of the gradients that were captured during the 400 epochs of training the base mini-scale model. Note that the near-zero datapoints in the beginning are from the time and class embedding MLP modules. Data points are averaged across 3 differently-seeded, trained models.

Figure 8 and Section 5.2.5 both point at the bottleneck as functionally distinct.

3 Related Work and Motivation

3.1 U-Nets and the Manifold Hypothesis

The manifold hypothesis is a foundational principle in machine learning, which says that "...the dimensionality of many data sets is only artificially high; though each data point consists of perhaps thousands of features, it may be described as a function of only a few underlying parameters. That is, the data points are actually samples from a low-dimensional manifold that is embedded in a high-dimensional space" [8]. Unlike linear dimensionality reduction methods such as Principal Component Analysis (PCA) [8], neural approaches can capture non-linear relationships in the data. For instance, autoencoders achieve this by mapping input data $x \in \mathbb{R}^{d_1}$ to a lower dimensional latent representation $h \in \mathbb{R}^{d_2}, d_1 \gg d_2$ via an encoder function and reconstructing it using a decoder. We refer to the lowest-resolution stage of the U-Net's encoder-decoder path, where spatial resolution is minimized and channel depth maximized, as the bottleneck. Note that the total data dimension in the bottleneck $C \times H \times W$ is not actually lower, it has more channels at the cost of less spatial positions. Deeper U-Net layers have larger receptive fields (see Appendix 5.2.5), and likely prioritize global semantic changes over fine-grained information (Page 7 of [29]) This was originally studied in the context of DDPMs, I assume this transfers to FM as established in 3.2. The U-Net's skip connections allow the model to preserve high-frequency pixel information such as textures despite the strongly spatially down-sampled bottleneck. This suggests that a U-Net Neural Network encodes the input into a latent representation that represents the data manifold, not the true high-dimensional space the data exists in. The decoder constructs an output by upsampling the bottleneck representation and using the skip connections to recover fine-grained information.

3.2 Connecting Autoencoders to Flows

Standard autoencoders can sometimes learn mappings that are overly sensitive to small variations in the input data. Denoising Autoencoders (DAEs) [40] intentionally corrupt (noise) the input data and then train to recover the original from the corrupted version (see figure 9). This forces the autoencoder

to learn features that capture the essential structure of the data, i. e. the data manifold instead of its structure in pixel space.

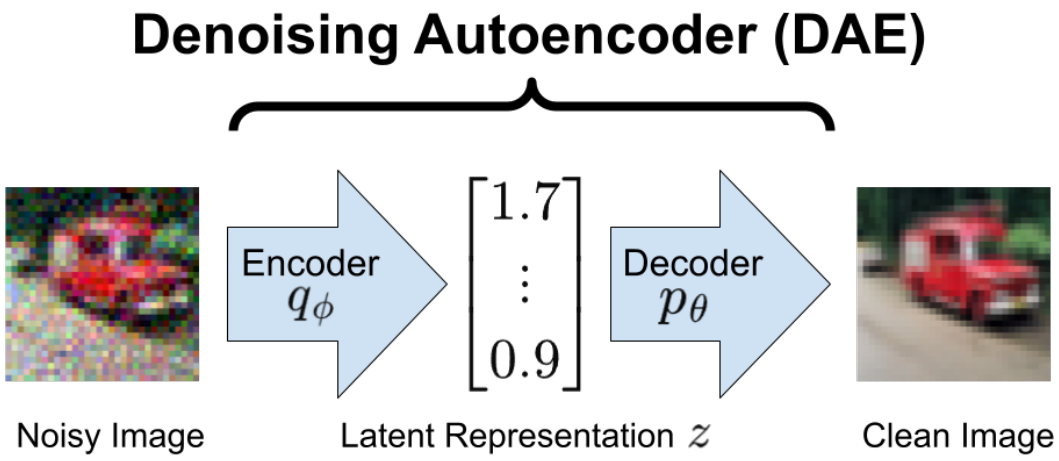


Figure 9: A scheme of a denoising autoencoder. The encoder q_ϕ encodes the noisy image and creates the latent representation z. The decoder p_θ decodes z into a clean image. Made by me in *Google Drawings*.

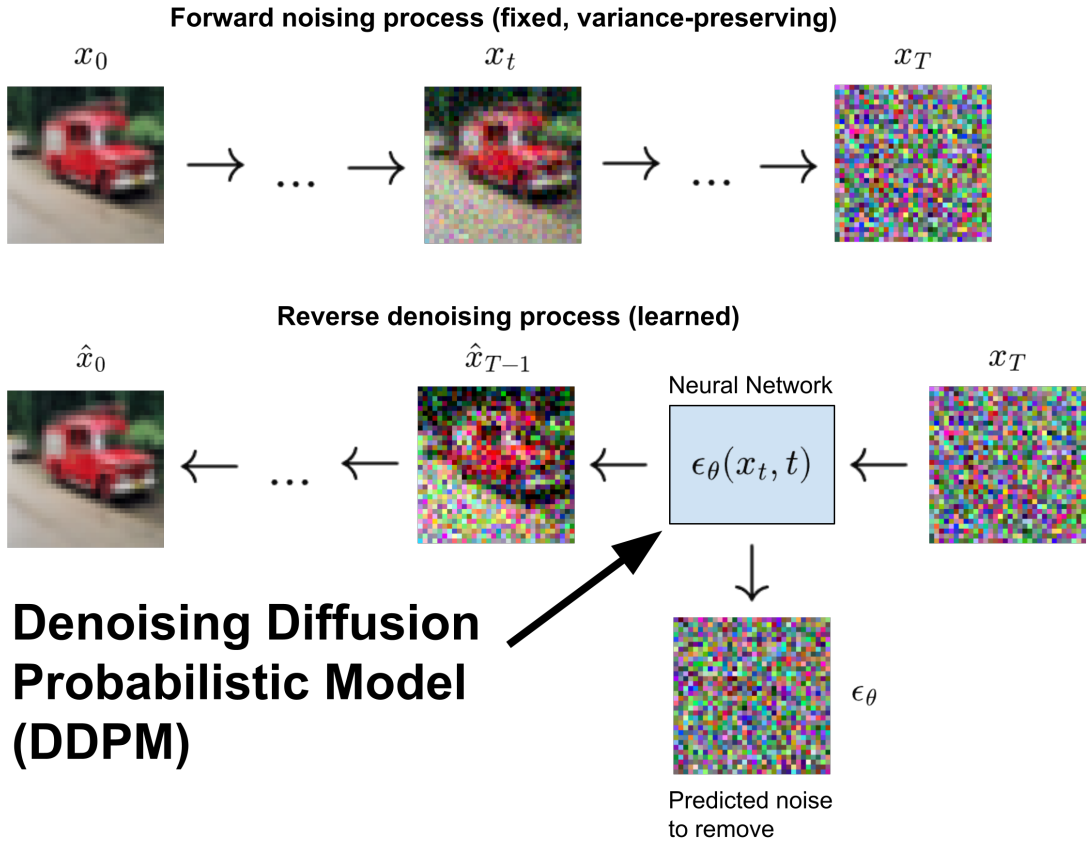


Figure 10: A scheme of a DDPM. The forward noising process is a Markov Chain that destroys the clean signal. The reverse noising process, parametrized by a neural network $\epsilon_\theta(x_t, t)$, iteratively removes noise from a pure-noise starting point x_T to arrive at a clean sample $\hat{x}_0$. Made by me in *Google Drawings*.

A DAE is a neural network that learns to remove a certain constant amount of noise from data. A DDPM [17] is a generative model that iteratively turns noise into data. In theory, a DDPM is a chain of DAEs [4] [17]. A DDPM breaks down the difficult task of generating an image from pure noise into a sequence of small denoising steps (see figure 10). In the reverse Markov process (generation), the network takes noisy data at step t and predicts the noise $\epsilon_\theta(x_t, t)$ to remove, given a noised image x_t and a timestep t, passing the cleaner output to step $t-1$. Functionally, every step in that chain acts like a time-conditioned Denoising Autoencoder. **A DDPM is a variance-preserving, time-conditioned DAE trained in closed form.** (By closed-form training, we refer to the fact that any intermediate noised image x_t can be sampled directly from the original image x_0 in a single step, without recursive construction of the earlier states $x_1, \ldots, x_{t-1}$. By variance-preserving, we refer to the fact that the DDPM closed-form noising formula keeps the variance of x_t approximately constant over time, provided that the data is normalized such that $\mathrm{Var}(x_0) \approx 1$.)

DDPM models with Markov chains of N total steps correspond to specific instances of variance-preserving SDEs in the limit of $N \to \infty$ (see Appendix C of [35]). A DDPM's network output $\epsilon_\theta(x_t, t)$ predicts the noise added to the clean sample. Training the neural network to predict the noise is analogous to score matching [35] since the noise prediction is a linear rescaling of the score function $\nabla_x \log p_t(x_t)$ (see C.2 for a proof of the conditional case, Appendix A of [12] holds the marginal claim). For all diffusion SDEs, there exists a corresponding ODE that shares the same marginal densities (section 4.3 of [35]). The three formulations describe the same underlying object: a time-dependent vector field on the data manifold that is reached via different training targets: noise $\epsilon_\theta(x_t, t)$ (DDPM), score $\nabla_x \log p_t(x_t)$ (score-based SDE models), or velocity $v_\theta(x_t, t)$ (Flow Matching). **DDPMs, SDE (Diffusions) and ODE models (Flows) are different parametrizations of a time-dependent vector field.**

3.3 Input Perturbation and Exposure Bias in Generative Models

Exposure bias A long sampling chain in Flow and Diffusion Models leads to an error accumulation. Exposure bias emerges because there is a discrepancy between training and sampling distributions. The model is trained on ground-truth pairs where it always sees an $x_t = tx_1 + (1-t)x_0$ where $x_1 \sim p_{data}$ ("Teacher Forcing"). During inference, the model iteratively generates a sequence of samples based on the results of previous steps (e. g. through an Euler ODE Solver), possibly accumulating errors. Since the model was never trained to predict the correct velocity from an inaccurate point on the trajectory, this can cause the model to go off-manifold during the sampling chain.

Input perturbation The authors of the DDPM-IP paper [25] propose a solution to alleviate the exposure bias problem in DDPMs by modeling the sampling error as a Gaussian during training time. This is done using a random noise vector ξ used to create a perturbed version of x_t. The new noise term ξ is used *asymmetrically*, meaning it is applied to the input, but not the prediction target. Similar to contractive autoencoders (CAEs) [33], DDPM-IP aims to make small changes in the input correspond to even smaller changes in the output. CAEs encourage changes in their latent space to be smaller, DDPM-IP cares about changes in the model output. I hypothesize that encouraging small changes in the U-Net bottleneck's latent representation to correspond to small changes in the output makes the model more robust to weak latent representations during sampling and allows the bottleneck to perform its task of global semantic processing (see 2.5, Appendix A.1).

Extension to the bottleneck Bishop (1995) [5] shows that training with additive noise is asymptotically equivalent to penalizing large values inside of the network's Jacobian (I test this empirically in 6.3). I extend this to noise injected at an internal (pre-bottleneck) representation rather than the input. We refer to this as Bottleneck Input Perturbation (b-IP). This extension is intuitively justified because we can look at the bottleneck and decoder layers as one network object. By adding perturbations to this sub-network's input (which are the pre-bottleneck activations) we are penalizing large values in the Jacobian $\frac{\partial y}{\partial h}$ where h is this subnetwork's input.

Table 1: Comparison between the effects of CAEs, DDPM-IP and my proposal. Here, h is the latent representation or bottleneck activations, x is the input and y the output.

Contractive Autoencoders (CAEs)		DDPM-IP		My proposal (b-IP)	
$\left\lvert\frac{\partial h}{\partial x}\right\rvert$	small	$\frac{\partial y}{\partial x}$	small	$\frac{\partial y}{\partial h}$	small

3.4 Gaussian Structure in U-Nets

The Gaussian distribution is the PDF that maximizes differential entropy under the constraints of normalization, fixed mean and fixed variance. For some helpful intuition on how this can be proven, see Appendix C, for a full proof, see [38]. Because the Gaussian distribution maximizes differential entropy for a specified mean and variance, it represents the least structured distribution consistent with these constraints. This makes Gaussian distributions a natural choice when no additional structure is intended to be imposed.

Information Bottleneck The Information Bottleneck principle [37] intuitively says that to obtain a "good" latent representation of a variable we must preserve the information relevant to the target and minimize the mutual information of the latent representation and the original variable, thus enforcing compression. To minimize the mutual information $I(X, Z)$ of the latent variable Z and the input X we can impose constraints on the distribution of the latent variable. Variational Autoencoders [7] do this by penalizing the KL-Divergence between the distribution of Z given by the encoder $q_\phi(Z|X)$ and a unit Gaussian $\mathcal{N}(0, I)$:

"[...], the KL divergence term $D_{KL}(q_\phi(Z|X)||p(Z))$ of the β-VAE objective can be seen as an upper bound on the amount of information that can be transmitted through the latent channels per data sample [...] [7]."

Here $X \sim p_{data}(X)$ is a sample from the dataset, $q(Z|X)$ is the encoder distribution (given an input, produce a distribution over latent variables) and $p(Z)$ is the distribution that we enforce in latent space, e. g. $\mathcal{N}(0, I)$.

The Gaussian distribution having minimum structure is key here, it adds no structure other than a fixed mean and covariance to the latent variable Z, keeping constraints present but minimal. In the VAE framework, minimizing the KL-to-unit-Gaussian penalty minimizes an upper bound on the mutual information between input and latent representation. (for a proof, see Appendix C of [20])

In this case, it does not directly upper-bound the mutual information since my U-Net's encoder and decoder are deterministic. The proposed approach is a distribution-matching regularizer motivated by the VAE KL term and the IB principle.

Proposed penalty I hypothesize that adding a KL-to-unit-Gaussian loss penalty to the latent distributions of the U-Net's bottleneck may enforce compression through its regularizing effect and push the bottleneck to focus on semantic information even more. Fine-grained pixel structure is preserved via skip-connections. By the manifold hypothesis 3.1, I suppose that a compressed/regularized latent representation might push the bottleneck to learn the underlying structure of the data rather than memorizing high-dimensional details.

4 Central Hypotheses

Motivated by the manifold hypothesis 3.1, the Information Bottleneck principle, the effects of input perturbation training 3.3 and the connection between denoising autoencoders and flow-matching models established in 3.2, this project tests two related, but distinct, hypotheses tied to experiments on the U-Net's bottleneck.

Hypothesis 1. ***Bottleneck Input Perturbation:*** *Encouraging the bottleneck-to-output mapping to be less sensitive to perturbations in its input representation, analogous to the effect Bishop (1995) shows for input-level noise, and to DDPM-IP [25], will reduce the teacher-forcing (training)/autoregressive (sampling) distribution gap in the bottleneck's activations, and may improve generation quality relative to an unperturbed baseline. Concretely, I inspect the exposure-bias gap, the model's partial derivatives and training set vs. test set loss.*

Falsification: *This hypothesis is evaluated on two independent claims. (1) Representational claim: if bottleneck IP does not show any statistically meaningful change in the used metrics, this claim is falsified regardless of generation quality. (2) Quality claim: if the representational claim holds but FID shows no improvement over baseline, I conclude the representational change does not translate into improved generation quality in this setting. The two claims are interpreted separately.*

Hypothesis 2. ***Bottleneck KL Penalty:*** *Explicitly penalizing the KL-divergence between the bottleneck's activation distribution and a unit Gaussian, minimizing the mutual information $I(X, Z)$ per the Information Bottleneck principle, will enforce additional compression on the bottleneck representation, pushing it toward semantic rather than fine-grained content, and may improve generation quality relative to an unpenalized baseline.*
Falsification: *This hypothesis is likewise evaluated on two independent claims: (1) Bottleneck functionality claim: if evaluating the effect on various metrics shows no meaningful changes this is falsified. (2) Quality claim: If I find that the KL penalty has a measurable effect on internal model dynamics but does not lead to a better FID score over baseline, I conclude the functionality claim does not entail improved generation quality.*

An improvement in FID despite no representational change is an interesting result of its own that supports the quality claim but falsifies the representational claim. Such a result suggests that there is a property not included in my metrics that improves the generative quality of U-Net neural networks in a flow-matching setting.

These hypotheses are evaluated independently. Both experiments do not rely on a measurable increase in generation quality to be considered successful, the evaluations themselves and any meaningful interpretation/takeaways are valuable to this work. In addition to these hypotheses, the section on Input Perturbation Models (6.3) aims to examine the effects of IP in my Flow-Matching U-Net section. However, it must be said that due to limited compute most evaluations will not be done across multiple training runs/seeds. I try to avoid making statements that would require additional empirical evidence but allow myself to formulate key takeaways and interpret results where effects are sufficiently large.

5 Methods

5.1 Dataset

The only dataset that I use in this project is the CIFAR-10 dataset (Section 3.1 of [21]), which consists of 60'000 32×32 RGB images in 10 mutually exclusive classes of 6'000 images each. The training set consists of 50'000 images, and the test set contains the remaining 10'000. I use the training set for training all of the models, the test set is only used for evaluation metrics such as the training vs. sampling KL gap.

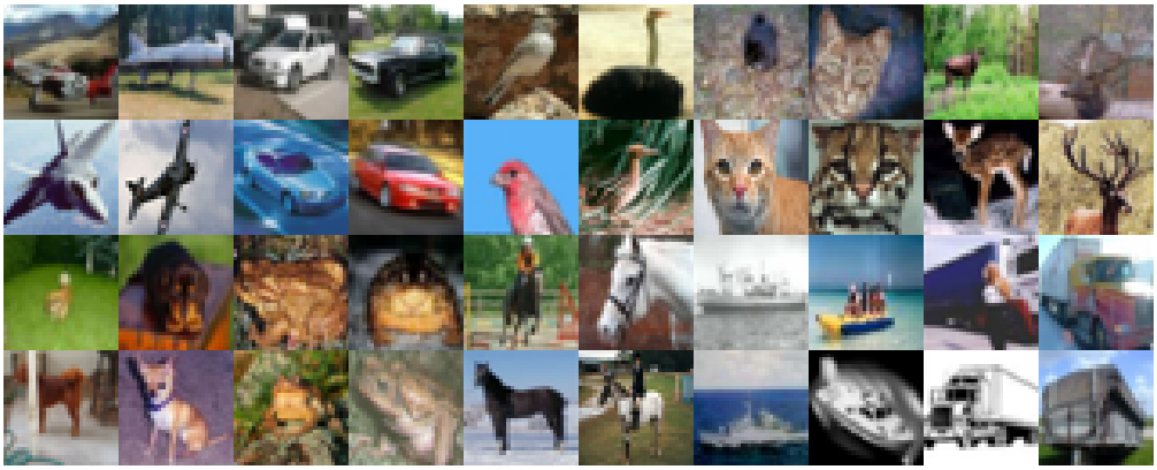


Figure 11: Four random images per class from the CIFAR-10 training set.

Additionally, the image's pixel values are rescaled from the range [0, 1] to [-1, 1] through $x_{normalized} = 2x - 1$. This is done because our probability path's starting points are sampled from $p_0 = \mathcal{N}(0, I)$. Ensuring $x_1 \in [-1, 1]$ and mean$(x_1) \approx 0$ makes sure that x_0 and x_1 are equally centered and similar in scale, preventing either endpoint from dominating the conditional flow's linear interpolation $\psi_t(x_0) = tx_1 + (1 - t)x_0$.

5.2 Metrics

The code for these three metrics can be found in this project's GitHub repository (see Code Availability).

5.2.1 Fréchet Inception Distance (FID)

Throughout this project, the metric used to quantify image quality is the Fréchet Inception Distance (Appendix A1 of [16]). FID encodes a large quantity of images (usually 50k) from the dataset and the models output distribution using an Inception CNN and computes the Wasserstein-2 distance between both distributions. The distributions are assumed to be multivariable Gaussians with full (not diagonal) covariance matrices. When I only need a metric for relative image quality, I go as low as FID-5k. For the final evaluation I use FID-50k. In practice, I use the `clean-fid` [27] python library with the CIFAR-10 train set. Note that FID operates on distributions, not individual samples, simply using a very high CFG scale will not improve FID as the samples might visually seem better but the sample distribution will not match the training set data distribution.

Note the documented limitations of FID as a measure of global structural coherence [36]: "FID does not correlate with human error rate". Here "human error rate" refers to the frequency with which humans fail to distinguish an AI-generated image from a real photo.

5.2.2 Teacher-Forcing vs. Autoregressive Generation KL Gap

Approach Exposure bias, as detailed in 3.3, is a distribution shift generative models experience when moving from ground-truth training inputs to iteratively generated samples that potentially do not lie on the data manifold. Since the model never learned to predict the velocity $v_\theta(x_t, t)$ from off-manifold points x_t its performance can suffer. I use an approximate KL-divergence metric to quantify this distribution shift: I encode exact conditional flow and iterative Euler integration versions of x_t using the same model that was used to generate them.

$$x_t^{TF} = \psi_t(x_0) = (1 - t)x_0 + tx_1$$

$$x_t^{AR} = x_{t-1}^{AR} + v_\theta(x_{t-1}^{AR}, t)dt$$

Here, v_θ comes from the model whose distribution shift KL-gap I want to analyze. In practice, I use the model to encode both x_t by running the U-Net until the post-bottleneck activations. Those activations are globally average-pooled across spatial dimensions (H, W) so we get a (B, C) tensor where C is the amount of channels in the U-Net bottleneck. I approximate TF and AR distributions as full-covariance multivariable Gaussians using 9600 samples (100 test-set batches) using `torch.distributions.MultivariateNormal` and compute the KL divergence between those two. I also use `torch.distributions.kl_divergence` for simplicity and to not risk implementation mistakes since my linear algebra knowledge is not sufficient. To visualize the results, I bin the results by timestep (averaging inside each bin). The noise x_0 is deterministically seeded per batch so TF and AR share the same initial states.

Limitations The following limitations of this method must be considered:

- **Gaussian Approximation**: Only the mean and covariance of the features are compared since distributions are approximated as multivariable Gaussians. Fréchet Inception Distance (FID) 5.2.1 uses the same simplifying assumption.
- **Exposure Bias Proxy**: This metric is merely a proxy, not exposure bias itself. "Exposure bias" itself does not have a well-defined value so we must use estimative metrics.
- **First Bin**: At $t = 0$ the teacher-forced and generated x_t are identical by construction, so KL is 0. Bin 0 is therefore partially trivial.

Interpretation For our purposes, interpreting the resulting values/plots is straight-forward: a high KL value for one bin implies a larger exposure bias distributions shift for the timesteps that fall in that bin.

5.2.3 Model Derivatives

To approximate the network's derivatives mentioned in table 1 I use the Frobenius norms of projected Jacobian matrices: The approximate sensitivities are Frobenius norms of the Jacobian, estimated per image with one Gaussian-probe backward pass ($|J^T\vec{v}|_F, \vec{v} \sim \mathcal{N}(0, I)$), multiplied by the standard deviation of the input tensor (and divided by the std of the output tensor for $\frac{\partial h}{\partial x}$), and averaged over 9'600 test images in `bfloat16`. I use the Jacobian's Frobenius norm to get a scalar value that represents the "rate of change" ($|J|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n}|j_{ij}|^2}$). PyTorch's autograd never actually builds the full Jacobian J since it would be of shape (d_{in}, d_{out}) because it captures how every output is influenced by every input, effectively this would mean 32 backward passes per image, which is not computationally feasible. Instead, one backward pass takes a vector $\vec{v}$ and returns the matrix-vector product $J^T\vec{v}$ which has the same shape as the input (d_{in}). Each instance of $J^T\vec{v}$ is a random linear combination of the rows of J. Because the entries of $\vec{v}$ are independent with zero mean and equal non-zero variance across entries, each row contributes equally in expectation. I therefore use $|J^T\vec{v}|_F$ with $\vec{v} \sim \mathcal{N}(0, I)$ as an approximation of the Frobenius norm of the Jacobian. I multiply by the std of the tensor that was differentiated with respect to, so the result represents change per one standard deviation change in the input, making it more comparable across models.

5.2.4 Cross-Channel Correlation

A neural network's channels being highly correlated indicates feature redundancy and "wasteful" allocation of representational power. Therefore, we denote models with lower cross-channel-covariance relative to a baseline as better utilizing their latent space's capacity for high-dimensional information. This connects to the manifold hypothesis (section 3.1): Independency across channels suggests that a model is learning the inherent low-dimensional structure of the data, not "memorizing" information. Similar to the covariance regularization term in [3], I calculate the average absolute cross-channel covariance as follows:

$$\frac{1}{n}\sum_{i=1}^{n}|\operatorname{Cov}(x_i)|$$

Where $n = C(C-1)$ is the number of off-diagonal items in the covariance matrix. In practice, the post-bottleneck encoded states h_{out} are globally average pooled to obtain one vector of shape (C) per image. I compute the covariance using 28'800 instances (9'600 images $\times$ 3 repetitions). To make these values comparable across models, I standardize all samples per channel to zero mean and unit variance so the covariance is not influenced by individual variances, which might vary across models and I use the same random seeds for each model. For this metric, I sample the timestep t from a uniform distribution.

5.2.5 Effective Receptive Field

In order to quantify a layer's reliance on global context I use the effective receptive field (ERF) [23]. Appendix A explains why the theoretical receptive field is not a suitable metric. To calculate the ERF of a model, I start by extracting a `ResBlock`'s output using a PyTorch forward hook. The inputs are sampled from $\mathcal{N}(0, I)$ with $t \sim U(0, 1)$, therefore the ERF is averaged over timesteps and measured on noise inputs. We find the approximate center pixel of these activations which gives us central activations of shape $(B, C, 1, 1)$. Next we want to find the gradient of the activations with respect to the input image. Since the central activations are not a scalar, we backpropagate starting from a unit gradient, effectively assuming each entry in the $(B, C, 1, 1)$ activations received an upstream gradient of 1.0. Backpropagation yields a gradient map of size $(B, 3, H, W)$. We sum the squared gradients across channels to obtain one (H, W) summed gradient map per batch instance that represents how much a change in each input pixel affects the central unit of our selected activations. These gradient maps are averaged across n_{reps} random input image tensors sampled from $\mathcal{N}(0, I)$. Once we have one average gradient map per module whose receptive field we want to analyze, all `ResBlocks` in our case, we can turn it into a scalar ERF diameter values using an RMS radius. For each gradient map, we

calculate the total weight w_{total} by summing all pixel values. Next, we find the map's spatial centroid $(x_{\text{center}}, y_{\text{center}})$ as the intensity-weighted mean of the pixel coordinates:

$$x_{\text{center}} = \frac{\sum_{x,y} x \cdot \text{Intensity}(x, y)}{w_{\text{total}}}$$

$$y_{\text{center}} = \frac{\sum_{x,y} y \cdot \text{Intensity}(x, y)}{w_{\text{total}}}$$

The effective radius (r_{RMS}) is then calculated as the root mean squared distance of each pixel from the centroid, weighted by pixel intensities. Finally, the radius is multiplied by two to obtain a scalar effective diameter value for each gradient map:

$$\text{RMS Diameter} = 2.0 \cdot r_{\text{RMS}}$$

In practice, I use $n_{reps} = 100$ and report the standard deviation across training seeds for models that were re-seeded. We can compute the ERF of a randomly initialized model with my mini-scale U-Net architecture to get an idea of the ERF that the model architecture is natively equipped with:

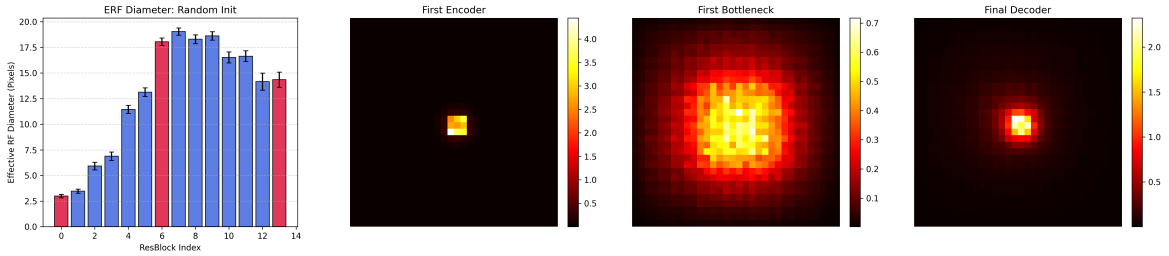


Figure 12: ERF RMS diameters and average gradient maps of selected `ResBlock`s for randomly initialized models, averaged across 10 random models seeds and 100 reps per seed. Standard deviations in the bar plots are sample standard deviations across seeds.

Figure 12 shows how an untrained model has the largest receptive field in the second/last bottleneck block (`ResBlock` 7).

5.2.6 Train vs. Test Loss Increase

I use the mean loss increase when moving from train set to test set images as a metric for the model's generalization. This metric measures the outcome, it does not give us any information about why a model acts a certain way. A loop across test and train loader in parallel track the train and test CFM loss values with identical $x_0 \sim \mathcal{N}(0, I)$ and $t \sim U(0, 1)$ for test and train trajectories. After one evaluation pass over $N = 9'600$ images, we get a single number for the increase:

$$\text{Ratio} = \frac{\frac{1}{n}\sum_{i=1}^{n} \mathcal{L}_{test}}{\frac{1}{n}\sum_{i=1}^{n} \mathcal{L}_{train}} - 1$$

A ratio of means is used instead of a mean of ratios to avoid very small values in the denominator disproportionally increasing the standard deviation. I report the mean across $M = 20$ epochs and report the standard deviation across training runs for models that were re-seeded.

5.3 Compute and Implementations

Throughout the entire project I use a single NVIDIA GeForce RTX 4060 GPU, which has 8GB RAM. Using the hardware optimizations detailed in Appendix B I achieve a peak throughput of $\approx$ 1120 images/s for $3 \times 32 \times 32$ CIFAR-10 images on the mini-scale models.

Throughout this project, a total of 20 mini-scale models were trained. At $\approx$ 6.2 hours per model this equates to roughly 124 hours of training. I used a training script that allowed me to queue multiple models for training and leave it running for upwards of 18 hours uninterruptedly.

The deep learning framework I use is consistently `PyTorch`. Other central libraries include `Numpy` which is occasionally used for array operations and `matplotlib.pyplot`, which is used for virtually all plots.

5.4 Seeding

Due to limited compute, it is not possible for me to seed every experimental model multiple times. I decided on the compromise to train the different experimental models once and only re-seed the base model and each experiment's best (FID-wise) model twice, so they have three separated data points on various metrics. The sample standard deviation across seeds is reported where useful. For a full list of all trained models, see Appendix D.

5.5 Implementation Bugs

These two bugs were not caught before executing preliminary and experimental scripts and partially affect the following sections, which is why I detail them here:

Learning-Rate Schedule Bug

During development, I found an implementation error in the learning-rate schedule. Although the schedule was correctly computed, it was not passed to the optimizer, meaning that all models trained prior to this fix used a constant learning rate of $\mathrm{lr}_{\max}$. I corrected the implementation and re-trained the base mini model using the intended schedule. The corrected model achieved a lower FID-50k score.
I discovered this issue after a substantial portion of the training runs had already been completed. Given the remaining compute budget, it was not feasible to re-train all models with the corrected learning-rate implementation. I therefore retained the constant learning rate for the mini-scale experiments and acknowledge this implementation detail as a limitation of the study.
Importantly, this error does not invalidate the experiments themselves: the models were trained consistently under the same implementation, and the error concerns the intended learning-rate schedule rather than the measurement or evaluation procedure. **For all following sections, mini-scale models were trained with constant learning rate schedules.**

Standard Deviation Gradient Leak

In my implementation of input and bottleneck perturbations using my custom noise weighting, the perturbation vector is scaled by the instance-wise standard deviation of the activations:

$$\hat{x} = x + \gamma^*(t)\,\mathrm{std}(x)\xi, \quad \xi \sim \mathcal{N}(0,1)$$

Due to an implementation oversight during training, `.detach()` was omitted when calculating $\mathrm{std}(x)$, leaving $\mathrm{std}(x)$ attached to the PyTorch Autograd computational graph.
Consequently, backpropagation had an unintended gradient path through the standard deviation term. Conceptually, taking the derivative with respect to an activation element x yields an additional derivative term:

$$\frac{\partial\,\mathrm{std}(x)}{\partial x} \approx \frac{x - \bar{x}}{N \cdot \mathrm{std}(x)}$$

where $N = C \times H \times W$ is the total number of elements in the activation map.
Because the standard deviation aggregates over all N elements, the partial derivative of $\mathrm{std}(x)$ with respect to any single activation element x inherently scales inversely with N. Thus, the unwanted gradient path leaked back into each activation x is weakened across the entire tensor by a factor of $1/N$.
Because N is very large for feature maps ($N = 3072$ for input activations and $N = 4096$ for bottleneck activations), I measured the relative magnitude of this leaked gradient term against the primary gradient path to be minimal across experimental runs:

- **Input Perturbations**: The measured relative magnitude of the leaked gradient term is $\approx 3.3 \times 10^{-5}$ ($\approx 0.0033\%$).
- **Bottleneck Perturbations**: The measured relative magnitude of the leaked gradient term is $\approx 9 \times 10^{-5}$ ($\approx 0.009\%$).

Since all experiments were conducted in `bfloat16` mixed precision, which has a machine epsilon of $\epsilon_{\mathrm{mach}} = 2^{-7} \approx 7.8 \times 10^{-3}$, a leaked influence on the order of 10^{-5} falls below the precision floor.

While this implementation bug is present across all experimental runs, its numerical contribution is below the precision floor and produces no practical or statistically meaningful change in the trained model weights or FID.

The following bug was caught and all corrupted data was regenerated. It has no effect on the paper but is still worth mentioning:

In-place Operation Bug

For a long time, the U-Net's sinusoidal time embedding conversion from $[0, 1]$ to $[0, 1000]$ mutated the timestep tensor `t` in-place due to using `t *= 1000` instead of `t = t * 1000`. Consequently, the same `t` tensor scaled exponentially across consecutive model calls. This had no effect on training because the timestep is reassigned at every iteration, which is why the issue went undetected. FID calculations and sampling are unaffected despite using CFG because the conditional and unconditional outputs were generated in parallel in a single call by concatenating along the batch dimension. However, evaluation metrics, such as the KL training vs. sampling distribution shift calculation, called the model multiple times before reassigning `t`, thereby corrupting the time embedding for subsequent model calls. Crucially, the corrupted outputs did not immediately appear abnormal, making this bug particularly subtle and risky. After discovering this bug, all affected plots were regenerated and the relevant sections rewritten. As a result, no content in this paper or its GitHub repository remains affected.

One last thing I must mention is a subtle bug that only causes visual changes: FID-25k and path-straightness were measured at every 80 epochs across training for most models. But due to an unwanted configuration override one of the base mini seeds was trained with measurements at every 50 epochs. To plot these alongside other measurements I interpolated the data points which slightly changes the shape of the curves but does not affect the final value at epoch 399. Figure 13 shows the interpolation error.

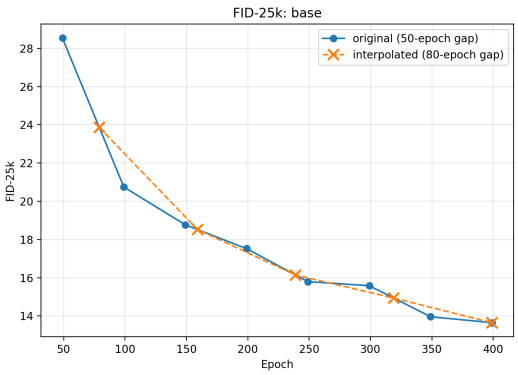


Figure 13: Original (50 epoch gap) vs. interpolated (80 epoch gap) FID-25k datapoints during training for the first of three base model seeds.

6 Preliminary Work

6.1 The Base Mini-Scale Model

In order to train many different models and evaluate them quickly under the constraint of the single-GPU compute budget, we define a mini-scale model configuration that does not achieve competitive FID scores but serves us well in frequently training and comparing different models. For the full architecture, see Appendix F.

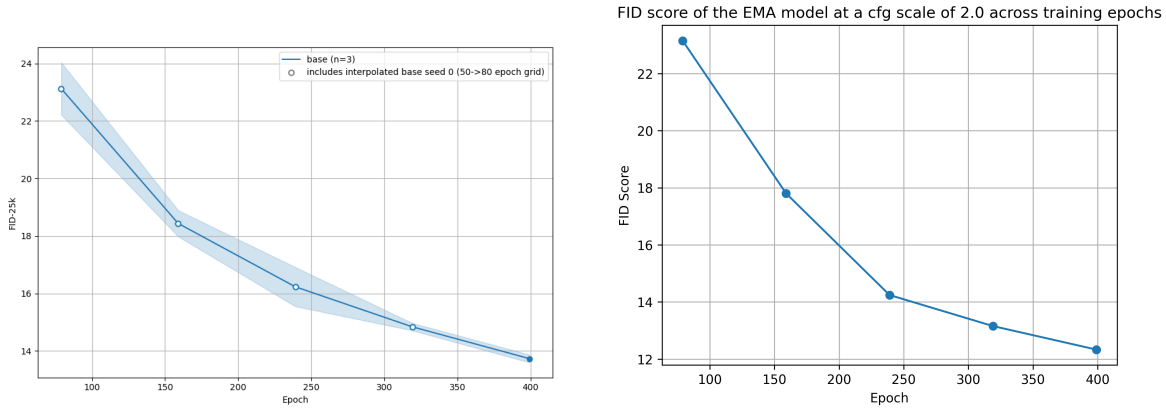
I trained the base mini-scale model once with a constant learning rate of 3×10^{-4} and once with a cosine decay from 3×10^{-4} to 10^{-5}. The base mini-scale model with cosine LR decay achieves a best FID-50k at 50 ODE steps of ≈ 12.02, the constant LR model achieves a best FID-50k at 50 ODE steps of ≈ 13.11.

Table 2: Summary of most relevant hyperparameters for the mini model scale. Note that the channel count increases as the U-Net performs spatial downsampling.

Hyperparameter	Value
Number of U-Net layers	3
Number of layers with attention layers (starting from the bottom)	1
Minimum channel count (first layer)	64
Maximum channel count (bottleneck layers)	256
EMA decay	0.9995
Total training epochs	400

Table 3: Parameter counts for the mini model scale, excluding timestep and class embeddings.

Model Part	Parameters	%
Encoder	4,271,809	39.37%
Bottleneck	3,150,850	29.04%
Decoder	3,276,353	30.19%
Total	**10,850,695**	**100.00%**



(a) Constant LR

(b) Cosine LR decay

Figure 14: FID-25k at 50 ODE steps of the base mini EMA model as training progresses. Left: constant LR (3 seeds), right: cosine LR decay. The models are likely not converged, but hardware limitations did not allow for longer training runs. A CFG scale of 2.0 was used for FID evaluations during training since the optimal scale had not been calculated yet and 2.0 is a safe default.

Figure 16: Samples from the trained base mini model with a cosine decay LR schedule at a CFG scale of 2.25. Conditioned on each of the 10 CIFAR-10 classes.

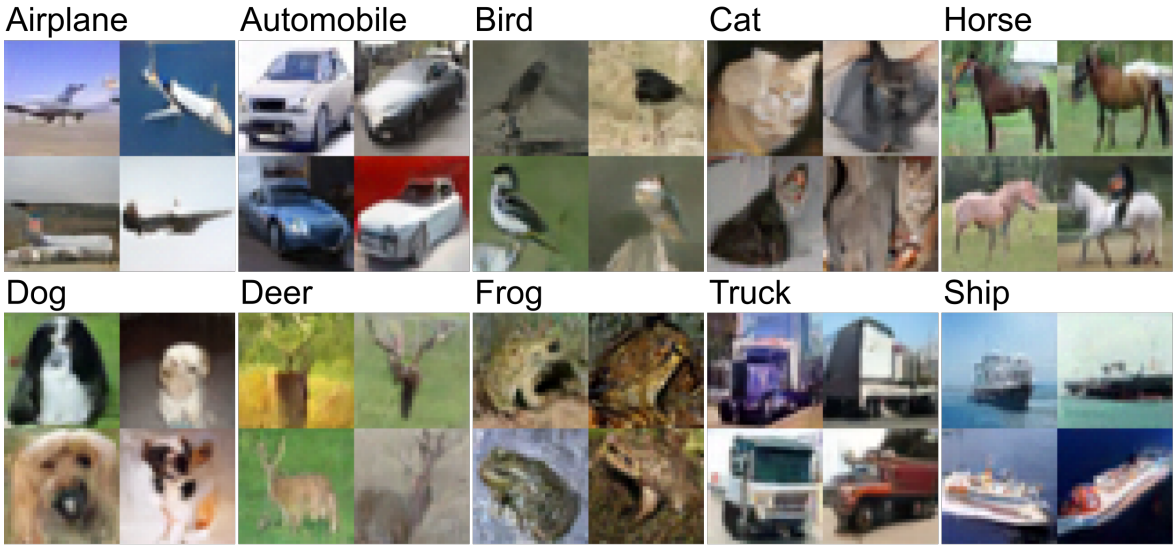


Figure 15: Samples from the trained base mini model with a constant LR. model at a CFG scale of 2.25. Conditioned on each of the 10 CIFAR-10 classes.

Because of the learning-rate schedule bug detailed in section 5.5, all experimental mini-scale models are trained with a constant learning rate and compared against the constant learning-rate base model.

6.2 Conditional Generation

For conditional CIFAR-10 generation, I condition the model on the class index for all training examples and embed the class index using an embedding table followed by a shared projection with the timestep embeddings. The combined time and class embedding vector is injected into each *ResBlock* throughout the U-Net using Adaptive Group Normalization (section 3.1 of [10]). During sampling, I use Classifier-Free Guidance [18].

The Classifier-Free Guidance scale is a hyperparameter that I empirically find on the base mini model and transfer over to all other mini-scale models.

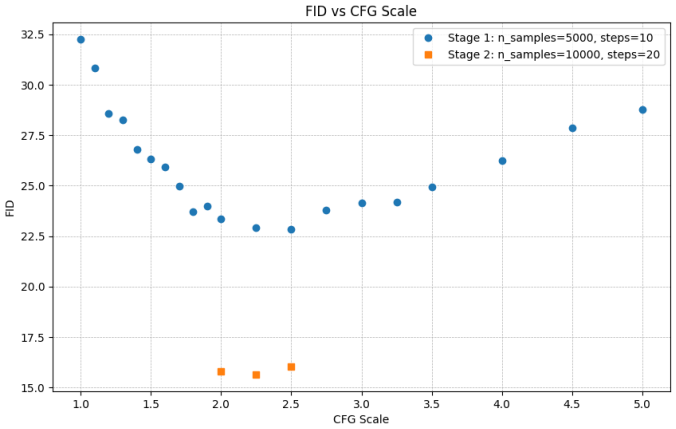


Figure 17: The Fréchet inception distance of the trained base mini model with a constant learning rate. Stage 1 used only 5k samples and refined the large initial search space into 3 candidates. Stage 2 used 10k samples and double the ODE steps to find the best out of the 3 candidates. The final best Classifier-Free Guidance scale for mini scale models with constant LR decay was found to be 2.25.

The choice of CFG scale is ultimatively "a trade-off between sample quality and diversity" (Ho et al. 2022 [18]). We can visualize this by generating a batch of samples for the same class and gradually increasing the CFG scale. This requires us to use a set seed to eliminate the influence of noise-sampling stochasticity.

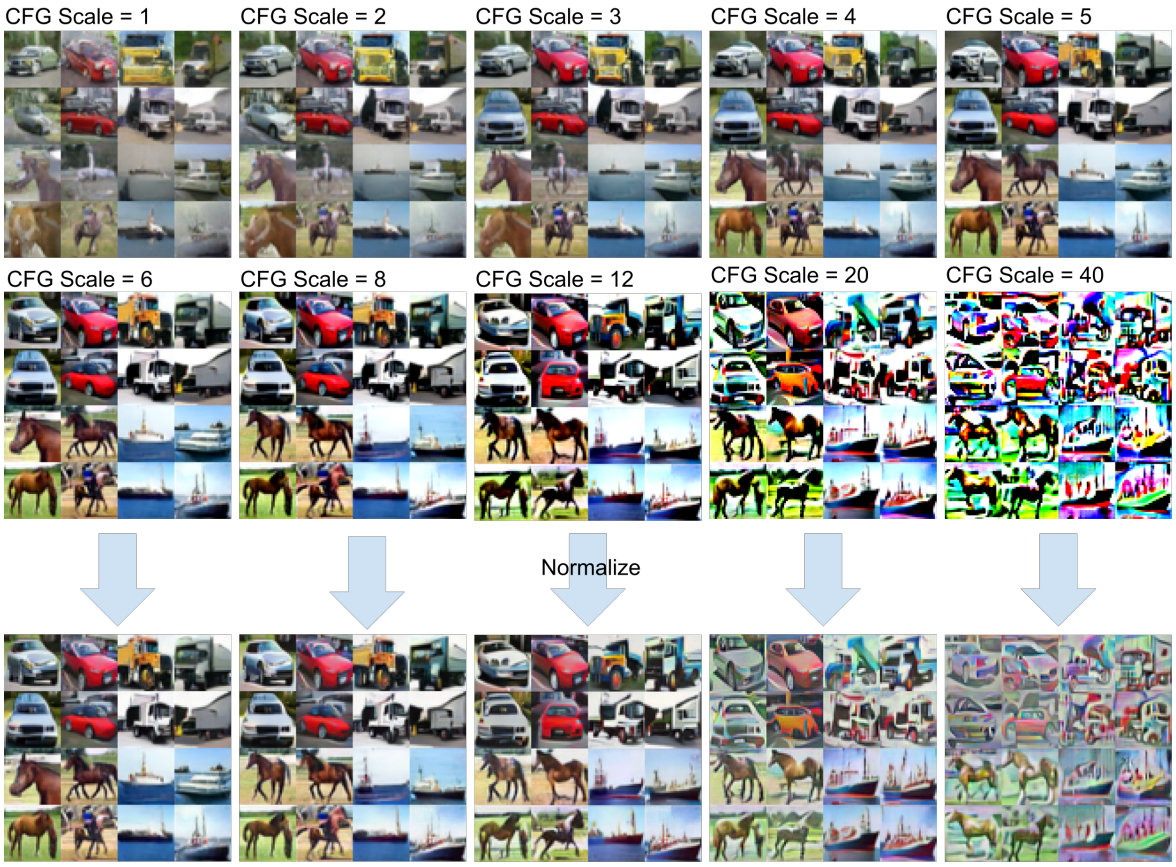


Figure 18: Conditional samples from the base mini model at different Classifier-Free Guidance (CFG) scales with a set random seed. The first two rows were clamped for visualization, the third row was normalized instead. By normalizing the values before plotting we can achieve high-quality images at CFG scales as high as 12.

Figure 18 shows how higher CFG scales achieve higher-quality images at the cost of sample diversity (e. g. the ship images all converge to a similar shape).

6.3 Input Perturbation Models

In this section we will use the metric to measure exposure bias that I described in Section 5.2.2 as well as the train to test set loss gap 5.2.6.

Constant noise scale I first tried to train a model identical to the base mini-scale model for 400 epochs while adding constant Gaussian noise:

$$\hat{x}_t = x_t + \xi, \quad \xi \sim N(0, \gamma(t)^2 I), \quad \gamma(t) = 0.1$$

Figure 19: Conditional samples of a mini model trained for 400 epochs with a constant input perturbation using a constant LR schedule. EMA weights were used to sample. All samples generated with 50 ODE steps and a CFG scale of 2.25. Notice that the global semantics and rough shapes are not bad, textures and colors are visibly noisy. Outputs clamped to [0, 1] for visualization.

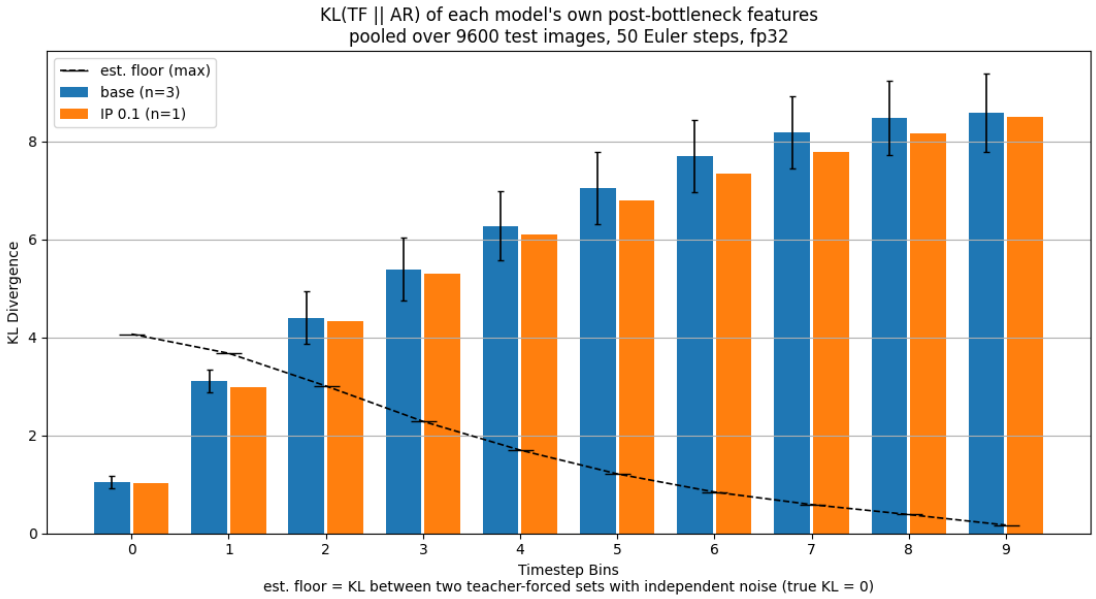


Figure 20: KL-Divergence between latent representations (U-Net bottleneck layer outputs) under teacher forcing and during autoregressive generation, 50 ODE steps, 10 bins. The base ("base_mini64") and IP 0.1 ("mini_IP64") (constant noise scale) models have identical architectures and training hyperparameters. The base model's data shows mean and standard deviation across 3 seeds. The estimated floor was computed as the KL divergence between two teacher-forced sets, with independent random seeds. It suggests the magnitude of KL-measurement noise.

Figure 20 shows how the training (teacher-forcing) vs. sampling (autoregressive sampling) KL gap increases as $t \to 1$, this confirms that base and constant noise scale IP models experience measurable

distribution shift due to being fed its own outputs rather than ground-truth points on the path $x_t = \psi_t(x_0)$. The model trained with a constant noise scale could have very slightly reduced the distribution shift KL gap, but its values lie within one standard deviation of the base model re-seeds which is why I interpret this plot as showing no clear difference. In addition, this model generates visibly noisy images (Figure 19). This result leads me to think that I need a timestep weighting function for the scale of the injected noise because I empirically observed that adding constant input perturbation noise across all timesteps resulted in a model that was unable to generate the fine-grained details at the end of the sampling chain.

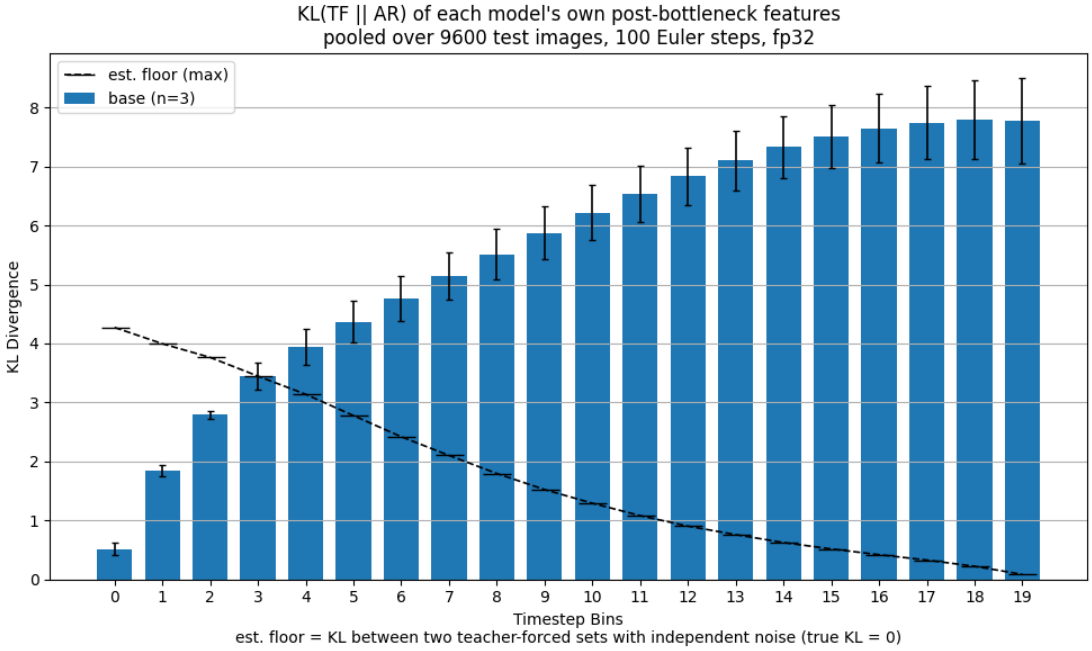


Figure 21: KL-Divergence between latent representations (U-Net bottleneck layer outputs) under teacher forcing and during autoregressive generation for the 3 trained base mini-scale models (no input perturbation at all). Standard deviation is reported across 3 seeds. 100 ODE steps, 20 bins. The 50 ODE steps, 10 bin equivalent is the blue data in figure 20.

Figure 21 shows how the shape of the KL gap across timestep bins stays roughly the same when using more ODE steps and finer timestep binning. The distribution shift increases with time in both cases.

Time-dependent noise schedule Initially, the intention was to fit a curve to the resulting shape and use it as a weighting function for the noise used for input perturbations since this would more accurately model the sampling distribution shift as noise during training. However, that would not solve the noisy-samples problem. I could not establish a theoretically based, reproducible shape and therefore used a constant function with exponential decay.

$$\gamma^*(t)_{\gamma_0,k,t_1,t_2} = \begin{cases} \gamma_0, & t \le t_1 \\ \gamma_0 e^{-k(t-t_1)}, & t_2 > t > t_1 \\ 0, & t \ge t_2 \end{cases}$$

Where $\gamma_0 = 0.1$, $t_1 = 0.7$ and $t_2 = 0.9$. The parameter k was set to 16 so roughly the last 20% of timesteps have $\gamma^*(t) < 0.2\gamma_0$. The constant region up until $t_1 = 0.7$ is chosen to cover the high exposure-bias region, while the decay preserves late-trajectory sensitivity. This schedule has a small discontinuity at t_2 to ensure the noise scale is exactly 0 at the very end, it is negligible relative to γ_0.

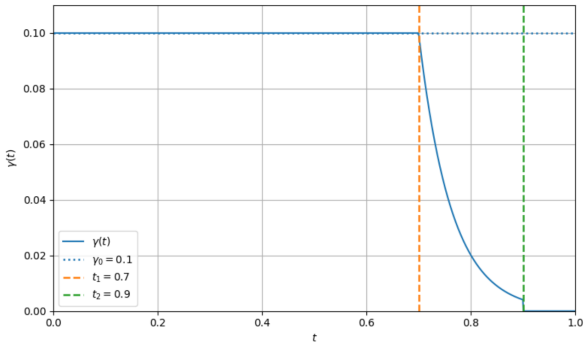


Figure 22: A plot of the proposed noise scale function $\gamma^*(t)$ using $\gamma_0 = 0.1$, $t_1 = 0.7$, $t_2 = 0.9$ and $k = 16$.

The average per-sample standard deviation of x_t across one entire epoch is ≈ 0.57 and lies inside $[0.52, 0.61]$. Using our new noise scale weighting function $\gamma^*(t)$ and also weighting by $\mathrm{std}(\mathrm{x_t})$ we add input perturbations as follows:

$$\hat{x}_t = x_t + \mathrm{std}(\mathrm{x_t})\,\gamma^*(t)\xi, \quad \xi \sim \mathcal{N}(0, I)$$

The noise is train-only, there is none at sampling time. The standard deviation is computed per-sample, not per-batch.

Figure 23: Conditional samples of a mini model trained for 400 epochs with the custom input perturbation scale schedule $\gamma^*(t)$ and $\gamma_0 = 0.1$. EMA weights were used to sample. All samples generated with 50 ODE steps and a CFG scale of 2.25. Outputs clamped to [0, 1] for visualization.

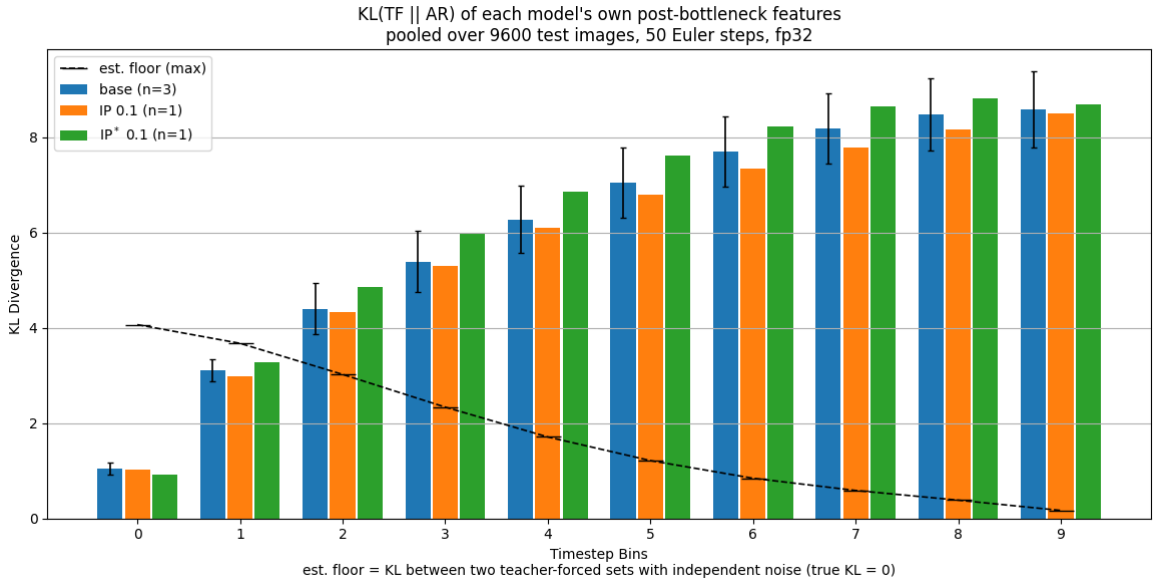


Figure 24: KL-Divergence between latent representations (U-Net bottleneck layer outputs) under teacher forcing and during autoregressive generation, 50 ODE steps, 10 bins. All 3 models have identical architectures and hyperparameters (except for the input perturbations). The base model's measurement series reports mean and standard deviation across 3 seeds. The base and constant noise models in this figure are identical to the ones in figure 20, the random seed for the set of measurements is not the same.

To match the relative noise scale that performed best in the bottleneck IP experiment (section 7), I trained another mini-scale IP model with $\gamma_0 = 0.2$. This model's final FID-50k ended up being 21.8208. Because of this model's poor performance I use IP* 0.1 as the representative model for standard input perturbation and use this IP* 0.2 model only for comparison in 7. I am aware that since FID increased with the noise scale, the optimal γ_0 value for IP* is likely in the range $[0, 0.1]$. Due to my compute budget, I did not plan IP* training runs across values for γ_0.

IP Model Specs

Model	IP type	IP scale γ_0	noise weight	FID 50k
base (summary)	none	-	-	13.2442 ±0.1698
IP 0.1	network input	0.1	1	94.0929
IP* 0.1	network input	0.1	$\text{std}(x)\gamma^*(t)$	13.4081
IP* 0.2	network input	0.2	$\text{std}(x)\gamma^*(t)$	21.8208

After training IP* 0.1 I successfully trained an IP model that does not suffer from noisy samples (figure 23) but that still does not measurably alleviate exposure bias relative to the base model. This shows that in my case, the proposed noise scale schedule $\gamma^*(t)$, paired with standard deviation weighting yields superior results as opposed to a constant noise schedule. It should be said that this model likely did not fully converge (see figure 14), I attribute this to insufficient training epochs and potentially a high EMA decay value (table 10). The final FID-50k value of the IP* 0.1 model lies within one standard deviation of the base model's value, thus I consider change in FID insignificant.

Generalization To evaluate the generalization of base, IP 0.1 and IP* 0.1 models, we can plot the gap between average test set and test set loss. The metric is detailed in Section 5.2.6.

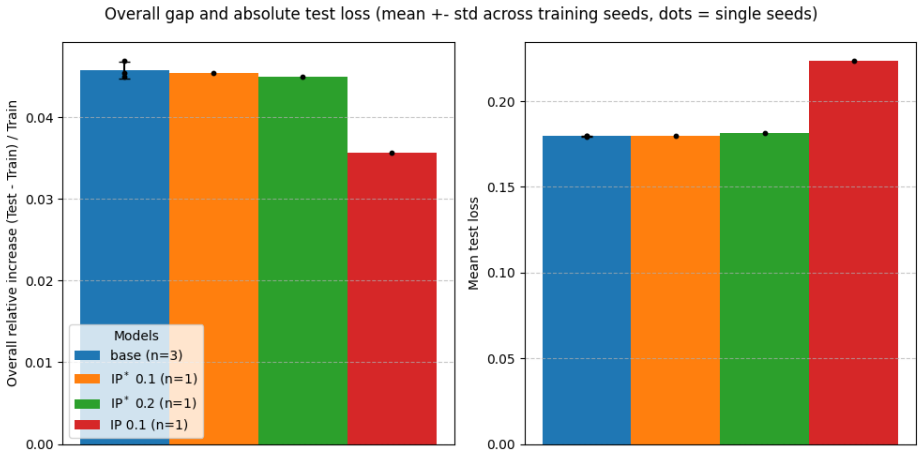


Figure 25: Training vs. test set loss gap (left) and absolute test loss (right). Timesteps were sampled uniformly. Averaged across 9600 images and 10 repeated evaluations of those images. Sample standard deviation across 3 seeds is reported for the base model.

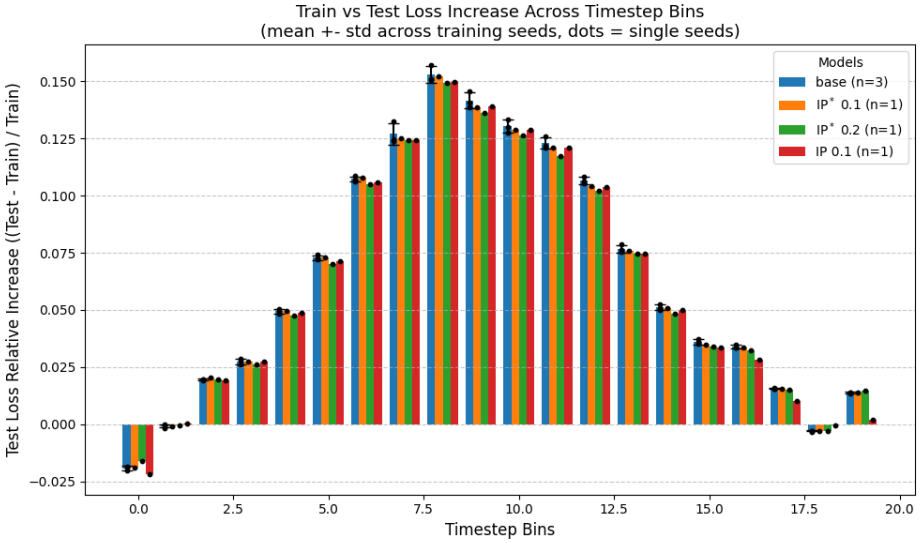


Figure 26: Training vs. test set loss gap across timestep bins. Timesteps were sampled uniformly. Averaged across 9600 images and 10 repeated evaluations of those images. Sample standard deviation across 3 seeds is reported for the base model.

Figure 25 shows how IP* 0.1 and IP* 0.2 models do not have a measurably better generalization than the base model. The constant noise model, IP 0.1, does have a lower train-test loss gap but its test loss is higher. An interesting observation we can make here is that despite having a FID-50k value that is almost 10 times higher, IP 0.1's test loss is not that much higher, which supports my earlier claim that the noisy samples are a consequence of not being able to generate fine details at the end of the sampling chain. All four models have the largest train vs. test loss increase at $t \approx 0.5$. This is consistent with previous research [31]: "Assume the integration timestep is uniformly sampled, $t \sim U(0, 1)$, and the target data variable x1 is standardized such that its expectation $\mu_{x_1} = 0$ and covariance $\sigma_{x_1} = I$. Under these structural conditions, the expected memorization risk across the entire continuous trajectory is strictly upper-bounded by the memorization evaluated at the trajectory midpoint. That is, the **maximal memorization occurs at t = 0.5**." My results agree despite using a logit-normal timestep distribution and my data not being exactly zero centered (see the dataset Section 5.1). However, the changes between models are too small relative to the base model's standard deviation to make meaningful observations.

Partial derivatives In 3.3, I referenced [5] for training with additive noise being asymptotically equivalent to penalizing large values inside of the network's Jacobian $\frac{dy}{dx}$ where x is the input and y is the output. To empirically evaluate how this applies to my situation, I approximated the average partial derivatives seen in table 1 for the trained base and IP* 0.1 models using the metric detailed in 5.2.3.

Table 4: Comparison of the derivative results for the base, IP 0.1 and IP* 0.1 models. The reported uncertainties correspond to standard deviations computed over 5 random seeds. All results were rounded to 2 decimals since bf16 only has 3 significant decimal digits of precision. Here, x is the network input, y is the output and h are the U-Net's pre-bottleneck activations. Since I use Jacobian matrix Frobenius norms, the data represents absolute values, i. e. the change in magnitude, not the actual numerical change.

Model	$\mathrm{d}y/\mathrm{d}x$	$\mathrm{d}y/\mathrm{d}h$	$\mathrm{d}h/\mathrm{d}x$
base	67.92 ± 0.30	3.34 ± 0.01	29.98 ± 0.04
IP 0.1	60.68 ± 0.14	3.13 ± 0.01	28.48 ± 0.05
Change relative to base model	$\mathbf{-10.65\%}$	$\mathbf{-6.25\%}$	$\mathbf{-4.99\%}$
IP* 0.1	68.07 ± 0.30	3.16 ± 0.01	30.21 ± 0.05
Change relative to base model	$\mathbf{+0.22\%}$	$\mathbf{-5.53\%}$	$\mathbf{+0.77\%}$

I am aware that by comparing two EMA model weights with a single training seed, these results are not generally representative. In accordance to my Bishop-motivated hypothesis in 3.3, the model trained with a constant noise schedule (IP 0.1) did learn a more contractive mapping (smaller dy/dx). The model trained with my custom noise weight schedule and standard deviation weighting (IP* 0.1) does not show any measurable change in dy/dx relative to the base model, I attribute this to the fact that it injects less noise in the absolute since it weights by $\mathrm{std}(x_t) \approx 0.57$ and decays to 0 as $t \to 1$.

Regarding my experiments with standard input perturbation at a constant noise scale, I make the following observations:

1. **Effect on** $\mathrm{d}y/\mathrm{d}x$**:** IP *measurably decreased* the norm of the network's Jacobian when noise was injected at a constant, larger scale. I assume that the push towards a "contractive" mapping is near unnoticeable when effectively injecting less noise, which is what I did via my custom noise weighting.

2. **Effect on** $\mathrm{d}y/\mathrm{d}h$**:** IP *measurably decreased* the norm of the partial derivative of the output y with respect to the pre-bottleneck latent representations h for both constant and custom noise weights. I hypothesize that this follows as a natural consequence of training with perturbations, which might make the encoded representations h have a lower relative clean signal content.

3. **Effect on** $\mathrm{d}h/\mathrm{d}x$**:** IP *measurably decreased* the partial derivative of the encoded state h with respect to the input when noise was injected at a constant, larger scale. This is analogous to my assumption in (1.).

4. **Effect on the KL-Divergence of post-bottleneck activations under teacher forcing and during autoregressive sampling:** As seen in figure 20, IP did not measurably decrease the training vs. sampling KL gap.

5. **Consequences of a constant noise scale:** The model that was trained using IP with a constant noise scale schedule $\gamma(t) = 0.1$ yields semantically good samples with very weak textures and colors (see figure 19). This suggests that it struggles with fine-grained details [29] at the end of the sampling chain. My proposed noise scale function $\gamma^*(t)$, paired with standard deviation weighting fixed the noisy image problem as far as my experiments went.

These findings have empirically shown us that constant noise weight IP (IP 0.1) seems to lead to a more "contractive mapping" while producing noisy, high-FID samples whereas standard-deviation weighted, time-decaying noise weight IP (IP* 0.1) does not affect the models partial derivatives in the same way but fixes the noisy-sample problem (Table 4). We found that IP does not measurably reduce the teacher forcing vs. sampling distribution KL gap (figure 20).

Path straightness Out of curiosity regarding the straightness of the flow paths, I tracked the path straightness (effective path length over theoretical straight path length) across training for the base and IP mini models.

$$\text{Straightness} = \mathbb{E}_{x_0,x_1}\left[\frac{\int_0^1 \|v_\theta(x_t,t)\|\ dt}{\|x_1 - x_0\|}\right]$$

A path straightness of 1.0 would be the shortest possible (linear) path. Higher values indicate less "efficient" paths.

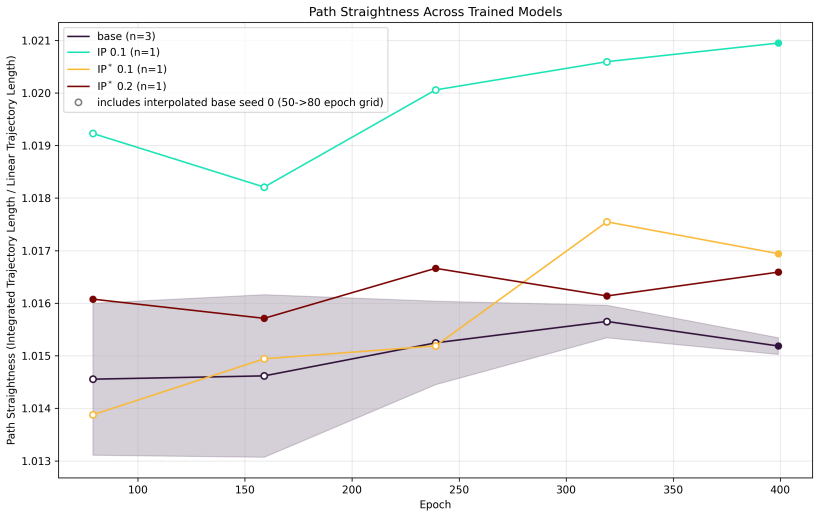


Figure 27: Path straightness across training for base, constant IP and custom schedule IP models. Mean and standard deviation across 3 seeds are reported for the base model. Evaluated across 32 images and 8 noise draws per image. 100 ODE steps were used to prevent rough step sizes from affecting the trajectory shape.

In my single-run experiments the models using my custom noise schedule $\gamma^*(t)$ show straighter paths across training than the constant noise scale model (IP 0.1). Since the relative increase is only very minimal, I cannot make empirical claims here.

Key Takeaway I: Standard input perturbation does not measurably reduce the teacher-forcing/autoregressive sampling KL gap in my flow-matching setting. Relating to my hypothesis motivated by Bishop [5], I found that the norm of the networks Jacobian dy/dx decreases for constant noise weight IP (IP 0.1) but stays roughly the same for the IP model that uses my custom noise weighting (IP* 0.1).

Key Takeaway II: In my setting, a naive constant-weight noise injection destroys fine-detail generation and yields noisy samples. My proposed time-dependent schedule that decays to zero late in the trajectory restores sample quality.

6.4 Model Initialization Activations

Before and during this project I built `ModelAnalyzer`: a hook-based module-level distribution inspection interface for PyTorch. The code can be found in the project GitHub repository. Data was collected at regular intervals during training, the wall-time slow down caused by this is negligible. This section uses the `ModelAnalyzer` to inspect some of the model activation statistics across training.

These two plots show that the model's forward pass activations tend to have higher standard deviations and KL-Divergences to unit Gaussians in early training. Both metrics quickly decrease and stay almost constant for the rest of training. I observed a similar behaviour across base, IP 0.1 and IP* 0.1 mini-scale models. One key observation is that for the bottleneck layers (between last encoder

layer and first decoder layer), the early training high standard deviation and KL-to-unit-Gaussian phenomenon is less noticeable, if at all. In accordance with sections 3.1, 2.5 and Appendix A this suggests that the U-Net bottleneck is functionally different.

7 Experiment 1: Bottleneck Input Perturbation

7.1 Setup

I want to investigate how adding input perturbations to the pre-bottleneck hidden activations (see figure 28) affects the model's generative quality, its partial derivatives and training vs. sampling KL-gap. To do so, I run the U-Net encoder to get the intermediate latent representation h and then pass the perturbed version $\hat{h}$ into the U-Net bottleneck and decoder layers.

$$\hat{h} = h + \text{std}(h)\gamma^*(t)\xi, \quad \xi \sim \mathcal{N}(0, I)$$

I choose to keep the standard deviation weighting and time-dependent weighting function $\gamma^*(t)$ to ensure that the noise's relative significance isn't affected by the specific instance of x_t and especially so this experiment injects noise in the exact same way as I did for the input perturbation model in 6.3. The noise is train-only, there is none at sampling time. The standard deviation of h is computed per-sample, not per-batch.

Search space For my experiments with standard input perturbations I used a standard value for γ_0 of 0.1. For the bottleneck input perturbation experiments I define a search-space of values for γ_0:

$$\gamma_0 \in \{0.025, 0.05, 0.1, 0.2, 0.4\}$$

I chose these values by first settling on a total of 5 runs and roughly spacing the values around 0.1, which is what chapter 6.3 and DDPM-IP [25] use. To evaluate the generative quality of these models I will use FID-25k vs. training epoch graphs and evaluate their final FID-50k.

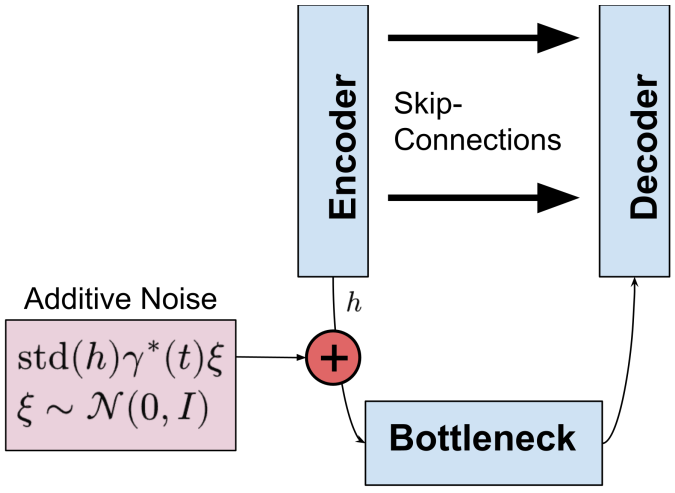


Figure 28: A scheme of the bottleneck input perturbation setup in practice. Made by me in Google Drawings.

7.2 Hypothesis

My hypothesis for this experiment, as seen in section 4, is the following:

Hypothesis 1. ***Bottleneck Input Perturbation:*** *Encouraging the bottleneck-to-output mapping to be less sensitive to perturbations in its input representation, analogous to the effect Bishop (1995) shows for input-level noise, and to DDPM-IP [25], will reduce the teacher-forcing (training)/autoregressive (sampling) distribution gap in the bottleneck's activations, and may improve generation quality relative*

to an unperturbed baseline. Concretely, I inspect the exposure-bias gap, the model's partial derivatives and training set vs. test set loss.
Falsification: *This hypothesis is evaluated on two independent claims. (1) Representational claim: if bottleneck IP does not show any statistically meaningful change in the used metrics, this claim is falsified regardless of generation quality. (2) Quality claim: if the representational claim holds but FID shows no improvement over baseline, I conclude the representational change does not translate into improved generation quality in this setting. The two claims are interpreted separately.*

7.3 Results

Keep in mind that all results in this section come from a measurement series where each model configuration was only trained once (single-seeded). Consequently, any observations may be influenced by random initialization variance. Some metrics detailed in Section 5.2.1 are not shown in this section because the results were both statistically insignificant and not directly relevant to the experiment. Statistically insignificant but relevant metrics are included.

7.3.1 FID Results

Due to compute limitations, models were trained for a fixed number of epochs rather than to full convergence. Consequently, the FID scores reported here reflect performance at a specific training checkpoint and do not distinguish convergence speed from final generative quality. For the sake of evaluating the experiment, I treat the final FID score as a proxy for generative quality. See Section 9.2 for a discussion on the experimental validity of this study.

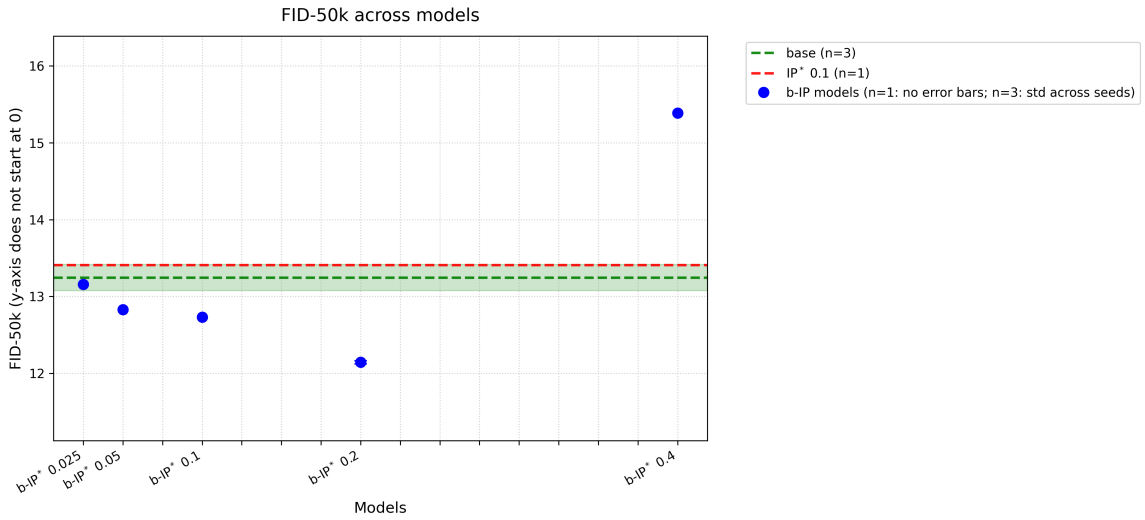


Figure 29: Comparison of final FID-50k values across different model configurations: the base model (green bar), the model with IP using a custom schedule with $\gamma_0 = 0.1$ (red bar), and varying bottleneck IP scales (blue data points). All models share the same architecture and training hyperparameters, they only differ in the type of perturbation used. For the base and b-IP* 0.2 models, means and standard deviations are reported across 3 seeds. The standard deviation of the b-IP* 0.2 is very small and thus not very well visible in the graph. The exact number is in table 5.

We observe that (see figure 29), up until a certain point, a higher bottleneck input perturbation scale γ_0 correlates with lower FID-50k. From my measurements, I can conclude that the γ_0 value for bottleneck IP that minimizes FID-50k lies somewhere in the range $(0.1, 0.4)$ since the best among tested was 0.2, bracketed by 0.1 and 0.4. Intuitively, it makes sense for the FID-related benefits of bottleneck IP to fade as the IP scale increases since the relative portion of clean signal in the pre-bottleneck activations decreases. Furthermore, I conclude that my approach of applying IP to pre-bottleneck activations might be more beneficial FID-wise for Flow-Matching U-Nets than applying IP to the network's input as in [25] since the standard IP model with $\gamma_0 = 0.1$ increases FID-50k relative to the base model while a model using bottleneck IP with the same γ_0 value decreases FID-50k by

3.87% relative to the base model. For $\gamma_0 = 0.2$, the gap is even larger and holds across 3 seeds (IP* 0.2 vs. b-IP* 0.2 in table 5). However, this is not a fair comparison, since the γ_0 search space was much larger (5) for b-IP* than it was for IP*. IP* with $\gamma_0 \sim [0, 0.1]$ might yield different results.

Table 5: A *lower* FID-50k value indicates that the distribution of generated samples is closer to the real data distribution, i.e., *better* generative quality.

	base	IP* 0.1	IP* 0.2	b-IP* 0.025	b-IP* 0.05	b-IP* 0.1	b-IP* 0.2	b-IP* 0.4
FID-50k	13.2442 ± 0.1698	13.4081	21.8208	13.1554	12.8298	12.7322	12.1410 ± 0.021	15.3890
Change relative to base	0%	+1.24%	+64.76%	-0.67%	-3.13%	-3.87%	-8.33%	+16.19%

We also observe that the bottleneck-IP approach is robust to larger relative noise scales in comparison to the standard network-input IP, however, an equal γ_0 does not mean the same effective noise level since one is scaled by std(x_t) at the input and the other by std(h) at the bottleneck. I found that the γ_0 value that minimizes FID-50k lies in the range [0.1, 0.4] (see table 5 b-IP* 0.1, b-IP* 0.2), whereas increasing γ_0 from 0.1 to 0.2 is enough to make FID-50k explode for network input IP models (see table 5 IP* 0.1, IP* 0.2).

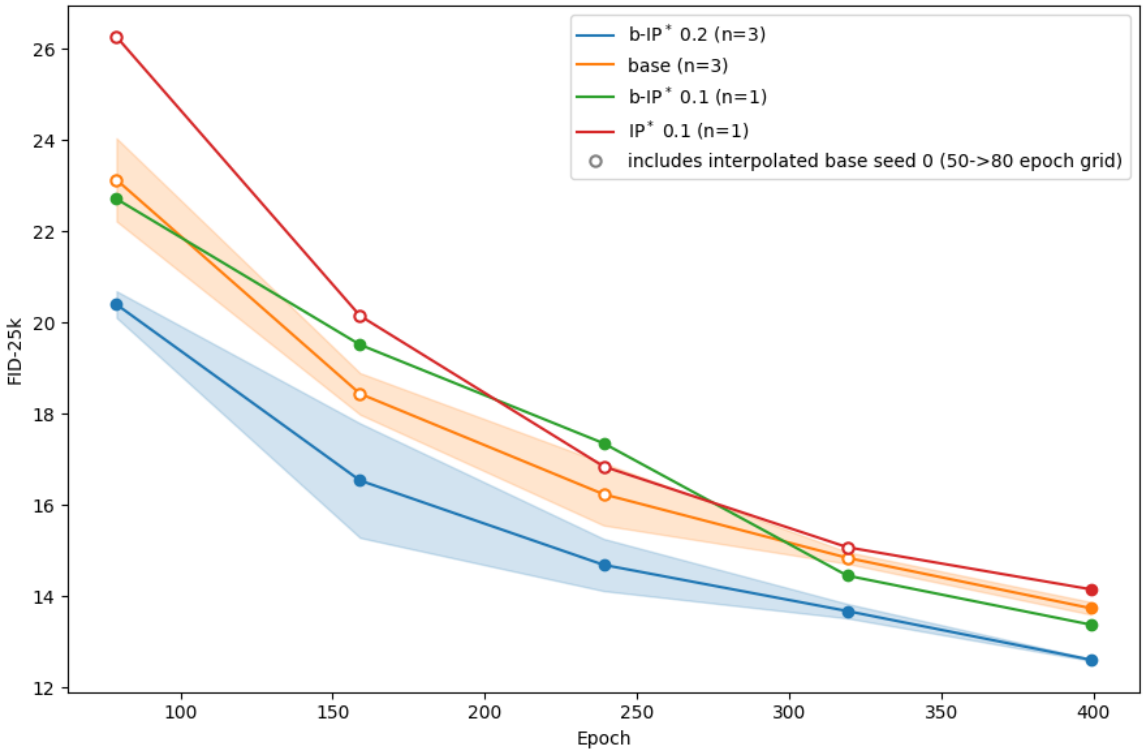


Figure 30: FID-25k of models across 400 epochs of training. IP* 0.1 and base model measurement series had 8 discrete data points by mistake which were interpolated to get 5. Base and IP* 0.2 models report mean and standard deviation across 3 seeds.

While the b-IP* 0.1 model only outperforms the base model in terms of FID-25k at the end of training, the b-IP* 0.2 model achieves a lower FID-25k value than the base model across the entirety of training.

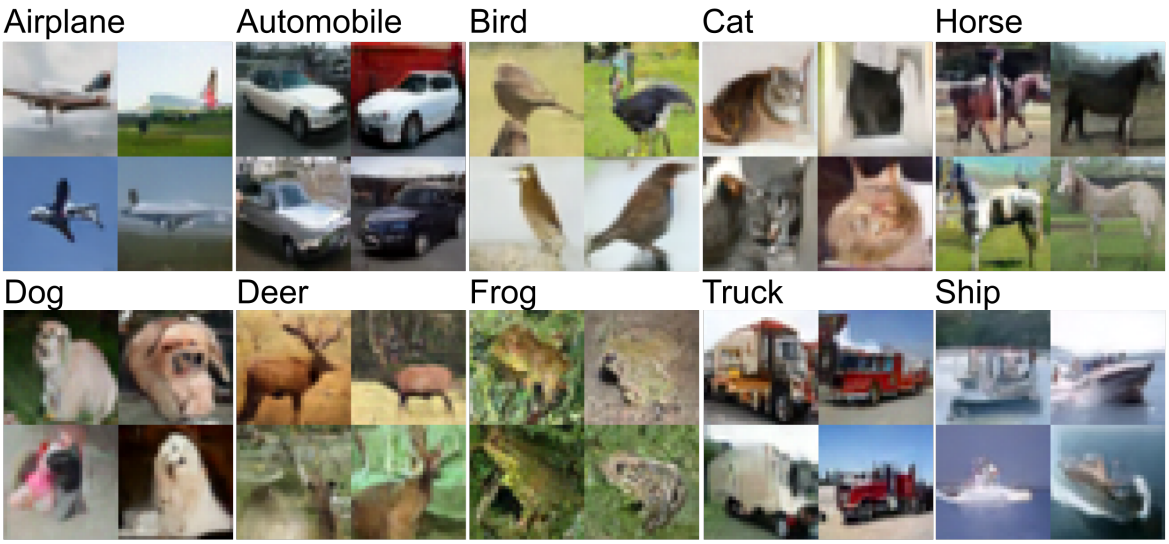


Figure 31: Conditional samples of the b-IP* 0.2 mini-scale model that was trained for 400 epochs with the custom input perturbation scale schedule $\gamma^*(t)$. EMA weights were used to sample. All samples generated with 50 ODE steps and a CFG scale of 2.25. Outputs clamped to [0, 1] for visualization.

7.3.2 Teacher-Forcing vs. Autoregressive Generation KL Gap Results

To evaluate if models trained with input perturbations successfully alleviate the exposure bias problem (see motivation 3.3) we measure the kl-divergence of post-bottleneck activation distributions under teacher forcing (training) vs. under autoregressive sampling. To do this, we encode 100 batches (9'600) images from the test set with teacher-forced ground truth x_t inputs and autoregressively generated x_t inputs and compute timestep-binned kl-divergence by approximating both distributions as multivariable gaussians. When encoding, we perform global average pooling across spatial dimensions. This metric is detailed in Section 5.2.2.

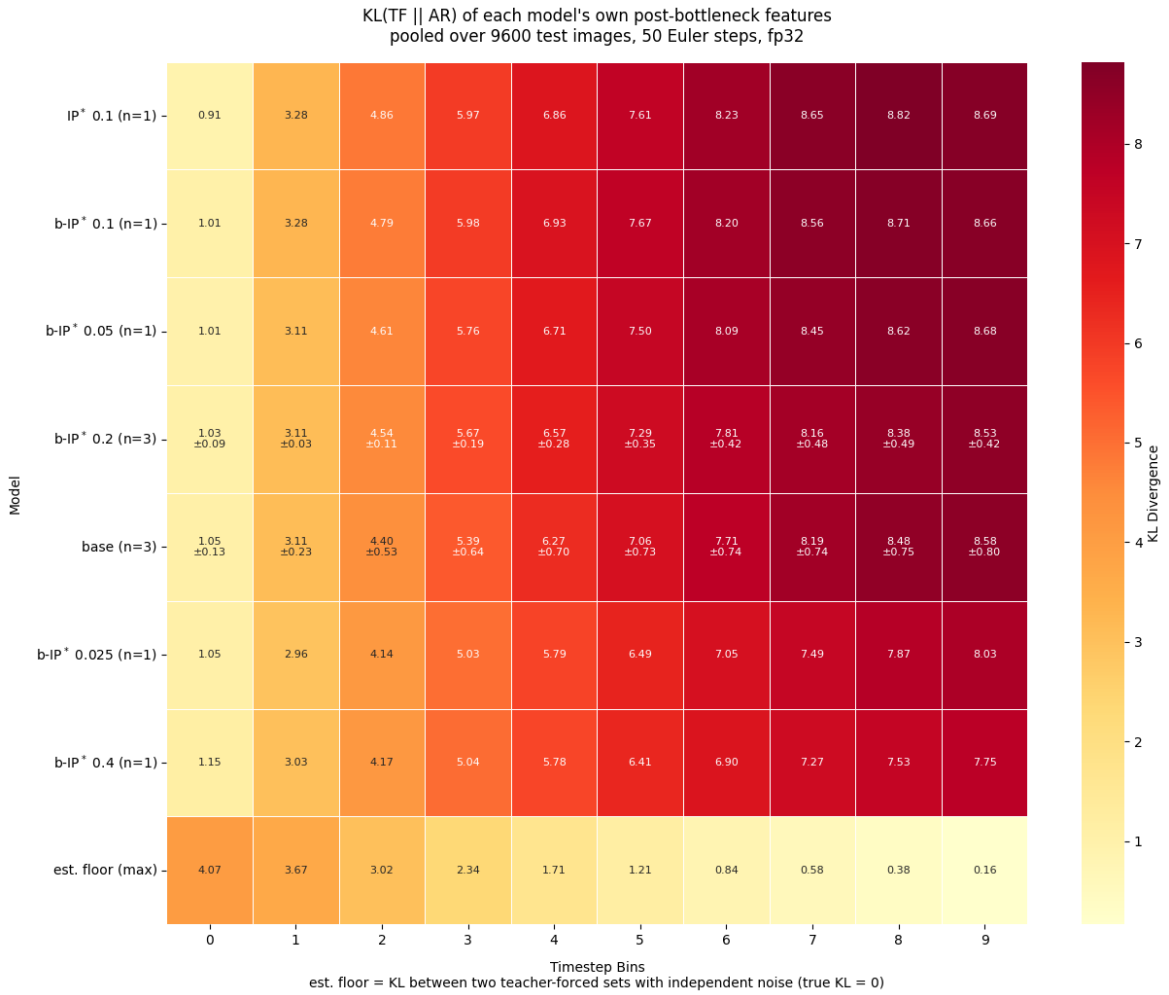


Figure 32: Heatmap of the timestep-binned TF vs. autoregressive sampling KL gap for various models. The rows are ordered by mean: the bottom row has the lowest mean KL gap across timesteps. Values were averaged across 100 batches (9'600 images). Base and b-IP* 0.2 models report the mean and standard deviation across 3 seeds.

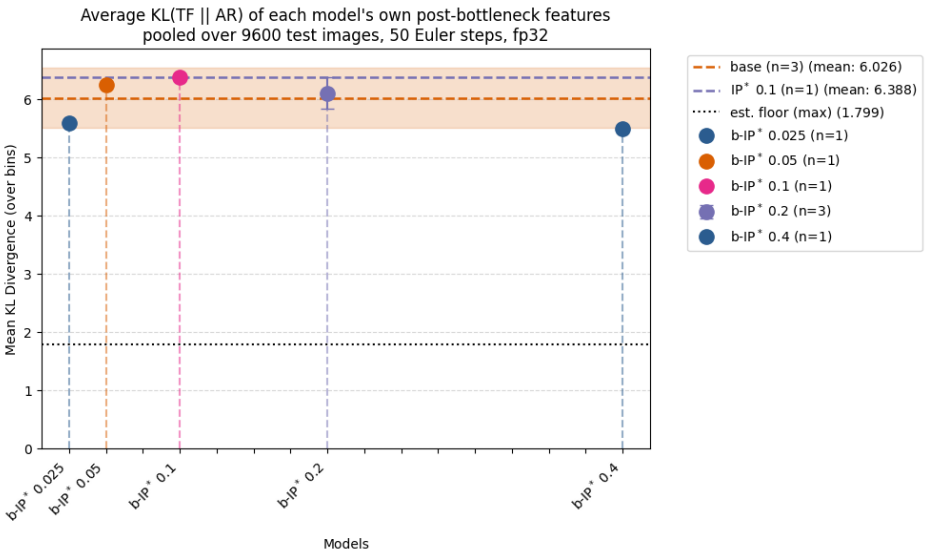


Figure 33: KL-gap averaged across timestep bins for the b-IP* models. The binned data was averaged across 100 batches (9'600 images).

Figures 32 and 33 show some mundane results: all data points lie few standard deviations away from the base model. This suggests that similar to regular IP, bottleneck-IP models do not have a measurable effect on the training vs. sampling KL gap. Figure 33 supports this supposition. Ultimately, I can't explain the measurable improvement in FID-50k as reduced exposure bias.

7.3.3 Test Set Loss Results

Using the metric defined in Section 5.2.6, I calculated the train set vs. test set loss gap and absolute test set loss for the base model and the b-IP* models.

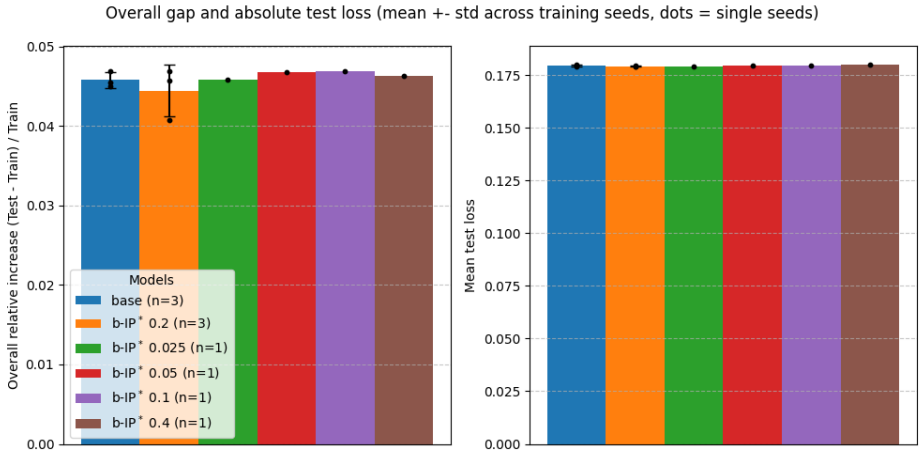


Figure 34: Average relative loss increase from train to test set across timesteps (left) and average test set loss (right). Base and b-IP* 0.2 models report mean and standard deviation across 3 seeds. The loss ratio was calculated using the average losses across 9'600 images and itself averaged across 10 evaluation repetitions.

Figure 34 shows how all b-IP models have an almost identical mean test loss despite having different FID-50k values. This suggests that the grade to which the desired distribution was learned, the generative quality of a model, does not correlate with test set MSE CFM loss.

7.3.4 Model Derivative Results

In this section we analyze the partial derivatives of the models. We use x for the model input, h for the pre-bottleneck hidden activations and y for the model output. The method used to obtain a scalar metric for these partial derivatives is described in section 6.3.

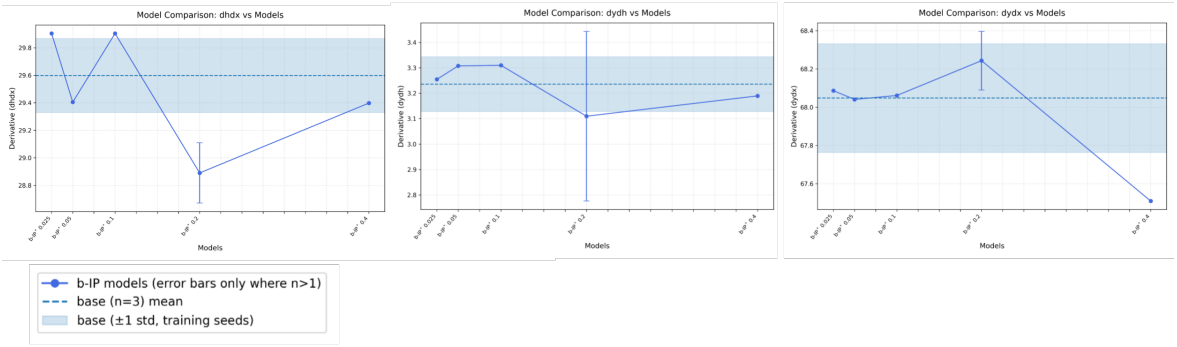


Figure 35: Partial derivatives of b-IP* models. The metric is detailed in Section 5.2.3. Data points are averaged over 100 test-set batches (9'600 images), reported standard deviations were calculated across 3 training seeds for base and b-IP* models.

The plots in figure 35 show that the changes in the three measured model derivatives relative to the base model are minimal, in contrast to my network-input perturbation models 6.3. Regardless, I make the following observations:

- The partial derivative dh/dx of latent representations h with respect to the input x is $\approx 3.7\%$ smaller for b-IP* 0.2 model than for the base model. However, the behaviour is irregular across noise scales which leads me to suspect that it has no connection to the actual bottleneck IP and is caused by weight initialization randomness.
- The partial derivative dy/dh of the output y with respect to the (pre-bottleneck) encoded state h is consistently within one standard deviation of the baseline for b-IP* models relative to the base model. This does not align with my initial Bishop-motivated [5] hypothesis (section 3.3 table 1) that the bottleneck + decoder subnetwork would learn a more contractive mapping. I hypothesize that this is because the skip-connections still provide the decoder with unperturbed signal.
- The U-Net's Jacobian dy/dx is irregular across noise scales. I can only suppose that b-IP* does not have a measurable effect on this metric.

7.4 Summary & Empirical Claims

I defined a new type of input perturbation that injects noise into the pre-bottleneck activations of the U-Net (Figure 28). I trained 5 models using bottleneck IP with my custom noise weight schedule $\gamma^*(t)$ and standard deviation weighting (see Section 6.3) and determined the optimal noise scale γ_0 within my search-space (in terms of FID-50k) to be 0.2. My b-IP* 0.2 model achieves a final FID-50k value that is 8.33% lower than the base model's (Table 5). I measured the KL-divergence of post-bottleneck (encoded) model distributions under teacher forcing vs. autoregressive sampling to measure the exposure bias problem (see motivation 3.3) and conclude that my b-IP* models do not measurably alleviate the exposure-bias problem. Finally, I analyzed the partial derivatives of various perturbed models and found mundane results that were largely within the base model's standard deviation. Both in my preliminary work 6.3 as well as this experiment, I was not able to make many meaningful statements on the relationship between perturbed training and the model's other derivatives.

Ultimately, the bottleneck noise injection experiment allows me to make the following empirical claims for Flow-Matching U-Nets:

1. **Generation Quality**: Bottleneck IP paired with my custom noise weight schedule allows me to use larger relative noise scales than standard IP without hurting generative quality. At suitable

relative noise scales γ_0, b-IP* achieves lower FID-50k relative to the base model whereas standard IP* does not improve generation quality (figure 29).

2. **Exposure Bias**: Bottleneck input perturbations do not measurably reduce the training vs. sampling KL gap.
3. **Model Derivatives**: The model's partial derivatives mostly appear to not relate to the relative noise scale γ_0 used for b-IP* in a predictable way (figure 35)
4. **Cause**: The gain in generative quality that b-IP* provides seems to not be caused by exposure bias alleviation nor generalization since both the training vs. sampling KL gap and the train set to test loss ratio do not measurably change relative to the base models.

These claims were not tested at large scales and are thus not guaranteed to be generally applicable.

At the end of this section, I answer my hypothesis from section 7.2:

(1) Representational claim: not supported (falsified in my scope). Bottleneck-IP does not reduce the training vs. sampling KL-gap, which suggests that it does not alleviate the exposure bias problem. Bottleneck-IP does not alter the model's partial derivatives relative to baseline in a way that enables me to make meaningful statements.

(2) Quality claim: supported. Bottleneck IP measurably improves FID-50k relative to the baseline at suitable relative noise scales ($\gamma_0 \in (0.1, 0.4]$), the best γ_0 among tested is 0.2, bracketed by 0.1 and 0.4. The 8.33% improvement in FID-50k is the best of 5 selected tested values for γ_0. It must be clarified that I was not able to determine what causes this improvement in generative quality using my chosen metrics.

b-IP* Model Specs

Model	IP type	IP scale γ_0	noise weight	FID 50k
b-IP* 0.025	bottleneck	0.025	std$(h)\gamma^*(t)$	13.1554
b-IP* 0.05	bottleneck	0.05	std$(h)\gamma^*(t)$	12.8298
b-IP* 0.1	bottleneck	0.1	std$(h)\gamma^*(t)$	12.7322
b-IP* 0.2 (3 seeds)	bottleneck	0.2	std$(h)\gamma^*(t)$	12.1410 $\pm$ 0.021
b-IP* 0.4	bottleneck	0.4	std$(h)\gamma^*(t)$	15.3890

8 Experiment 2: Bottleneck KL Loss Penalty

8.1 Setup

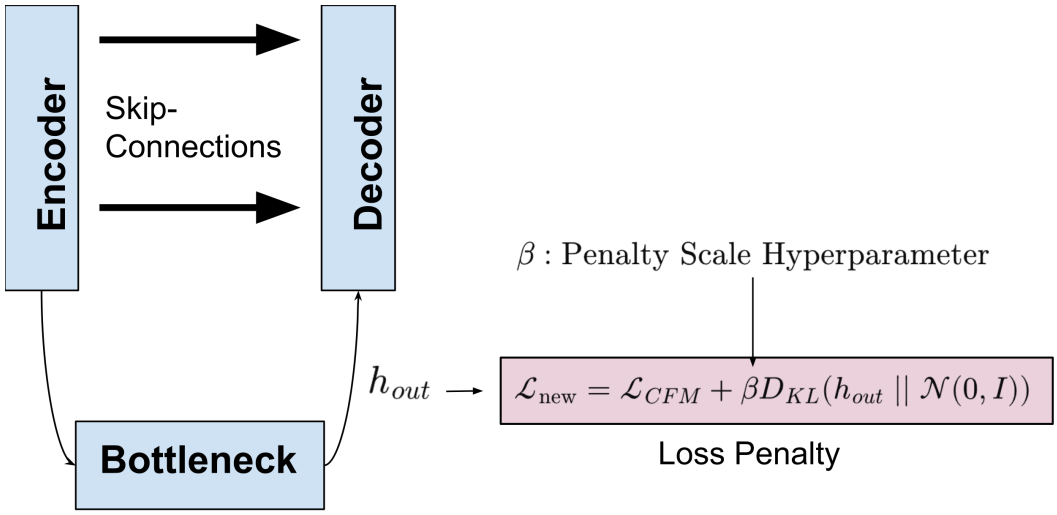


Figure 36: A scheme of the bottleneck KL-to-unit-Gaussian penalty setup in practice. Made by me in Google Drawings.

Placement of the penalty Note that my noise-injection experiment acts on the encoder's output immediately, before the bottleneck blocks, while my KL penalty targets the bottleneck's output. I test regularization at these two distinct points within the functional region of the U-Net bottleneck but a key affected part of the neural network is the bottleneck itself in both cases: When *adding noise to pre-bottleneck activations* the bottleneck layers must adapt to noised inputs, when *penalizing the KL-to-unit-Gaussian of post-bottleneck activations* the bottleneck layers are the ones responsible for producing outputs that get penalized less by the KL loss term.

Loss formulation Chapters 3.1, 2.5 and Appendix A suggest that the U-Net bottleneck layers are functionally different from up- and down-sampling layers and might focus on global semantics rather than fine-grained details. With the goal of encouraging the bottleneck layers to focus on "important" decisions rather than details and regularizing the latent activations, I propose my second experiment which is to introduce a KL-to-unit-Gaussian penalty (see figure 36) to the training loss, effectively introducing a distribution matching regulariser:

$$\mathcal{L}_{\text{new}} = \mathcal{L}_{CFM} + \beta \mathcal{L}_{\text{penalty}}$$

$$\mathcal{L}_{\text{penalty}} = D_{KL}(h_{out} \,||\, \mathcal{N}(0, I))$$

Here, h_{out} are the post-bottleneck activations and β is a weight I assign to the KL penalty. I compute the KL-divergence using the closed-form formula for two multivariable Gaussians and by treating each instance in a batch as a "sample" from the theoretical bottleneck output distribution.

Choice of β I chose a search-space for β with values ranging from 1 (standard VAE) to 0.05 (5% of the standard VAE penalty). To analyze what the values for β relatively constitute, I calculate the average loss of a base mini model from epochs 5-10 (the first 1.25%-2.5% of training) across 5 different seeds. I also calculate the average KL-to-unit-Gaussian of post-bottleneck activations in the same time window. Across 5 different seeds I obtain a mean conditional flow matching mse loss $\overline{\mathcal{L}_{CFM}} = 0.1542 \pm 0.0001$ and a mean post-bottleneck KL-to-unit-Gaussian $\overline{D_{KL}(h_{out} \,||\, \mathcal{N}(0, I))} = 2.0064 \pm 0.1903$. I get the resulting values for the relative loss scale p from the chosen values β relative to the mean loss and KL values:

$$\beta = p \cdot \frac{\overline{\mathcal{L}_{CFM}}}{D_{KL}(h_{out} \,||\, \mathcal{N}(0, I))}$$

Each value for β is such that, during an early-training reference window, the expected numerical contribution of the KL regularizer is p times the expected CFM loss.

I chose to analyze β values relative to the loss magnitude rather than the gradient magnitude because this provides an interpretable relative loss weight independent of the model's optimization dynamics, which might vary in different runs.

KL vanishing During calibration, I observed that once the KL penalty is active in the loss, the measure KL-to-unit-Gaussian of post-bottleneck activations collapses rapidly within the first several training batches, regardless of the specific β value. This is consistent with a known problem in KL-regularized models, often called *KL vanishing*: because the KL-to-unit-Gaussian term is comparatively easy to minimize relative to the CFM MSE loss, gradient descent tends to reduce it disproportionately quickly, sometimes before the task loss has meaningfully shaped the latent representation [6]. This shows me that the relative loss scale p is not entirely meaningful since the KL penalty loss term can be driven to zero much faster than the CFM loss. The relative scale p is only meaningful at initialization.

Table 6: Search space for KL penalty weights.

Effective β Value	0.05	0.1	0.25	0.5	1.0
Relative Scale p	0.65	1.30	3.25	6.50	13.00

I defined the search space for the loss scale β using five values bounded between 0.05 and 1.0. The upper bound (1.0) was selected to follow standard VAEs, while the lower bound (0.05) ensured a sufficiently small penalization baseline. The three intermediate values (0.25 and 0.10) were chosen by halving the upper bound twice and doubling the lower bound, respectively. It must be emphasized that the relative scale p that emerges from each β value is calibrated against an unpenalized early-training mini-scale model. Once training begins with the penalty active, $\mathcal{L}_{penalty}$ decreases rapidly meaning the loss ratio does not remain near p throughout training, which it was not intended to. I separately confirm (Figure 37), that my selected β values produce a monotonically increasing KL gradient contribution before collapse dominates, which is the property I rely on when interpreting the results.

Gradient magnitude ratio We can plot the gradient magnitude ratio

$$r = \frac{\|\nabla_\theta(\beta L_{\text{penalty}})\|_2}{\max(\|\nabla_\theta L_{\text{CFM}}\|_2, 10^{-12})}$$

for the β values in my search-space to confirm they increase with β. I do so by training a throwaway mini-scale model with each of the given β values and logging the gradient magnitude ratio for the first 10 batches and using 10 seeds.

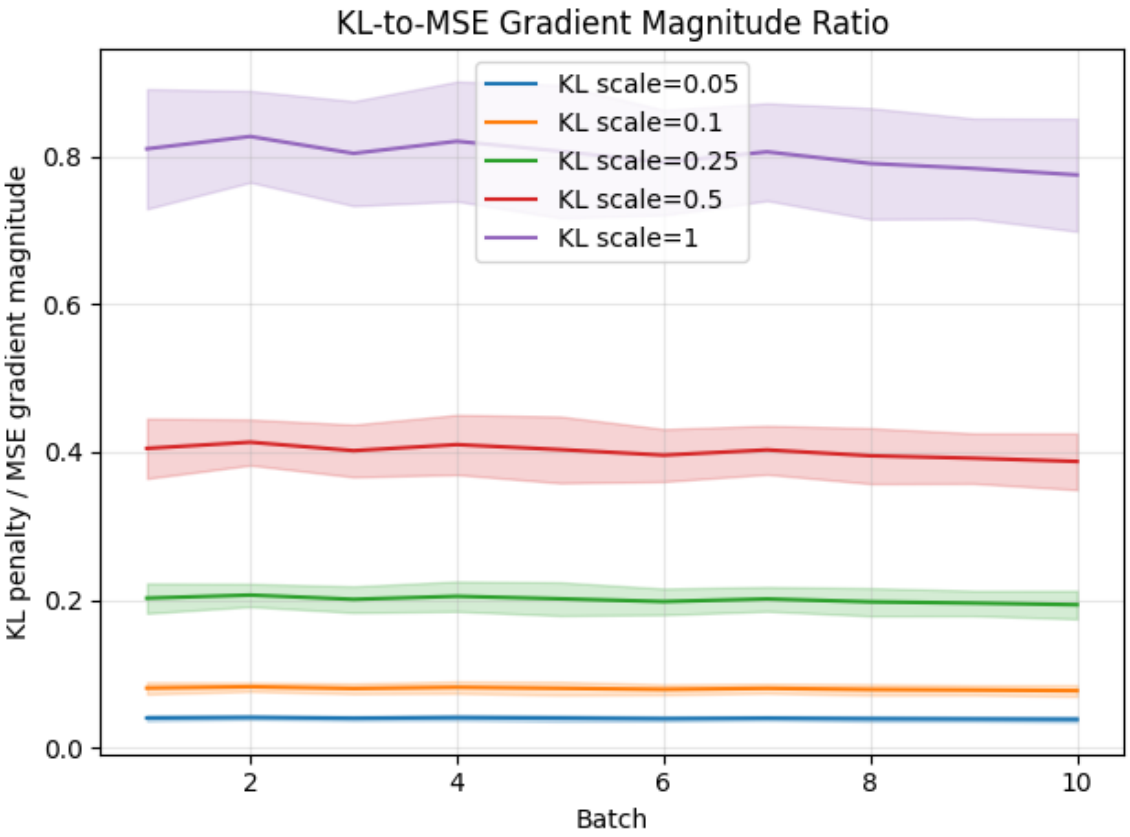


Figure 37: Gradient magnitude ratio of the KL penalty loss and the CFM MSE loss for the first 10 batches of training. The colored bands indicate the standard deviation across 10 seeds. The resulting magnitude ratio is naturally low because on the one hand, $\mathcal{L}_{penalty}$'s gradient is exactly zero for all decoder parameters since h_{out} is computed prior to the decoder. On the other hand, $\mathcal{L}_{CFM}$'s gradient is usually nonzero across the entire network.

Figure 37 shows how even the highest selected β value does not exceed a relative gradient magnitude of 1 at initialization. This ensures that I don't risk the KL penalty immensely overpowering the main CFM MSE loss term. The effective gradient magnitude ratio seems to be approximately proportional to the loss scale β:

$$r \approx 0.8 \cdot \beta$$

Bottleneck-swap test To test if the KL penalty drives the model to discard the bottleneck's representational power in order to fully minimize $\mathcal{L}_{penalty}$, I use a bottleneck-swap test: I exchange h_{out} from two different inputs at the same timestep and decode the result, comparing the resulting change in output. A representation that has degenerated to near-uninformative noise should produce outputs insensitive to this swap, since the decoder would rely mainly on the skip connections rather than h_{out}.

8.2 Hypothesis

My hypothesis for this experiment, as seen in section 4, is the following:

Hypothesis 2. ***Bottleneck KL Penalty:*** *Explicitly penalizing the KL-divergence between the bottleneck's activation distribution and a unit Gaussian, minimizing the mutual information $I(X, Z)$ per the Information Bottleneck principle, will enforce additional compression on the bottleneck representation, pushing it toward semantic rather than fine-grained content, and may improve generation quality relative to an unpenalized baseline.*

Falsification: *This hypothesis is likewise evaluated on two independent claims: (1) Bottleneck functionality claim: if evaluating the effect on various metrics shows no meaningful changes this is falsified. (2) Quality claim: If I find that the KL penalty has a measurable effect on internal model dynamics but does not lead to a better FID score over baseline, I conclude the functionality claim does not entail improved generation quality.*

8.3 Results

Keep in mind that all results in this section come from a measurement series where most model configurations were only trained once (single-seeded). Only the best (FID-wise) model was trained with 3 different seeds. Consequently, any observations may be influenced by random initialization variance.

8.3.1 FID Results

Analogous to experiment 1 7, models were trained for a fixed number of epochs rather than to full convergence due to compute limitations. Consequently, the FID scores reported here reflect performance at a specific training checkpoint and do not distinguish convergence speed from final generative quality. For the sake of evaluating the experiment, I treat the final FID score as a proxy for generative quality. See Section 9.2 for a discussion on the experimental validity of this study.

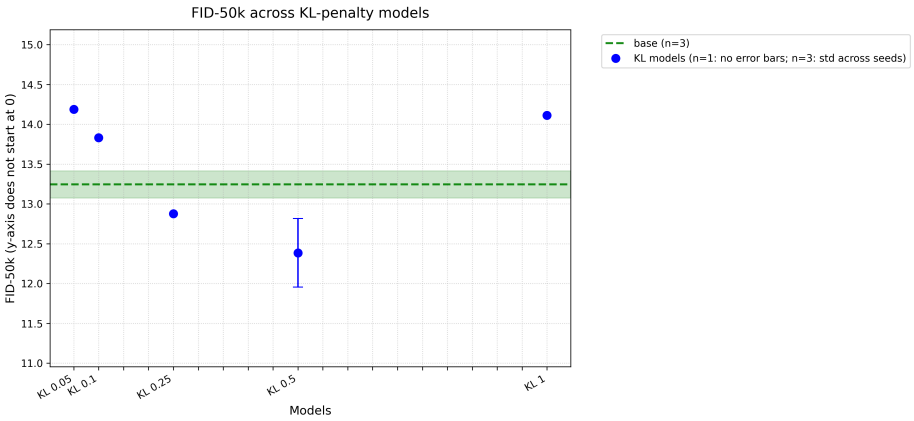


Figure 38: Final FID-50k of the KL penalty models in comparison to the base model. The base and KL 0.5 models report mean and standard deviation across 3 seeds.

We can infer from Figure 38 that the FID-50k-minimizing penalty scale β lies within [0.25, 1]. The FID-50k score of KL 0.05 and KL 0.1 also suggest that a KL penalty scale that is too low can harm the model's generative quality.

Table 7: A *lower* FID-50k value indicates that the distribution of generated samples is closer to the real data distribution, i.e., *better* generative quality.

	base	KL 0.05	KL 0.1	KL 0.25	KL 0.5	KL 1
FID-50k	13.2442 ± 0.1698	14.1872	13.8314	12.8780	12.3855 ± 0.4302	14.1124
Change relative to base	0%	+7.12%	+4.43%	-2.76%	-6.48%	+6.56%

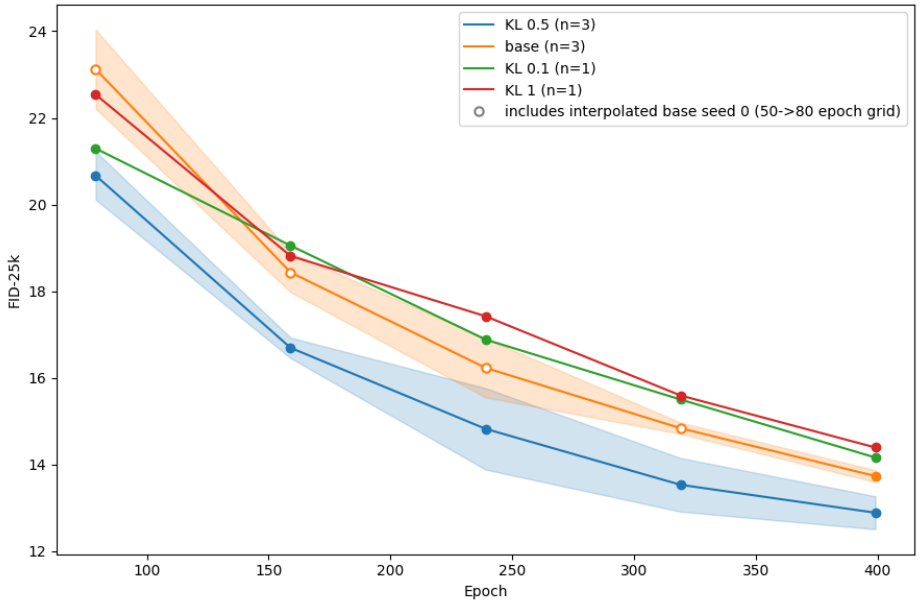


Figure 39: FID-25k for the base model and a subset of the KL models. Base and KL 0.5 models report mean and standard deviation across 3 seeds. One of the base model's seed's values were interpolated from an 80 epoch grid to 50 epochs.

Figure 39 shows how the KL 0.5 model has a consistently lower FID-25k score than the base model for the entirety of training.

8.3.2 Bottleneck Swap Test Results

The purpose of the bottleneck swap test is to ensure that the models that were trained with KL-to-unit-Gaussian loss penalties on their post-bottleneck activations did not just drive the bottlenecks output to meaningless noise and discard its representational power completely. This is more of a check rather than a result. To quantify the importance of the bottlenecks output we compute the instance-wise (meaning we reduce from (B, C, H, W) to (B)) mean squared error (MSE) between decoded states y generated from normal skips, embeddings and bottleneck outputs and decoded states $\hat{y}$, which were generated using the same embeddings and skips but had their bottleneck outputs (decoder inputs) shuffled across the batch dimension. The final MSE value is averaged across 9'600 test-set images and models that have multiple seeds have a standard deviation computed across seeds. Interpretation is straight-forward, the absolute MSE values are insignificant but by comparison with the base model we obtain our answer: An average bottleneck-swap MSE equal to or higher than baseline signals that the bottlenecks output is being used properly within the network whereas a significantly lower bottleneck-swap MSE indicates that the network is pushing the bottleneck's output towards total noise to minimize the loss penalty, discarding the bottleneck representational power in the process. For our mini-scale models, the bottleneck consists of $\approx 30\%$ of the networks parameters.

Figure 40 shows how the only KL model with a penalty scale of 1.0 fell more than one standard deviation below the baseline in bottleneck-swap MSE. This suggests itself as a plausible explanation for this model's low FID-50k. The other models show a trend: a lower penalty scale seems to correlate with higher bottleneck-swap MSE and thus more importance being attributed to the post-bottleneck activations, partially even exceeding the baseline.

8.3.3 Cross-Channel Correlation Results

To analyze if the approach of adding KL-to-unit-Gaussian penalties to our U-Net models pushed them to more efficiently utilize their representational power I use the metric detailed in Section 5.2.4 that measures the average absolute off-diagonal covariance entries. We denote models with lower cross-channel-covariance relative to a baseline as better utilizing their latent space's capacity for high-dimensional information.

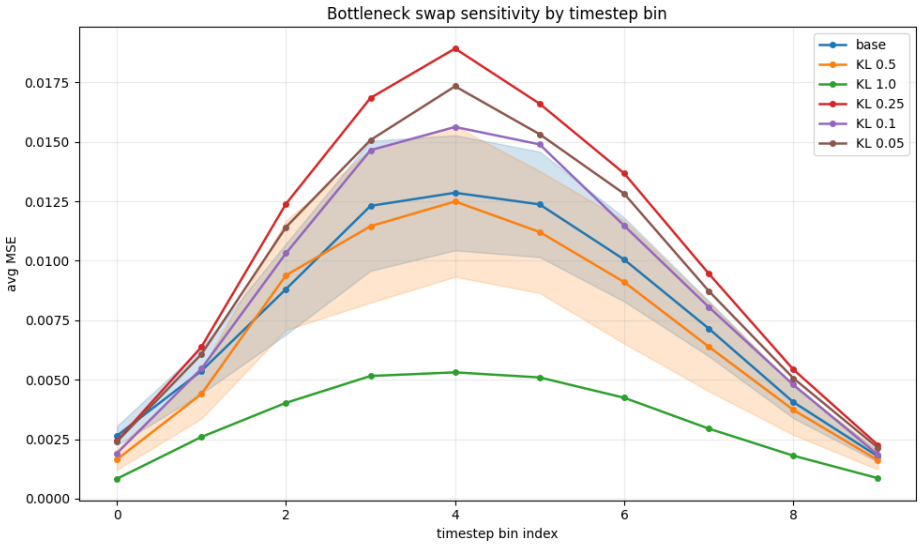
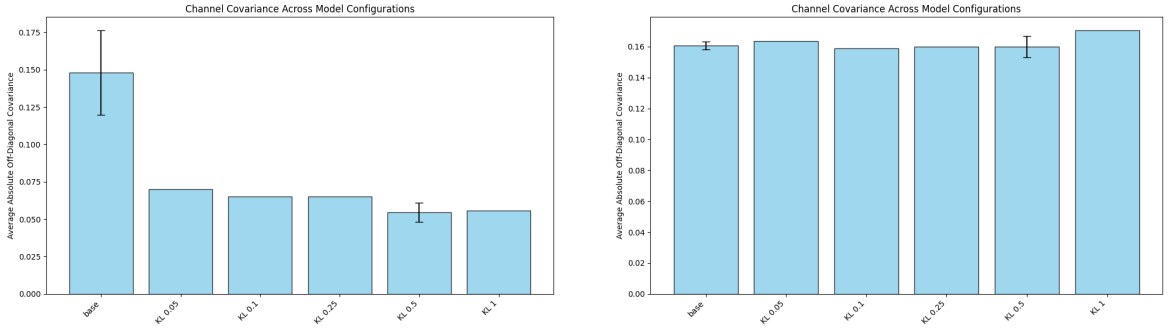


Figure 40: Bottleneck-swap MSE values for the base model and all KL models. Averaged across 9'600 test set images. Base and KL 0.5 models report means and standard deviations across 3 seeds.



(a) Post-bottleneck activations.

(b) Pre-bottleneck activations.

Figure 41: Average absolute off-diagonal cross-channel covariance of globally pooled activations for base and KL models. Averaged across 28'800 samples (9'600 images $\times$ 3 differently-seeded evaluations). The base and KL 0.5 models report mean and standard deviation across 3 training seeds. The other models are single-seeded.

Figure 41 shows how all models trained with a KL-to-unit-Gaussian penalty have a much lower average cross-channel correlation after the bottleneck than the base model. This suggests that the KL penalty pushes the model to properly utilize its bottleneck's representational capability to uncorrelated features in the data. We can observe that a higher penalty scale only increases this effect slightly, if at all. It seems that the mere presence of a KL penalty lowers cross-channel correlation.

At first I thought that low cross-channel correlation would be a measurable property across regularization methods and that it could explain gains in FID-50k. However, this metric does not correlate with the penalty scale or FID β for KL models and b-IP* models (Experiment 1: 7) all have values inside the base model's standard deviation. The true reason for the FID-50k improvements in this study remains unknown.

8.3.4 ERF Results

Using the effective receptive field diameter and average gradient maps (Metric details in 5.2.5) we can plot the ERF diameter and average gradient maps of specific `ResBlock`s. I suppose that a larger ERF diameter in the Bottleneck's `ResBlock`s signals that the bottleneck is paying more attention to global information rather than fine-grained details.

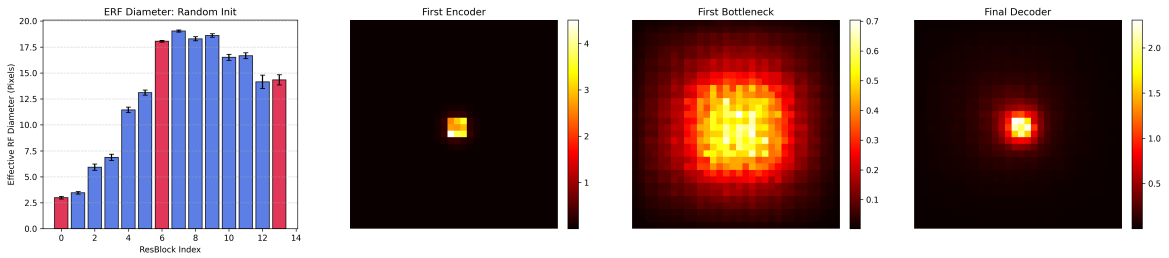


Figure 42: ERF master figure (diameter progression and selected gradient heatmaps) for a randomly initialized mini-scale U-Net model. Grad maps were averaged and diameter standard deviations are reported across 10 random seeds for the model's weight initialization.

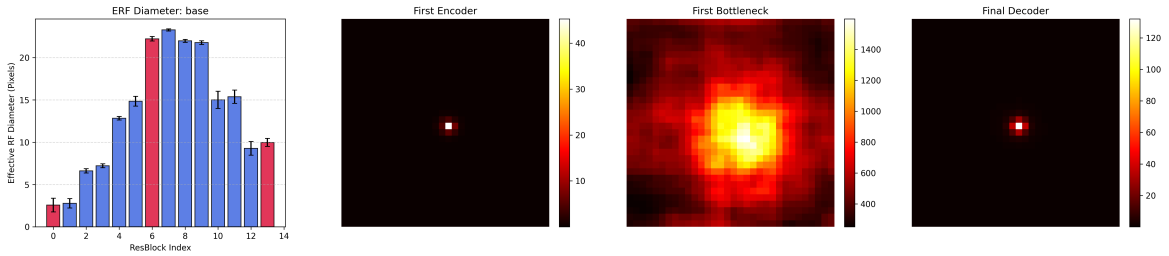


Figure 43: ERF master figure (diameter progression and selected gradient heatmaps) for the base mini-scale model. Grad maps were averaged and diameter standard deviations are reported across 3 random seeds.

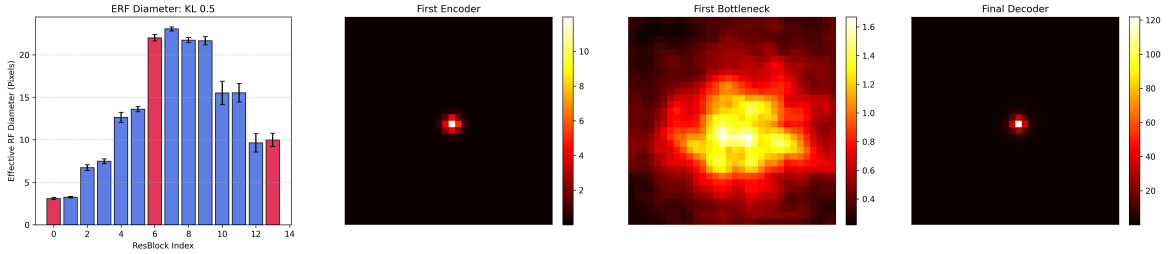


Figure 44: ERF master figure (diameter progression and selected gradient heatmaps) for the KL 0.5 model. Grad maps were averaged and diameter standard deviations are reported across 3 random seeds.

These three figures show that the second bottleneck `ResBlock` (index 7) consistently has the largest ERF diameter, even at initialization. This shows that the bottleneck layers are innately influenced by a larger area of the input image than encoder or decoder layers. I also observed that all trained models across this project have larger ERF diameters than a randomly initialized model: the ERF grows during training. To further inspect how training with a KL penalty affects the ERF diameter, we can plot the ERF diameter at `ResBlock` 7 for different models.

Figure 45 shows how the KL models' bottleneck layers do not have a larger ERF than the base model. As for the other `ResBlocks` in the network, the KL models' ERF diameters mostly lay within the base model's standard deviation band. Therefore, this experiment's functionality claim is falsified.

8.3.5 Test Set Loss Results

Figure 46 shows how high KL penalty scale values (0.5 and 1.0) achieve a lower train set to test set loss gap. Despite the KL 0.5 result having a high standard deviation, the decrease is statistically significant. However, KL models with lower noise scales show irregular results and even seem to have bigger train-test gaps than the base model which prevents us from making general claims on the

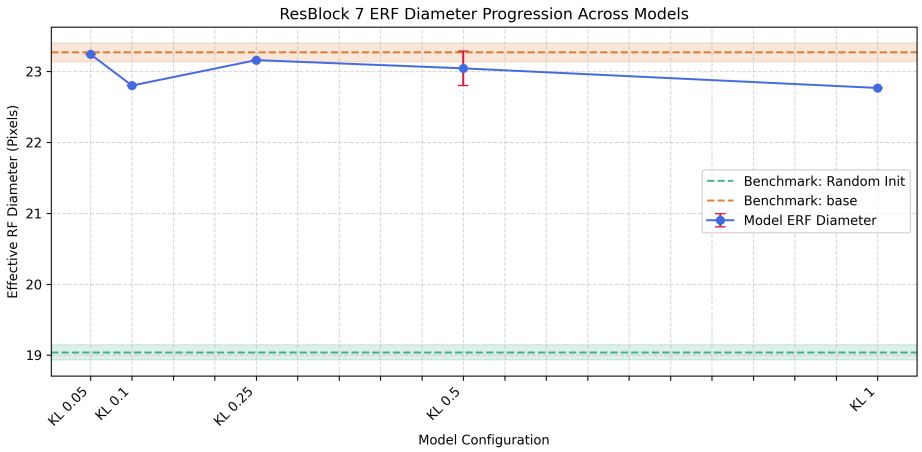


Figure 45: ERF diameter of `ResBlock` 7 for various models. The measurement is averaged across 200 noise draws. Base and KL 0.5 models report mean and sample standard deviation across 3 seeds.

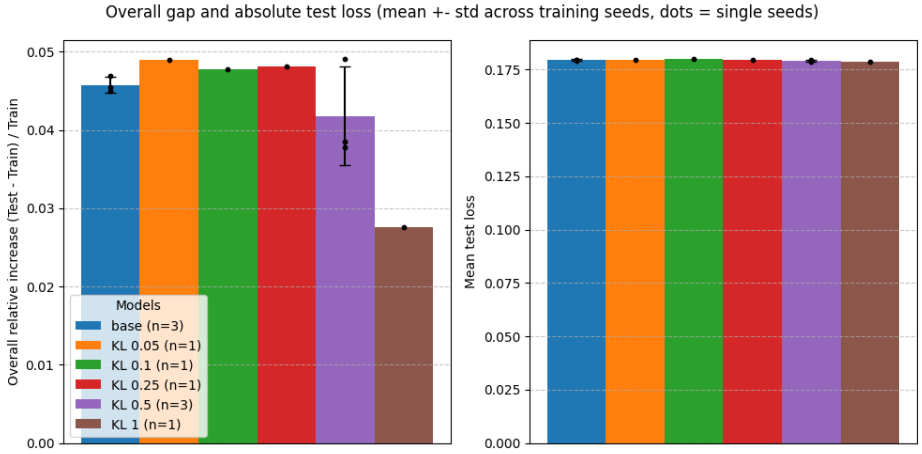


Figure 46: Average relative loss increase from train to test set across timesteps (left) and average test set loss (right). Base and KL 0.5 models report mean and standard deviation across 3 seeds. The loss ratio was calculated using the average losses across 9'600 images and itself averaged across 10 evaluation repetitions.

effect of the KL-to-unit-Gaussian penalties on the post-bottleneck activations on train set to test set generalization.

8.4 Summary & Empirical Claims

I introduced a KL-to-unit-Gaussian penalty on the post-bottleneck activations of the U-Net across a search space of loss scales $\beta \in \{0.05, 0.1, 0.25, 0.5, 1.0\}$ which corresponds to a relative noise scale space (at initialization of an unpenalized baseline) ranging from 0.65 to 13.00. I determined that the optimal penalty scale β for generative quality lies within $[0.25, 1]$, with my KL 0.5 model achieving a final FID-50k value that is 6.48% lower than the base model's (Table 7).

Ultimately, the bottleneck KL loss penalty experiment allows us to make the following empirical claims for Flow-Matching U-Nets:

1. **Generation Quality**: Explicitly penalizing the KL-divergence between the bottleneck's activation distribution and a unit Gaussian improves generative quality at suitable scales ($\beta \in [0.25, 1]$), with KL 0.5 achieving the best FID-50k improvement relative to the base model (Figure 38).

2. **Cross-Channel Representation**: The KL penalty significantly reduces average absolute cross-channel covariance after the bottleneck, indicating that the regularization drives the model to utilize its latent space capacity to learn uncorrelated features (Figure 41). However, this did not behave regularly across penalty scales.
3. **Receptive Field (ERF) Functionality**: Training with a bottleneck KL penalty does not measurably increase the effective receptive field (ERF) diameter of the bottleneck's `ResBlocks` relative to the base model (Figure 45).
4. **Representational Collapse**: Excessively high penalty scales (e.g., $\beta = 1.0$) cause the bottleneck representation to collapse toward uninformative noise, discarding its representational power and leading to degraded generative performance as confirmed by the bottleneck-swap test (Figure 40).

These claims were not tested at large scales and are thus not guaranteed to be generally applicable.

At the end of this section, I answer my hypothesis from section 8.1:

(1) Bottleneck functionality claim: falsified. The bottleneck KL penalty does not expand the ERF diameter or shift the bottleneck layers toward a broader global receptive field relative to the baseline, despite successfully reducing cross-channel correlation and altering internal representation dynamics.

(2) Quality claim: supported. Explicitly penalizing the bottleneck KL-divergence measurably improves FID-50k relative to the baseline at suitable relative scales ($\beta \in [0.25, 1]$), with $p = 0.5$ yielding the best improvement of 6.48%. However, this improvement occurs independently of the expected changes in receptive field expansion.

KL Penalty Model Specs

Model	KL penalty scale β	relative loss scale p at initialization	gradient ratio r at initialization	FID-50k
KL 0.05	0.05	0.65	≈ 0.04	14.1872
KL 0.1	0.1	1.30	≈ 0.08	13.8314
KL 0.25	0.25	3.25	≈ 0.2	12.8780
KL 0.5	0.5	6.50	≈ 0.4	12.3855 $\pm$ 0.4302
KL 1	1.0	13.00	≈ 0.8	14.1124

9 Discussion

9.1 Synthesis of Results and Research Question

We asked how perturbing or penalizing the activations of a flow-matching U-Net's bottleneck during training affects (i) the model's internal behavior and (ii) the quality of its samples. The two experiments give a consistent positive answer within our setup for (ii) and mostly null or FID-unrelated results for (i).

Sample quality Both interventions (bottleneck IP and the KL-to-unit-Gaussian penalty) improved FID-50k over the base model at moderate strengths. Bottleneck input perturbation with $\gamma_0 = 0.2$ lowered FID-50k by 8.33%, and the KL penalty with $\beta = 0.5$ lowered it by 6.48%. Both effects were confirmed across three seeds, and both models had a lower FID-25k than the base model during training. In both experiments the best value from my search-space was bracketed by worse values

on both sides ($\gamma_0 = 0.1$ and 0.4, $\beta = 0.25$ and 1.0). Because the neighboring runs are single-seeded, this only roughly locates the optimum. For β, KL 0.25 lies within about one standard deviation of KL 0.5. Too much regularization hurts: b-IP* 0.4 increased FID by 16.19%, and the bottleneck swap test suggests that KL 1.0 degraded the bottleneck representation. Standard input perturbation, in contrast, did not improve FID in my setting (IP* 0.1 was within one standard deviation of the base model, IP* 0.2 was far worse), although the compared search spaces were unequal (section 7).

Internal behavior In Experiment 1, the representational claim was not supported: neither the KL gap nor the derivatives changed measurably. In Experiment 2, the ERF functionality claim was falsified, but the penalty did change other properties (cross-channel covariance, and for $\beta \geq 0.5$ the train-test loss gap). Neither change alone explains the gain in FID: KL 0.05 and 0.1 have low covariance but worse FID than the base model, and KL 1.0 has a smaller gap but also worse FID.

The metrics I chose do not explain the improvements in FID.

- **Exposure bias:** Neither standard IP nor bottleneck IP measurably reduced the teacher-forcing vs. autoregressive KL gap.
- **Sensitivity:** Bottleneck IP did not change $\mathrm{d}y/\mathrm{d}h$ beyond the base model's standard deviation, so the Bishop-style contractive effect did not appear for the sub-network (I hypothesize that this is because skip connections still carry unperturbed signal). Constant-weight input IP did lower $\mathrm{d}y/\mathrm{d}x$.
- **Receptive field:** The KL penalty did not enlarge the ERF of the bottleneck `ResBlock`s.
- **Cross-channel covariance:** KL penalized models had much lower cross-channel covariance, but this did not scale with β and b-IP* models did not show it, so it cannot be the common cause of the FID gains.
- **Generalization:** Test loss was nearly identical across b-IP models, and the train-test gap decreased only for the two highest KL scales.

Table 8 summarizes these results and shows which metrics were measured for which experiment.

Metric	Exp. 1: b-IP*	Exp. 2: KL penalty
FID-50k	-8.33% at $\gamma_0 = 0.2$	-6.48% at $\beta = 0.5$
TF/AR KL gap	no measurable change	no measurable change
Derivatives	no consistent change (largest: $\mathrm{d}h/\mathrm{d}x$ -3.7% at $\gamma_0 = 0.2$, irregular across scales)	no measurable change
Train–test loss gap	no measurable change	lower for $\beta \geq 0.5$, irregular below
Cross-channel covariance	within base std	much lower for all β, not β-dependent
ERF (`ResBlock` 7)	not measured	no increase
Bottleneck swap test	not measured	lower than base only at $\beta = 1.0$

Table 8: Summary of results per metric and experiment, relative to the base model.

Answering the research question Both hypotheses were split into a representational and a quality claim, and in both experiments these came apart: the quality claims were supported, the representational claims were not (in Experiment 2, the changes I did observe do not explain the changes in FID). Regularizing the bottleneck improved FID, but not through the mechanisms I proposed. My motivation, that the bottleneck is functionally distinct (small relative weight updates, largest ERF, different activation statistics, Section 2.5, Section 5.2.5), is consistent with observations on the unregularized models, but I re-measured the ERF under an intervention (the KL penalty), and it did not change. The large bottleneck ERF is also partly architectural, since it is already present at random initialization.

What the two experiments have in common Both interventions are intended to make the clean signal portion at the bottleneck smaller, and both showed a "sweet spot" behavior regarding the generative quality across hyperparameter choices for γ_0 and β. Using the selected metrics, I was not able to determine why the modified models achieve better generative quality. The FID improvement

might be due to generic regularization that our metrics do not capture. For b-IP*, neither the train-test loss gap nor the test loss changed, so a generalization-based explanation is unsupported there, and without having tested regularization on other parts of the network we cannot empirically distinguish generic from bottleneck-specific regularization.

Open questions

- What property of the model, not captured by our metrics, changes and entails the improvement in FID? Was the underlying representational change swallowed by noise, missed due to imprecise evaluation or is it something not detectable with the chosen metrics?
- Despite improving in the relative sense, our achieved FID-50k values were nowhere near state-of-the-art U-Nets. Do the observed effects persist with a proper cosine LR schedule and/or at larger scale?
- Does regularizing other locations inside the U-Net (encoder, decoder) give similar gains, or is the bottleneck special?

9.2 Limitations and Validity of this Study

Statistical power and seeding Compute allowed one run per configuration, with only the base model and each experiment's best model re-seeded twice. Single-seeded results (all other γ_0 and β values) are affected by initialization randomness, which I could only estimate from the base model's standard deviation (0.1698 FID-50k). Selecting the best of five configurations by FID-50k also introduces selection bias for the best configuration. The re-seeding of base, b-IP* 0.2 and KL 0.5 reduces, but does not remove this concern. Even with three seeds, the standard deviation estimates are themselves uncertain: the KL 0.5 standard deviation (0.4302) is far larger than the b-IP* 0.2 one (0.021). However, the two re-seeded runs of each winning configuration were not involved in the selection, so they are unaffected by it.

Training length and learning-rate schedule Models were trained for a fixed 400 epochs and were almost certainly not converged, so FID differences may reflect convergence speed rather than final quality. All experimental models used a constant learning rate due to the schedule bug (section 5.5). Because all models, including the base model, used the same constant learning rate, the comparison between them is controlled. Transfer is the problem: the cosine-decayed base model reached an FID-50k of about 12.02 vs. 13.11 for the constant-LR base model (seed 0, the 3-seed mean is 13.24), a gain as large as the improvements reported here. A regularizer might recover part of what learning-rate decay provides, in which case the improvements could shrink under a proper schedule.

Bottleneck specificity I did not test regularization in other parts inside the network (e.g. the encoder or decoder), so I cannot say whether the improvements are specific to the bottleneck. The only related comparison, IP* vs. b-IP*, is slightly affected by the unequal search spaces and the different noise scaling (see Hyperparameter choices).

Scale and dataset Everything was done on class-conditional CIFAR-10 (32×32) with a 10.85M-parameter U-Net with a 3-level architecture. The CFG scale was tuned on the base model and transferred to all others, which may slightly favor the base model or disadvantage perturbed models. Claims about larger models, other datasets, or other architectures are not supported. Compute (a single 8GB GPU) restricted the study to mini-scale models.

Hyperparameter choices The custom noise weight schedule $\gamma^*(t)$ and its parameters (t_1, t_2, k) were chosen heuristically and not tuned. Because noise is scaled by $\mathrm{std}(x_t)$ at the input and $\mathrm{std}(h)$ at the bottleneck, equal γ_0 does not mean equal effective noise. The comparison between IP* and b-IP* is also unequal: five γ_0 values were searched for b-IP*, only two for IP*. For the KL penalty, the relative scale p only holds at initialization since the penalty collapses within the first few batches. I logged the gradient magnitude ratio to confirm the gradients coming from the KL penalty were of reasonable scale at initialization, but the search space is less interpretable than it looks.

Metric validity For detailed descriptions of all methods see Section 5.2.

- FID does not capture all aspects of image quality [36], and my quality claim rests on FID alone.
- The KL gap is a proxy for exposure bias, uses globally pooled features and a Gaussian approximation, and its first bin is trivially zero.
- The derivative metric uses a single Gaussian probe per image in `bfloat16`, which limits precision.
- The ERF was measured on pure-noise inputs averaged over timesteps and may not reflect receptive fields on realistic x_t.
- Cross-channel covariance was computed on pooled activations.

Null results on these metrics therefore show that no effect was *detected*, not that none exists.

Implementation issues Documentation of the implementation bugs can be found in Section 5.5. Except for the learning-rate schedule bug, none of the detected bugs measurably affects the experiment results. The standard-deviation gradient leak was measured to be far below the bfloat16 precision floor, so I consider it negligible, but it was present in every perturbation run. The in-place sinusoidal embedding bug was fixed and all affected data regenerated. The 50- vs. 80-epoch FID-25k interpolation only affects the shape of the training curves, not the final values.

Overall validity Taken together, I consider the following claims reasonably supported: bottleneck IP and a bottleneck KL penalty lower FID-50k at moderate strength in my setting (every seed of b-IP* 0.2 and of KL 0.5 has a lower FID-50k than every seed of the base model, with mean improvements of 8.33% and 6.48%), and the best values on my grid are flanked by worse values for both γ_0 and β, which locates the optimal range only coarsely. The measured metrics do not explain the gain. I do not claim that the improvements or any representational result generalize beyond the scope of this study.

10 Conclusion and Outlook

10.1 Conclusion

This project asked how perturbing or penalizing the activations of a flow-matching U-Net's bottleneck during training affects the model's internal behavior and the quality of its samples. On class-conditional CIFAR-10 with a 10.85M-parameter U-Net, both interventions improved FID-50k at moderate strengths. Bottleneck input perturbation with $\gamma_0 = 0.2$ lowered it by 8.33% (12.1410 vs. 13.2442) and a KL-to-unit-Gaussian penalty with $\beta = 0.5$ lowered it by 6.48% (12.3855 vs. 13.2442). In both cases every seed of the best configuration outperformed every seed of the base model. Standard network-input perturbation did not improve FID in my setting, and a constant noise scale produced noisy samples, which my time-dependent, standard-deviation-weighted schedule $\gamma^*(t)$ resolved.

The metrics that were used did not explain these gains. Bottleneck input perturbation did not measurably reduce the teacher-forcing vs. autoregressive KL gap or change the model's partial derivatives consistently, and the KL penalty did not enlarge the effective receptive field of the bottleneck. The KL penalty did lower cross-channel covariance, but this did not track FID. For both experiments, the quality claims of my hypotheses were therefore supported while the representational claims were not, and the cause of the improvement remains unexplained by my metrics.

Future work is needed to explore other explanations for the improvements in FID that were achieved in both experiments. Additionally, larger U-Nets should be investigated as well to determine whether the effects generalize to state-of-the-art models. Such efforts require access to significantly more compute than was available during this study.

10.2 Future Work

The following questions remain unanswered by this work and could potentially be tackled in the future:

- **Bottleneck specificity:** Do the same noise injection and KL penalty improve FID when applied at intermediate activations in the encoder or decoder? Similar gains would indicate generic regularization instead of a bottleneck-specific effect.
- **Explaining the FID gain:** The metrics in this study did not fully explain why b-IP* and the KL penalty improve FID. Other metrics might be needed to find the underlying internal change(s).
- Learning-rate schedule: How do the results, especially regarding FID, look with a proper cosine learning-rate schedule? If they shrink, b-IP* and the KL penalty may partly recover what learning-rate decay provides.
- **Larger scales**: How do b-IP* and the KL penalty perform for bigger U-Nets trained on a larger dataset, namely the CIFAR-100 dataset?

11 Reflection

In addition to the two implementation bugs detailed in 5.5, the following experimental oversights of this study should have been prevented:

- **Seeding**: Although re-seeding the models multiple times was not feasible under my compute budget, I could have trained all models using the same random seed for initialization to achieve more comparable results.
- **Noise Weight Schedule**: The noise weight schedule $\gamma^*(t)$ that I implemented in order to fix the noisy-samples problem in section 6.3 was chosen intuitively because I was not able to find a concise shape through the evaluated metrics. I simply chose exponential decay with clamping to 0 at the end because it is one of the easiest ways to design a time-dependent noise weight schedule that doesn't overly noise late timesteps. If I had invested more time into a carefully selected, properly justified noise weight schedule I might have found a simpler solution that fixes the noisy-samples problem.
- **ODE steps**: I naively chose to use a 50 step Euler ODE solver for all FID-50k and FID-25k calculations. However, after comparing Euler against two higher-order samplers (RK4 and RK45) and across different numbers of ODE steps I realized that I could not visually tell the difference between samples (from a mini-scale model) using Euler at 20 model calls and RK45 at over 2000 model calls (at the same seed) even if I tried my hardest. This suggests that the ODE trajectories my models produced were very straight and as low as 10 Euler ODE steps could have been used for FID-25k and FID-50k calculations. This naive choice cost me a non-negligible amount of compute time and could have been prevented by analyzing different ODE solver configurations prior to training the models that evaluate FID-25k at intervals during training.

The following methods in this project are notable successes in my opinion:

- **Search-Spaces**: For both the bottleneck-IP and KL penalty experiments, the search space for γ_0 and β, respectively, was chosen such that I was able to determine the interval in which the FID-50k-minimizing parameter choice lies. This is because in both cases, the best-performing parameter choice from my search space is bracketed by two worse performers. I attribute this success to reasoned search-space choices: for b-IP*, I used log-spaced γ_0-values centered around 0.1 since I had confirmed that to be a reasonable choice in my preliminary work in Section 6.3. For the KL-penalty experiments, I selected the number of experimental runs (5) as well as the upper and lower bounds of the search space. The intermediate values were chosen by halving the upper bound twice and doubling the lower bound once, respectively (see Table 6 in Section 8.1).

- **Two-part hypotheses:** Splitting each hypothesis into a representational claim and a quality claim, each with its own falsification, made sure none of the experiments depended on an increase in FID or a clear representational find to be informative. In both experiments the two claims came apart (quality supported, representational not), and this was only possible because they had been separated.
- **Controls and sanity checks for the metrics:** Metrics were validated before their results were interpreted: the KL-gap noise floor (two independent teacher-forced sets), the ERF of randomly initialized models as an architectural reference, the gradient-magnitude ratio of the KL penalty across 10 seeds, and the bottleneck-swap test to check that the KL penalty did not simply collapse the bottleneck. These make null results more informative since I know what I am comparing against.
- **Diagnosing and fixing the noisy-sample problem:** The constant-noise IP model's (IP 0.1: FID-50k 94.09) failure was examined through its noisy samples, which led to the weighted noise schedule $\gamma^*(t)$ (IP 0.1*: FID-50k 13.41).

11.1 Personal Reflection

Since I first got into machine learning through Andrej Karpathy's "Zero to hero" YouTube series on neural networks in November of last year, I have learned and built a lot; this project is my first attempt at empirical research in the field. The *Maturaarbeit* was a good opportunity for me to tackle a bigger project and do it start to finish. I must admit that I find learning new things (reading papers and courses, watching tutorials) and building/implementing models more exciting and rewarding than doing empirical work. Nonetheless, I think that this project went better than I initially expected, and I learned valuable things and gathered lots of experience.

Learning and Studying the Field Despite having already known the basics of neural networks and flow-matching models before getting started on this paper, I studied many new topics in the early stages of this project. I spent a lot of time studying and understanding CNNs, DDPMs, Autoencoders, and Flow Models, as well as general deep learning concepts such as the manifold hypothesis and exposure bias. Looking back, I must admit that I went down unrelated rabbit holes and built unrelated projects such as Diffusion Transformers [28] and the `ModelAnalyzer` logging utility, which was not used much in this project. Generative models in specific fascinate me, and I look forward to future projects. Despite everything, the biggest challenge and learning point was the empirical and experimental aspect. Motivating, planning, evaluating, and interpreting experiments was not easy, but I gained valuable experience and am confident that I would be able to better manage such a task in the future. An additional, smaller branch of learning was optimizing the code to run on my single GPU with common PyTorch practices such as fused optimizer kernels, autocasting, and dataloader parameter tuning. To be able to train batches of mini-scale models while leaving my computer completely unattended, I set up a master training script with email progress notifications, cool-down periods in between training runs, and saving and resuming training progress.

Difficulties As detailed in Section 5.5, a couple of bugs and setbacks occurred during this project. The bug with the biggest impact was the learning-rate schedule bug, which I did not discover in time to re-train all models and had to keep in the final evaluation. The most devastating error was the in-place sinusoidal timestep embedding bug ($t* = 1000$ was being used instead of $t = t * 1000$) because it went undetected for extremely long, and the experiment results with the corrupted data were actually a lot more favorable: the KL gap measurably decreased for IP and b-IP* models, and the derivatives showed much bigger changes. I found this bug only two weeks before submission and lost more than 8 hours panicking, regenerating all plots, and coming to terms with the more mundane, but correct data. On another note, I initially confidently expected the bottleneck input perturbations to change some of the internal metrics, such as the KL gap, and the KL penalties to increase the ERF diameter. Having achieved two model types that improve FID-50k without being able to explain the underlying representational changes left me disappointed. During this project, I also experienced many difficulties due to my poor time management and absence of proper planning ahead of time. The last month before submission was a bit stressful, but all in all, I am happy with the result.

Usage of LLMs Throughout the project, I used LLMs a lot for tasks like debugging, reviewing, researching, and learning. In my personal experience, LLMs provide the most value for learning; they are amazing tutors and thinking partners for reading papers and understanding new things, especially in a new field. LLMs also allowed me to dodge coding bottlenecks. Although I can confidently write Python code and use PyTorch, the use of AI eliminated some coding bottlenecks caused by lack of experience: plotting with `matplotlib`, building a sampling interface with `tkinter`, etc.

What I'd do next Having arrived at the end of this project, I would like to pursue a better understanding of *why* b-IP* and the KL models improved generative quality. Beyond empirical research I am looking forward to learning more in the field of generative modelling and building new projects.

To sum up, this project gave me a proper taste of deep learning research and inspired me to potentially pursue similar research questions in the future.

Bibliography

References

[1] Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions, 2025.

[2] André Araujo, Wade Norris, and Jack Sim. Computing receptive fields of convolutional neural networks. *Distill*, 2019.

[3] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. *International Conference on Learning Representations (ICLR)*, 2022.

[4] Yoshua Bengio, Li Yao, Guillaume Alain, and Pascal Vincent. Generalized denoising auto-encoders as generative models. *Advances in Neural Information Processing Systems 26*, 2013.

[5] Christopher M. Bishop. Training with noise is equivalent to tikhonov regularization. *Neural Computation*, 1995.

[6] Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew M. Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL)*, 2016.

[7] Christopher P. Burgess, Irina Higgins, Arka Pal, Loic Matthey, Nick Watters, Guillaume Desjardins, and Alexander Lerchner. Understanding disentangling in β-VAE. *arXiv preprint arXiv:1804.03599*, 2018.

[8] Lawrence Cayton. Algorithms for manifold learning. *Technical Report CS2008-0923, University of California, San Diego*, 2005.

[9] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv preprint arXiv:2205.14135*, 2022.

[10] Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis. *Advances in Neural Information Processing Systems*, 2021.

[11] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. *International Conference on Machine Learning*, 2024.

[12] Tobias Sebastian Finn, Lucas Disson, Alban Farchi, Marc Bocquet, and Charlotte Durand. Representation learning with unconditional denoising diffusion models for dynamical systems. *Nonlinear Processes in Geophysics*, 2024.

[13] Bela A. Frigyik, Santosh Srivastava, and Maya R. Gupta. An introduction to functional derivatives. *UWEE Technical Report UWEETR-2008-0001, University of Washington*, 2008.

[14] Valentin Gabeur, Shangbang Long, Songyou Peng, Paul Voigtlaender, Shuyang Sun, Yanan Bao, Karen Truong, Zhicheng Wang, Wenlei Zhou, Jonathan T. Barron, Kyle Genova, Nithish Kannen, Sherry Ben, Yandong Li, Mandy Guo, Suhas Yogin, Yiming Gu, Huizhong Chen, Oliver Wang, Saining Xie, Howard Zhou, Kaiming He, Thomas Funkhouser, Jean-Baptiste Alayrac, and Radu Soricut. Image generators are generalist vision learners. *arXiv preprint arXiv:2604.20329*, 2026.

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015.

[16] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in Neural Information Processing Systems*, 2017.

[17] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 2020.

[18] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[19] Peter Holderrieth and Ezra Erives. An introduction to flow matching and diffusion models. *arXiv preprint arXiv:2506.02070*, 2025.

[20] Hyunjik Kim and Andriy Mnih. Disentangling by factorising. *Proceedings of the 35th International Conference on Machine Learning*, 2018.

[21] Alex Krizhevsky. Learning multiple layers of features from tiny images. *Technical Report, University of Toronto*, 2009.

[22] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *International Conference on Learning Representations*, 2023.

[23] Wenjie Luo, Yujia Li, Raquel Urtasun, and Richard Zemel. Understanding the effective receptive field in deep convolutional networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[24] Osval A. Montesinos-López, Abelardo Montesinos-López, and Jose Crossa. Convolutional neural networks. *Multivariate Statistical Machine Learning Methods for Genomic Prediction, Springer*, 2022.

[25] Mang Ning, Enver Sangineto, Angelo Porrello, Simone Calderara, and Rita Cucchiara. Input perturbation reduces exposure bias in diffusion models. *International Conference on Machine Learning*, 2023.

[26] Peter J. Olver. Calculus of variations. *Lecture Notes, University of Minnesota*, 2021.

[27] Gaurav Parmar, Richard Zhang, and Jun-Yan Zhu. On aliased resizing and surprising subtleties in gan evaluation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[28] William Peebles and Saining Xie. Scalable diffusion models with transformers. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[29] Vidya Prasad, Zhu-Tian Chen, Anna Vilanova, Hanspeter Pfister, Nicola Pezzotti, and Hendrik Strobelt. Unraveling the temporal dynamics of the unet in diffusion models. *Second Workshop on Deployable AI @ AAAI 2024*, 2023.

[30] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Improving image generation with better captions. *arXiv preprint arXiv:2305.10452*, 2023.

[31] Mingxing Rao and Daniel Moyer. Generalization and memorization in rectified flow. *arXiv preprint arXiv:2603.13421*, 2026.

[32] Anthony Rhodes. Gaussian models & gaussian processes. *Lecture Notes, Portland State University*, n.d.

[33] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive auto-encoders: Explicit invariance during feature extraction. *International Conference on Machine Learning*, 2011.

[34] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. *arXiv preprint arXiv:1505.04597*, 2015.

[35] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *International Conference on Learning Representations*, 2021.

[36] George Stein, Jesse C. Cresswell, Rasa Hosseinzadeh, Yi Sui, Brendan Leigh Ross, Valentin Villecroze, Zhaoyan Liu, Anthony L. Caterini, J. Eric T. Taylor, and Gabriel Loaiza-Ganem. Exposing flaws of generative model evaluation metrics and their unfair treatment of diffusion models. *Advances in Neural Information Processing Systems*, 2023.

[37] Naftali Tishby, Fernando C. Pereira, and William Bialek. The information bottleneck method. *Proceedings of the 37th Annual Allerton Conference on Communication, Control, and Computing*, 1999.

[38] David Tse. Ee/stats 376a: Information theory, lecture 15. *Lecture Notes, Stanford University*, 2017.

[39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

[40] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. *Proceedings of the 25th International Conference on Machine Learning*, 2008.

[41] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

A Theoretical RF vs. ERF

The theoretical receptive field size of a deep CNN increases exponentially with amount of downsampling layers, since the jump (change in distance between two pixels in the feature map, measured on the input image) increases with every strided convolution. I use the "jump"-formulation of the formula from the "Single-path networks" section of [2]:

$$j_l = j_{l-1} \cdot s_l, \qquad r_l = r_{l-1} + (k_l - 1) \cdot j_{l-1}$$

Here l is the layer index, s is the stride, k is the kernel size, j is the "jump" and r is the theoretical receptive field size. The $(k_l - 1)$ term is the number of steps between kernel elements, and each step has length j_{l-1}. The jump j_l tells us how far two pixels in the feature map at layer l are apart in the input image.

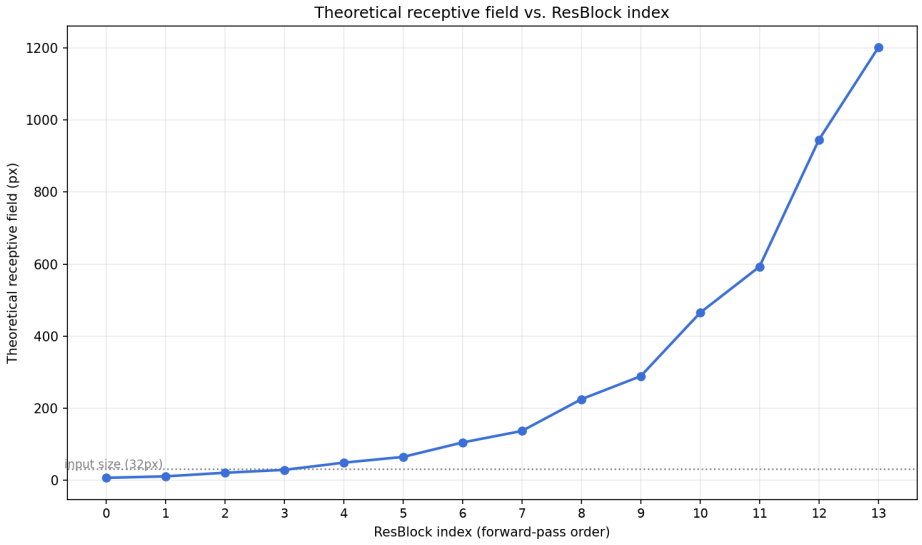


Figure 47: The diameter of the theoretical receptive field increases rapidly. This graph accounts for upsampling ($k = 3, s = 2$) and downsampling ($k = 4, s = 2$) operations in the U-Net's UpBlocks and DownBlocks. Each downsampling or upsampling operation doubles the jump J.

However, the theoretical receptive field size quickly loses its meaning as it becomes much larger than the input image. To obtain a more interpretable measure of the receptive field, I use the effective receptive field (ERF) method from [23] that calculates the receptive fields by measuring how much a pixel in the feature map is influenced by changes in the input. The implementation and functionality is detailed in Section 5.2.5.

B Hardware Optimizations

This section summarizes optimization techniques used to train large generative models on a single RTX 4060 NVIDIA GPU in a reasonable amount of training time.

1. **FlashAttention:**
   FlashAttention [9] is an attention [39] algorithm that reduces the number of memory reads/writes between GPU high bandwidth memory (HBM) and GPU on-chip SRAM and never materializes the full attention matrix. All attention blocks in this project's U-Nets used FlashAttention.

2. **Mixed Precision Training:**
   PyTorch autocasting to `bfloat16` "Brain Floating Point" was used for the forward pass and loss calculation in the training loop and ODE solver model calls. In this project, this was the single optimization that provided the most significant wall time per epoch decrease. Because `bfloat16` shares `FP32`'s exponent range, it avoids the overflow and underflow issues that other 16-bit float formats (`FP16`) experience and does not need gradient scaling. Its machine epsilon is $\epsilon_{\text{mach}} = 2^{-7} \approx 7.8 \times 10^{-3}$.

Spec	FP32	FP16	BF16
Exponent Bits	8	5	8
Precision Bits	23	10	7
Total Bits	32	16	16

Table 9: Bits arrangements for `FP32`, `FP16` and `BF16` formats.

3. **Fused AdamW Kernel:**
   The PyTorch fused AdamW kernel was used, this reduces the number of GPU-CPU synchronizations.

4. **Non-Blocking GPU Transfers:**
   Batches were transferred to the GPU device in non-blocking mode, meaning the CPU does not wait for the transfer to happen before parsing the next line.

The dataloader hyperparameters for batch size, number of workers and prefetch factor were chosen by testing a search-space of 64 combinations and picking the one that enables the highest images per second throughput. The best training throughput that I managed to get for 32×32 RGB images on a single RTX 4060 GPU was ≈ 1120 images/s. With a custom plotting tool, I confirm that the GPU usage is high, there is no CPU bottleneck and no RAM pressure.

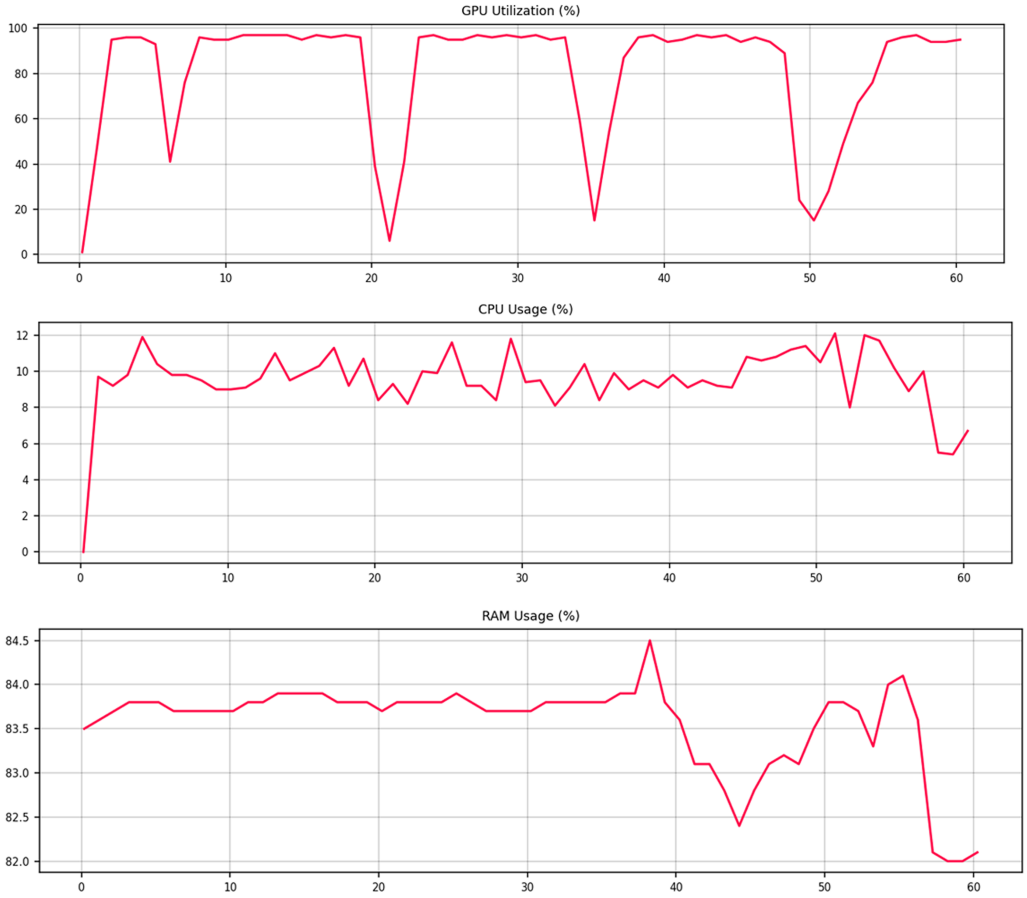


Figure 48: Screenshot from real-time training performance plots for a partial mini scale model training run, the first epoch's performance is skipped in the graph. Note that the drops in GPU usage I still see are due to the way PyTorch gets the GPU usage, the task manager shows me that the GPU is consistently at almost 100% usage.

C Derivations

C.1 The Gaussian Distribution maximizes Differential Entropy

In this work, the Gaussian distribution is often described as the probability distribution that maximizes differential entropy $h(x)$ under a fixed mean and standard deviation. Note that this is not a full derivation and only serves to give an idea on why the Gaussian PDF maximizes differential entropy. We will provide intuition on how this can be proven using Lagrange Multipliers [26] to find the maximum entropy function given the three following constraints:

$$\int_{-\infty}^{\infty} p(x)dx - 1 = 0 \qquad \text{(Normalization)}$$

$$\mathbb{E}[x] = \int_{-\infty}^{\infty} xp(x)dx - \mu = 0 \qquad \text{(Fixed Mean)}$$

$$\mathbb{E}[(x-\mu)^2] = \int_{-\infty}^{\infty} (x-\mu)^2\, p(x)\, dx - \sigma^2 = 0 \qquad \text{(Fixed Variance)}$$

The differential entropy of a continuous random variable X is defined as:

$$h(p) = \mathbb{E}_{x\sim X}[S(x)] = \mathbb{E}_{x\sim X}[-\ln p(x)] = -\int_{-\infty}^{\infty} p(x)\ln p(x)\, dx$$

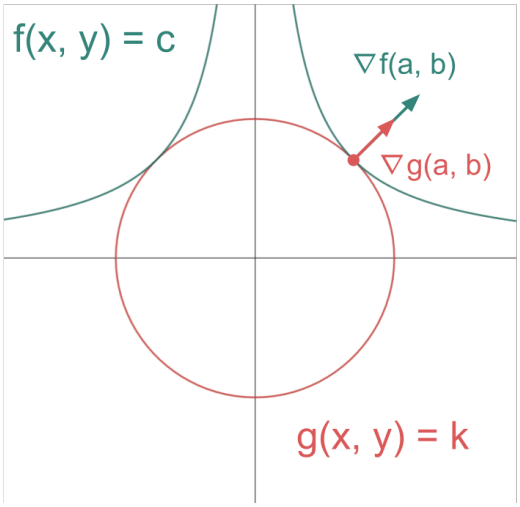


Figure 49: A contour line of a two dimensional multivariable scalar-valued function and a constraint curve, they are tangent. All outputs of f on the contour line are the same. The constraint function is also a scalar-valued multivariable function that is set equal to a constant. We can imagine this plot as a slice of a three dimensional "mountain". The **gradient is always orthogonal to a contour line.** Note that this illustrates the single-constraint case. The derivation, which uses multiple constraints generalizes parallel gradients to ∇f being in the span of the three constraint gradients.

First, we set up the Lagrangian [26]. At the constrained optimum, any direction that doesn't cause any first-order change to all three constraints must leave the objective's value unchanged. Otherwise the objective could still be improved while remaining feasible under the constraints. By requiring there to be a weighted sum of constraint gradients that is exactly the same as the gradient of the objective function at the point we are solving for, we guarantee that no change in $p(x)$ can achieve first-order change in the objective function (differential entropy). This is because the set of possible directions (those preserving all constraints) consists exactly of the directions orthogonal to every constraint gradient, or equivalently, tangent to the constraint contour lines (because we are not allowed to change the value of the constraint functions). So, requiring the objective's gradient to be orthogonal to all feasible directions (all directions orthogonal to the constraint gradients) ensures that no first-order change in f is possible while preserving the constraint values. In other words, ∇f lies inside the span of the constraint gradients, if ∇f had a nonzero component outside of the span of the constraint gradients, that component would itself be a direction where f could still change, contradicting optimality. To put it concisely, there is no feasible first-degree change in f that doesn't also change the outputs of the constraint functions. So, for ∇f to lie inside the span of the gradients: $\nabla f - \lambda_1\nabla g_1 - \lambda_2\nabla g_2 - \lambda_3\nabla g_3 = 0$. The Lagrangian whose gradient we want to set equal to 0 is the following:

$$
\begin{aligned}
L = \int_{-\infty}^{\infty} -p(x)\ln p(x)\,dx &- \lambda_1\left(\int_{-\infty}^{\infty} p(x)\,dx - 1\right) \\
&- \lambda_2\left(\int_{-\infty}^{\infty} xp(x)\,dx - \mu\right) \\
&- \lambda_3\left(\int_{-\infty}^{\infty} (x-\mu)^2 p(x)\,dx - \sigma^2\right)
\end{aligned} \tag{1}
$$

Since the unknown is an entire function $p(x)$ rather than a finite set of variables, this constrained optimization is solved using calculus of variations rather than ordinary Lagrange multipliers. The functional derivative with respect to $p(x)$ is set to zero [13]. We skip this technical step here.

$$
\boxed{p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)} \qquad \text{(PDF of } \mathcal{N}(\mu, \sigma^2)\text{)}
$$

For a derivation using Lagrange Multipliers, see [32]. For a full derivation using KL-Divergence, see [38].

C.2 The Noise-Prediction is proportional to the Conditional Score

I will show that the noise $\epsilon_\theta(x_t, t)$ is proportional to the **conditional** score $\nabla_{x_t} \log q(x_t \mid x_0)$.

Note that this derivation follows the standard DDPM notation [17], in which t increases from 0 (clean data) to T (pure noise) and x_0 is the clean sample. This is the reverse of the Flow Matching notation used elsewhere in this project, where t increases from 0 (noise) to 1 (data) and x_0 denotes a noise sample.

This section derives the score of the forward conditional, not the marginal score $\nabla_{x_t} \log p_t(x_t)$ as seen in [35]. However, it is still useful since it proves that the noise prediction is proportional to the gradient of $\log q(x_t \mid x_0)$, which points in the direction of fastest change in probability density for the current point x_t under the assumption that we want to arrive at a fixed x_0. Even though the MSE loss objective targets the conditional score rather than the intractable marginal score $\nabla_{x_t} \log p_t(x_t)$ directly, optimizing over all $x_0 \sim p_{\text{data}}$ makes the expectation of the conditional score equal to the true marginal score. The claim that training the neural network to predict the noise is equivalent to score matching since the noise prediction is a linear rescaling of the score function can be found in Appendix A of [12].

Recall the closed-form sampling for the DDPM forward process [17]:

$$
\begin{aligned}
x_t &= \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon\,, \quad \epsilon \sim \mathcal{N}(0, I) \\
\implies q(x_t \mid x_0) &= \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t} x_0, (1-\bar{\alpha}_t) I\right)
\end{aligned} \tag{1}
$$

Recall the Probability Density Function of a Gaussian Distribution:

$$
p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{2}
$$

Starting from (1) and using (2), we can compute the log-density:

$$
\log q(x_t \mid x_0) = \underbrace{\log(\dots)}_{\substack{\text{constant} \\ \text{w.r.t. } x_t}} - \frac{\left(x_t - \sqrt{\bar{\alpha}_t} x_0\right)^2}{2(1-\bar{\alpha}_t)}
$$

Now differentiate with respect to x_t:

$$
\nabla_{x_t} \log q(x_t \mid x_0) = -2\left(x_t - \sqrt{\bar{\alpha}_t} x_0\right) \cdot \frac{1}{2(1-\bar{\alpha}_t)} = -\frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{1-\bar{\alpha}_t}
$$

Again, the closed-form sampling for the DDPM forward process:

$$
x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon \iff x_t - \sqrt{\bar{\alpha}_t} x_0 = \sqrt{1-\bar{\alpha}_t}\epsilon
$$

Substitute this into the log-gradient above:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{\sqrt{1-\bar{\alpha}_t}\epsilon}{1-\bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}$$

$$\implies \epsilon = \underbrace{-\sqrt{1-\bar{\alpha}_t}}_{\text{constant}} \cdot \nabla_{x_t} \log q(x_t \mid x_0) \iff \epsilon \propto \nabla_{x_t} \log q(x_t \mid x_0)$$

What does this entail?
Recall the update step in the DDPM sampling algorithm:

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \quad z \sim \mathcal{N}(0, I)$$

During actual sampling, the network has no access to x_0. It only sees (x_t, t), so the noise $\epsilon_\theta(x_t, t)$ is proportional to the marginal score $\nabla_{x_t} \log p_t(x_t)$ per the citation above. With that substitution: μ_θ subtracts a term proportional to ϵ_θ, which subtracts a term proportional to $-\nabla_{x_t} \log p_t(x_t)$, i.e. it adds a step in the direction of the score. So each denoising step nudges x_t along the direction of steepest increase in log-density, drifting the sample toward higher-probability regions of the data distribution before the injected noise $\sigma_t z$ scatters it slightly to actually *sample* from the learned reverse conditional distribution $p_\theta(x_{t-1} \mid x_t)$, rather than simply following the score toward its highest-density regions.

C.3 FM and CFM Loss Gradient Equivalence

We will prove that the marginal and conditional flow matching losses provide identical gradients with respect to the model parameters θ. This is theorem 2 from the paper *Flow Matching For Generative Modelling* [22] and the original proof can be found its appendix A.

Our goal is to show that

$$\nabla_\theta \mathcal{L}_{FM} = \nabla_\theta \mathcal{L}_{CFM}$$

Where

$$\mathcal{L}_{FM} = \mathbb{E}_{t\sim\mathcal{U}[0,1],\, x\sim p_t(x)} \left[\|v_\theta(x,t) - u(x,t)\|^2 \right],$$

$$\mathcal{L}_{CFM} = \mathbb{E}_{t\sim\mathcal{U}[0,1],\, x_1\sim p_1(x_1),\, x\sim \boxed{p_t(x|x_1)}} \left[\|v_\theta(x,t) - u(x,t \mid x_1)\|^2 \right],$$

$$u(x,t) = \int u(x,t \mid x_1)\, p_t(x_1 \mid x)\, dx_1,$$

$$u(x,t \mid x_1) = \frac{d}{dt}\psi_t(x_0), \quad x_0 = \psi_t^{-1}(x),$$

$$p_t(x) = \int p_t(x|x_1)\, p_1(x_1)\, dx_1.$$

We only need to show $(-2ab)_{FM} = (-2ab)_{CFM}$ since $\|a-b\|^2 = a^2 - 2ab + b^2$ (with $a = v_\theta(x,t)$ and $b = u(x,t)$ resp. $u(x,t|x_1)$), the ground-truth VF term b^2 is independent of θ and the NN term a^2 is identical for FM and CFM (the a^2 terms agree because $\mathbb{E}_{x_1\sim p_1(x_1),\, x\sim p_t(x|x_1)}[f(x)] = \mathbb{E}_{x\sim p_t(x)}[f(x)]$ by the definition of $p_t(x)$ above). Note that the factor of -2 cancels.

$$\begin{aligned}
&\mathbb{E}_{t\sim\mathcal{U}[0,1],\, x\sim p_t(x)} \left[v_\theta(x,t) \cdot u(x,t) \right] \\
&\quad \stackrel{?}{=} \mathbb{E}_{t\sim\mathcal{U}[0,1],\, x_1\sim p_1(x_1),\, x\sim p_t(x|x_1)} \left[v_\theta(x,t) \cdot u(x,t|x_1) \right]
\end{aligned}$$

Start by expanding the marginal form:

$$\begin{aligned}
&\mathbb{E}_{t\sim\mathcal{U}[0,1],\, x\sim p_t(x)} \left[v_\theta(x,t) \cdot \int u(x,t \mid x_1)\, \boxed{p_t(x_1|x)}\, dx_1 \right] \\
&\quad = \iint \left(v_\theta(x,t) \cdot \int u(x,t \mid x_1)\, \boxed{p_t(x_1|x)}\, dx_1 \right) p_t(x)\, dx\, dt
\end{aligned} \tag{1}$$

Now we can apply Bayes' Rule:

$$\left.\begin{aligned} P(A|B) &= \frac{P(B|A)\cdot P(A)}{P(B)} \\ \Leftrightarrow P(A|B)\,P(B) &= P(A,B) = P(B|A)\,P(A) \end{aligned}\right\} \qquad p_t(x_1|x) = \frac{p_t(x|x_1)\cdot q(x_1)}{p_t(x)} \tag{2}$$

Now insert (2) in (1):

$$\ldots = \iiint v_\theta(x,t)\,u(x,t\,|\,x_1)\,\frac{p_t(x|x_1)\,q(x_1)}{\cancel{p_t(x)}}\,\cancel{p_t(x)}\,dx_1\,dx\,dt \tag{3}$$

Here we assume $p_t(x) > 0$ for all x and t, since we divide by $p_t(x)$ in (2) before it cancels in (3). This is also assumed in the FM paper [22].

Now,

$$\int(\ldots)\,q(x_1)\,dx_1 = \mathbb{E}_{x_1\sim q(x_1)}[\ldots] \qquad \int(\ldots)\,\rho(t)\,dt = \mathbb{E}_{t\sim\rho(t)}[\ldots] \quad (\text{where } \rho(t)=1 \;\; \forall\, t\in[0,1])$$

and

$$\int(\ldots)\,p_t(x|x_1)\,dx = \mathbb{E}_{x\sim p_t(x|x_1)}[\ldots]$$

Finally, (3) becomes:

$$\mathbb{E}_{x_1\sim q(x_1),\,t\sim\mathcal{U}[0,1],\,x\sim p_t(x|x_1)}\left[v_\theta(x,t)\cdot u(x,t\,|\,x_1)\right]$$

This is the ab term of $\mathcal{L}_{CFM}$. We have proven that the gradient-relevant parts of $\mathcal{L}_{FM}$ and $\mathcal{L}_{CFM}$ are identical, and the remaining terms do not depend on θ. Thus,

$$\boxed{\nabla_\theta\mathcal{L}_{FM} = \nabla_\theta\mathcal{L}_{CFM}}$$

D List of All Trained Models

The following mini-scale models were trained without any perturbations or penalties:

Base Models

Model	FID 50k
base seed 0	13.1149
base seed 1	13.4365
base seed 2	13.1811
base (summary)	13.2442 ± 0.1698

The following mini-scale models were trained with some type of input perturbation. FID-50k scores that lie within one standard deviation of the base model's mean result are *italic*.

IP Models

Model	IP Type	Noise Weight	IP scale γ_0	FID-50k
IP 0.1	Network Input	1	0.1	94.0929
IP* 0.1		$\text{std}(x)\gamma^*(t)$	0.1	*13.4081*
IP* 0.2		$\text{std}(x)\gamma^*(t)$	0.2	21.8208
b-IP* 0.025	Pre-Bottleneck	$\text{std}(h)\gamma^*(t)$	0.025	*13.1554*
b-IP* 0.05			0.05	12.8298
b-IP* 0.1			0.1	12.7322
b-IP* 0.2 seed 0			0.2	12.1542
b-IP* 0.2 seed 1			0.2	12.1543
b-IP* 0.2 seed 2			0.2	12.1174
b-IP* 0.2 (summary)			0.2	12.1410 ± 0.021
b-IP* 0.4			0.4	15.3890

These mini-scale models were trained with a KL-to-unit-Gaussian loss penalty on their post-bottleneck activations:

KL Models

Model	KL penalty scale β	relative loss scale p at initialization	gradient ratio r at initialization	FID-50k
KL 0.05	0.05	0.65	≈ 0.04	14.1872
KL 0.1	0.1	1.30	≈ 0.08	13.8314
KL 0.25	0.25	3.25	≈ 0.2	12.8780
KL 0.5 seed 0	0.5	6.50	≈ 0.4	12.4850
KL 0.5 seed 1	0.5	6.50	≈ 0.4	11.9143
KL 0.5 seed 2	0.5	6.50	≈ 0.4	12.7573
KL 0.5 (summary)	0.5	6.50	≈ 0.4	12.3855 ± 0.4302
KL 1	1.0	13.00	≈ 0.8	14.1124

E Hyperparameters

This is a list of training hyperparameters and relevant configuration choices for the training of mini-scale models. Note that models were trained with a constant learning rate of lr_max and did not decay to lr_min (see Section 5.5), only the base model was retrained with a cosine lr schedule (see 6.1). Also note that the KL models in Experiment 2 have an additional KL penalty loss term, not just MSE loss.

Parameter	Value
Data	
Dataset	CIFAR-10 (train, 50k images)
Resolution	32×32
Normalization	$[-1, 1]$
Number of classes	10 (+1 null token)
Batch size	96
Architecture	
Network	UNet
Base channels	64
Levels per side	3
Channels per level	$64 \to 128 \to 256$
Attention levels	lowest resolution only ($8 \times 8 \to 4 \times 4$)
Bottleneck	$2 \times$ (ResBlock + Attention), 256 channels
Attention heads	$\min(8, \text{dim}/32)$
Normalization layer	GroupNorm (up to 32 groups)
Activation	SiLU
Conditioning	AdaGN (scale and shift)
Time embedding	sinusoidal
Embedding dimension	256
Dropout	0.1
Parameters	10.85M

Parameter	Value
Objective	
Training objective	Conditional flow matching
Loss	MSE
p_0	$\mathcal{N}(0, I)$
Time sampling	logit-normal
Class-label dropout (CFG)	0.1
Optimization	
Optimizer	AdamW
Weight decay	0.01 (0 for norms, biases, embeddings)
Max learning rate	3×10^{-4}
Minimum learning rate	1×10^{-5}
LR schedule	constant (see 5.5)
Epochs	400
Gradient clipping (L2 norm)	1.0
EMA decay	0.9995
Precision	bfloat16 autocast
Evaluation and Sampling	
FID implementation	clean-FID
FID reference set	CIFAR-10 train
Evaluated weights	EMA
FID during training	25k samples, every 80 epochs
Final FID	50k samples
FID Sampler	Euler, 50 steps
FID CFG scale	2.0

Table 10: Full mini-scale model configuration

F Full Model Architecture Graph

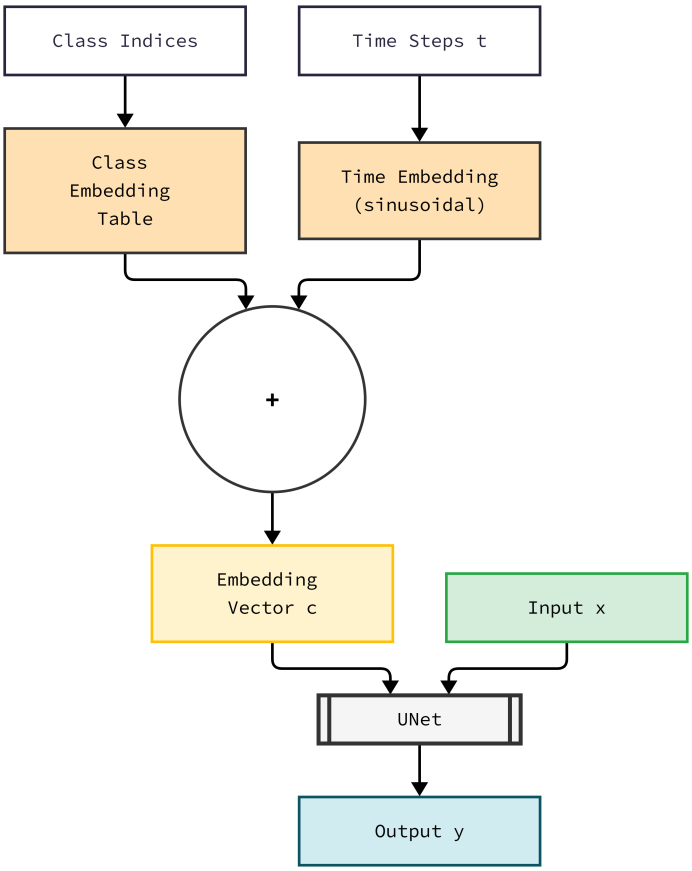


Figure 50: MermaidJS graph of the complete U-Net pipeline.

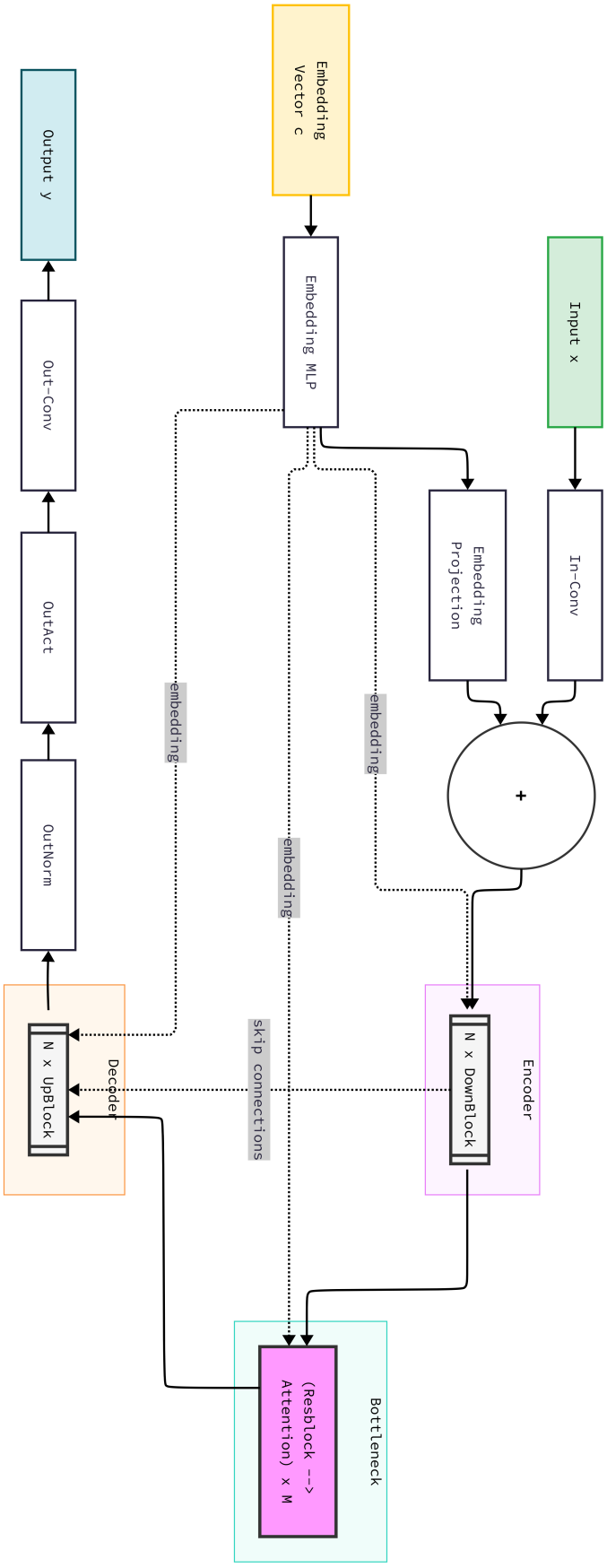


Figure 51: MermaidJS graph of the U-Net architecture.

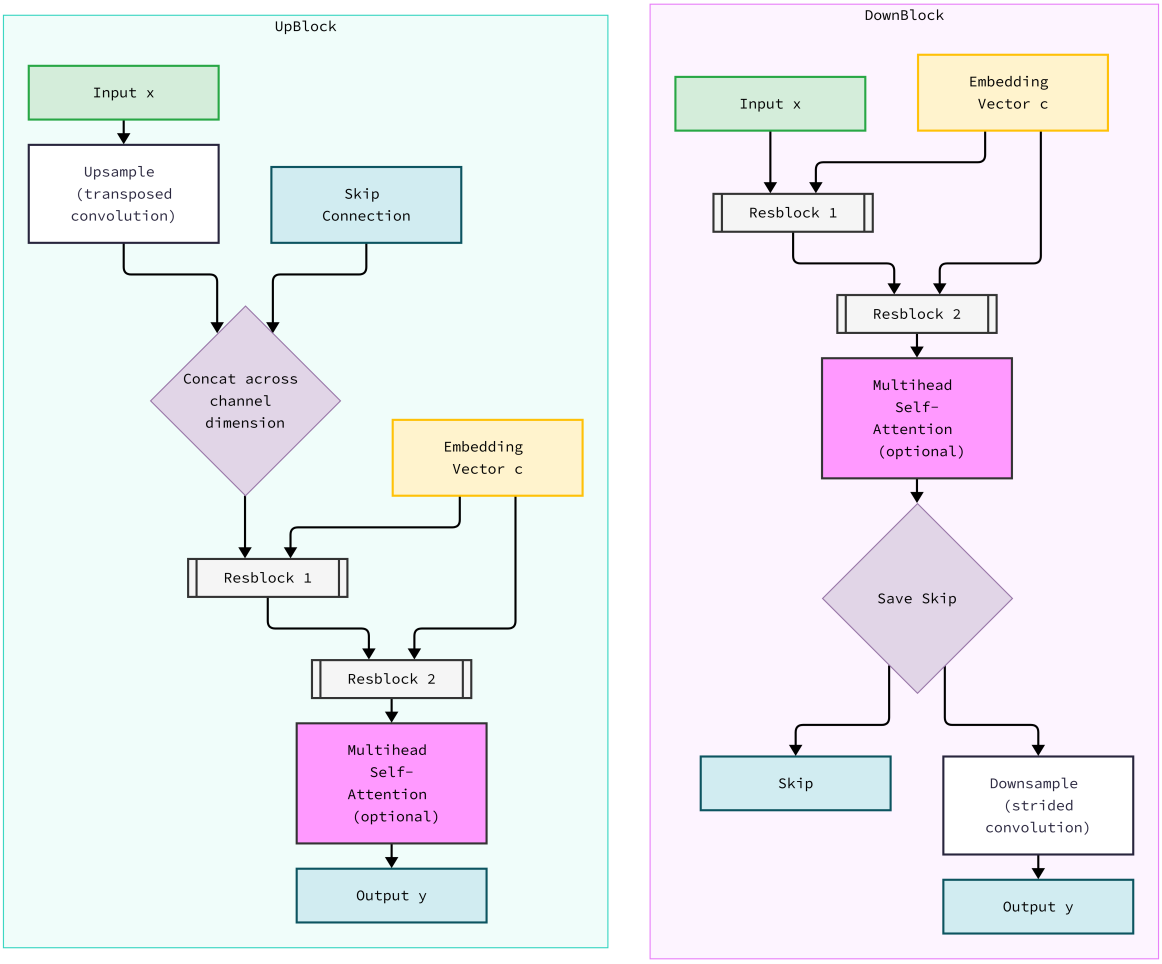


Figure 52: MermaidJS graph of the UpBlock and DownBlock modules. UpBlocks upsample the spatial resolution in the decoder, DownBlock downsamples the spatial resolution in the encoder.

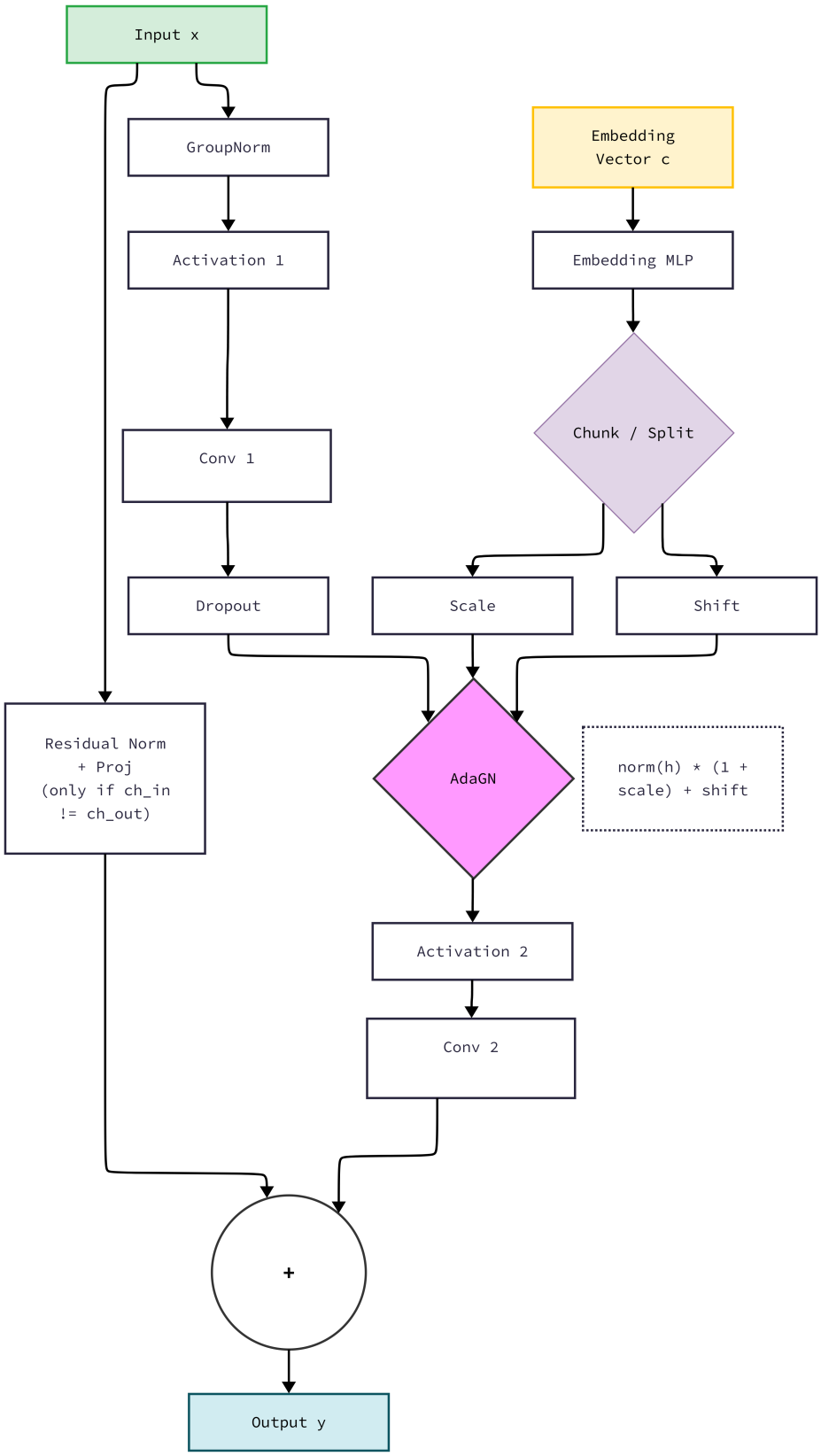


Figure 53: MermaidJS graph of the ResBlock module.

G Additional Samples

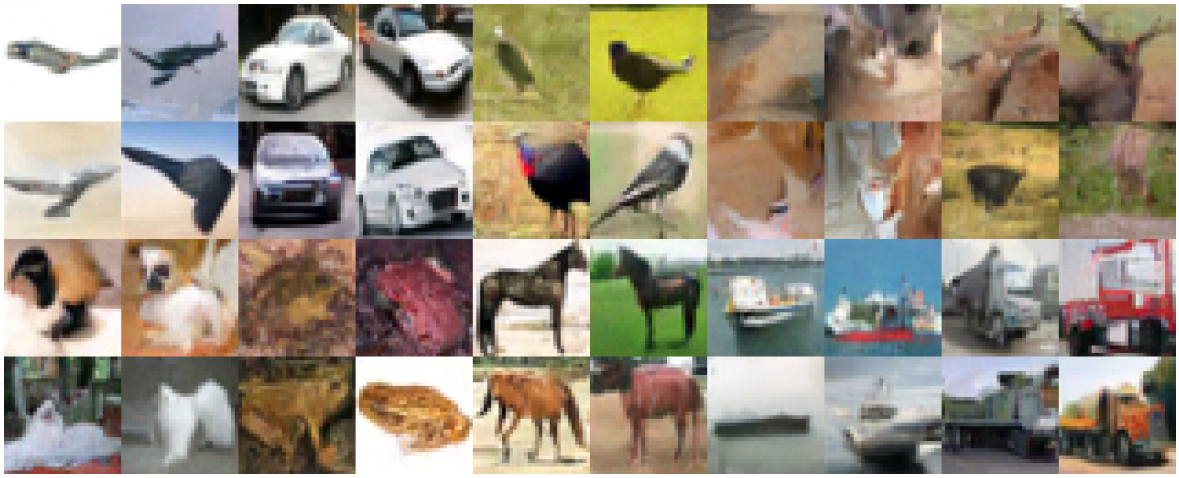


Figure 54: Samples from the lowest FID-50k seed of the base model (13.1149). Generated at a CFG scale of 2.25 and 100 steps of an Euler ODE solver.

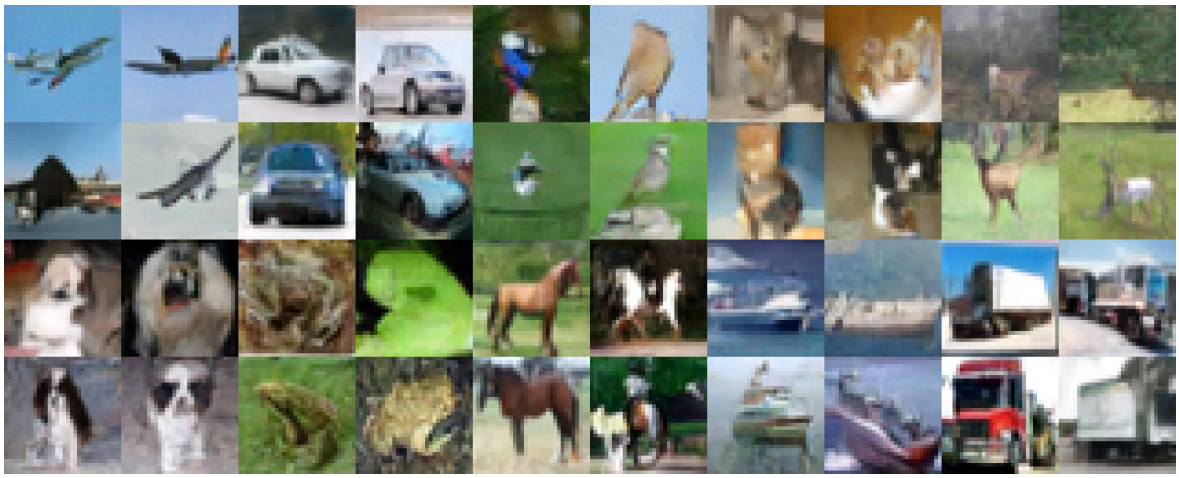


Figure 55: Samples from the lowest FID-50k seed of the b-IP*0.2 model (12.1174). Generated at a CFG scale of 2.25 and 100 steps of an Euler ODE solver.

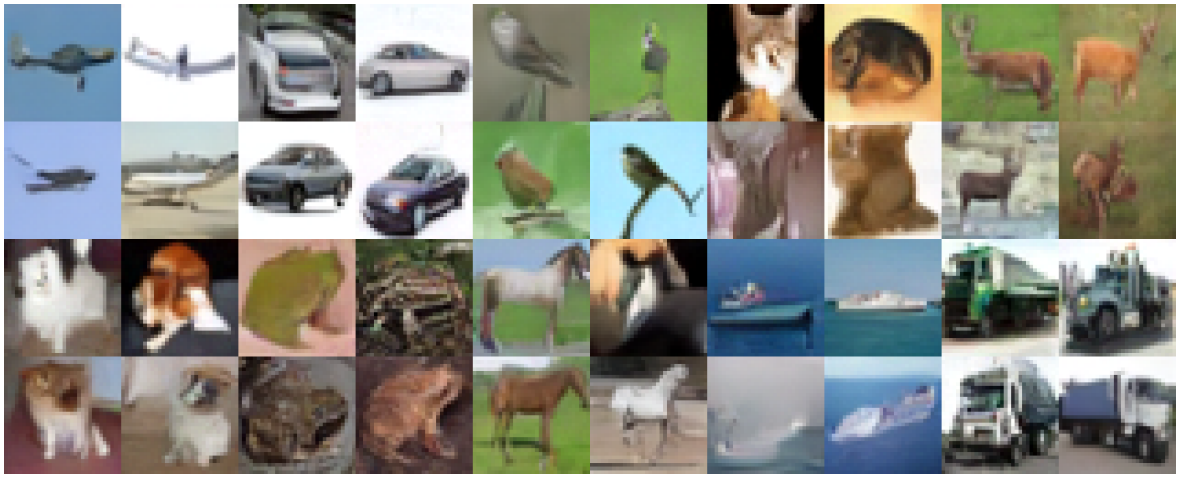


Figure 56: Samples from the lowest FID-50k seed of the KL 0.5 model (11.9143). Generated at a CFG scale of 2.25 and 100 steps of an Euler ODE solver.